

微网与外部电网电力调控策略

摘要

微网可在电价低谷充电、高峰放电,在满足小区负载的前提下降低购电费用。本文以混合整数线性规划为核心工具:先建立问题一的单日确定性购电计划模型,再针对问题二负载与光伏逐日变化、计划不足须按5倍电价紧急补购的情形,建立“预测—风险修正—日前优化—实时执行”的三层滚动模型。

问题一将全天离散为144个10分钟时段($\Delta t = 1/6$ h),以电量(kWh)为决策单位,变量含计划购电量、充放电量、弃光电量与时段末储电量,并以二元变量保证充放电互斥;目标为全天购电费最小。约束包括供需平衡、状态递推(单程效率0.90,往返效率0.81)、容量[1200,10800] kWh与日循环 $E_0 = E_{144} = 6000$ kWh。该MILP共864个变量,由HiGHS求得**全局最优解**;LP松弛的最优解中同时充放电时段数为零且目标值一致(差 7.3×10^{-12} 元),说明它本身已是原模型最优解。结果:全天计划购电量59482.70 kWh、购电费35126.95元(均价0.5905元/kWh),储能充放电量之比恰为往返效率0.81,储电量两端均被**紧约束**触及,**全天弃光电量为零**。灵敏度分析表明10分钟粒度较4小时粒度节省24.3%,本模型较“无储能”与“无光伏无储能”方案分别节省26.90%与60.23%。

问题二的电价仍逐日相同,负载与光伏改为附件2的全年实际值。**预测层**先识别出负载存在极强星期结构(周一至周四及周日约124000 kWh/日,周五、周六仅约78000 kWh/日),而光伏以季节性为主,故采用按周衰减的近期加权预测、负载只用同星期历史日,使净负荷预测的日绝对误差由19796.2降至3116.9 kWh/日。**计划层**注意到少买须付5倍罚款、多买不退款,由一阶条件可得最优计划量应取净负荷的80%分位数,遂用历史**预测误差**(而非拟合残差)的经验分位数构造风险净负荷 $\tilde{N} = \hat{N} + Q_\alpha$,求解含日末储备约束的日MILP,在0:00冻结全天144段计划购电量。**执行层**逐段观测实际值,按因果规则决定充放电、紧急购电与弃电,实际日末储电量传给次日,不要求日循环;参数由滚动窗口的实际总费用标定。

以2025年1月为预热期,正式区间为2月1日–12月31日共334天。计划购电量21701732.99 kWh、紧急购电量124045.19 kWh,**合计购电费14021565.12元**,其中紧急购电费662827.17元、仅占4.73%。8次滚动标定选出的 α 全部落在0.70–0.90(0.80出现6次),与**理论导出的80%分位数高度吻合**。相对本策略自身费用,较“预测均值策略”节省7.44%、较“无储能策略”节省32.34%,“事后理想”乐观下界为12082248元。信息时点、物理可行性、跨日连续性、状态累计、费用一致性与汇总一致性六项强制检查全部通过。

问题三在每天0:00、6:00、12:00、18:00各获得一次未来24小时的预报,且调整购电量按1.5倍电价收费。为消除“同一时段被多次修订如何结算”的歧义,本文严格区分三个量:**初始计划量** g_t^0 (0:00冻结、永久保存)、**最终生效量** x_t (完整量而非增量)与**紧急购电量** r_t ,实际收到电量为 $G^{\text{received}} = \sum x_t + \sum r_t$;并采用**交付块策略**,每个六小时块只在块首正式调整一次。附件3经对齐扫描判定为**整点瞬时功率**(对齐偏差191.51 kW,显著优于平移一小时与按小时平均),版本内作分段线性插值。由 $5p \Pr(N > x) > 1.5p$ 得临界分位数由0.8降至0.7;调减不退款时另有 $Q_{0.9}$,二者构成 $[Q_{0.7}, Q_{0.9}]$ 的“不必调整区间”。0:00计划沿用问题二的MILP,此后三个时点以**当前实际储电量**为初值滚动重排,仅提交当前块。四种预报的精度依次提高(全天电量偏差3903.9/2110.0/1197.7/461.3 kWh/日)。全年334天回放结果:**合计购电费13457697.61元**(计划12492137.14、调整675387.19、紧急290173.29),较仅用0:00预报的14207402.23元节省5.28%;其中73.2%来自状态刷新与滚动重优化、26.8%来自新发布的预报版本本身。全年七项强制检查全部通过,主口径下全年调减量恒为零。边际分析表明18:00与6:00的预报不可替代,而12:00在已有{6,18}时仅再省1.68万元(0.12%),可以省去。

关键字: 混合整数线性规划 微网调度 储能优化 峰谷套利 风险分位数 滚动决策 灵敏度分析

一、 问题重述

1.1 问题背景

微网是为解决光伏发电等分布式能源本地消纳、减少并网问题而快速发展起来的小型发用电系统。出于经济方面的考虑，非孤岛式微网可在外部电网（以下简称外网）电价较低或分布式能源有剩余电量的时间段为储能设备充电，并在外网电价较高的时间段释放电能。

微网与外网调控的目标是：利用小区负载、光伏发电量和储能设备当前储电量等数据，尽可能精准地给出微网的购电量和储能设备的充放电量，使微网提供的电力既满足小区负载，又尽可能节省购电费用。

1.2 问题提出

问题一：若每天的电价和小区负载相同，同时给出光伏发电功率一天的预测数据，要求微网提供的电能不低于小区负载，且储能设备在 0:00 和 24:00 的储电量相同。需要在每天 0:00 制定微网当天计划购电策略的数学模型；根据附件 1 和附录 1 的数据给出具体策略，并以规定格式给出指定时段的购电量、全天购电量与购电费，以及储能设备的充放电量与 0:00、24:00 储电量，完整策略保存至 `result1.xlsx`。

问题二：若每天的电价相同，而小区负载和光伏发电功率随时间变化，且微网提供的电能不可低于小区负载，如果低于负载需向外网紧急购电，其电价是交易时刻电价的 5 倍；除紧急购电费用外，其他时间段的购电费用均按计划购电量计算。需要在每天 0:00 制定当天的计划购电策略。根据附件 1 的电价和附件 2 的数据，给出表 3 中指定日期（2025.3.20、2025.6.21、2025.9.23、2025.12.21）的结果，并以规定格式给出紧急购电结果，完整策略保存至 `result2.xlsx`。

二、 问题分析

三问共享同一套物理模型（储能、平衡与效率约束），差别全部在**信息的可得性**与**计费规则**上。本节只辨认每问的结构性难点与应对路线，具体模型在第 5–7 节展开。

2.1 问题一分析

问题一的关键在于识别其**确定性**与**单日性**：各日运行条件完全相同，光伏又是给定的一日预测，故只需制定一个具有代表性的单日计划（附件 5 中 `result1.xlsx` 仅含 144 行，亦印证这一点）。

设定下微网面临的是**带储能的多时段能量调度问题**，节费来自两条相互关联的渠道：**峰谷套利**（低价多购存入储能、高价放电替代购电）与**光伏消纳**（把光伏盈余存入储能留待无光照时段）。二者都要求决策**跨时段协调**——某段多买一度电会改变后续所有时段的可行域，故不能逐段贪心，必须整体优化。数学结构上，目标与全部约束（平衡、递推、容量、互斥）都是线性的，除模式变量 $z_t \in \{0, 1\}$ 外是 LP；引入 z_t 后为 MILP，其解的最优性可由分支定界严格保证。

需要辨析的建模要点有四，详见第 5.2 节：(1) 以**显式弃光变量**把平衡写成等式，使富余光伏的去向可追踪；(2) 充电在平衡式中属于“消耗”，须与购电、光伏放在同一式联合考虑；(3) 日循环约束使单日模型可逐日重复而不漂移，并由递推式推出 $\sum_t d_t = \eta_c \eta_d \sum_t c_t$ 这一强校验关系；(4) 同一时段不得同时充放电，必须用二元变量强制互斥，否则在光伏免费富余或存在多重最优解时求解器可能返回违反物理约束的解。

2.2 问题二分析

问题二引入三处本质改变。

其一，信息结构由“完全已知”变为“逐日滚动”。负载与光伏改为每天都在变化的实际值，0:00 制定计划时当天尚未发生，因此必须引入预测环节，且预测、误差统计与参数选择都必须**严格遵守信息时点**。这是本题最容易在实现中被违反的地方：一旦某个统计量（如分位数）不小心用到了当天或未来的数据，算出的费用只是“虚假的优秀”。本文以结构隔离保证这一点——预测器只暴露 $i < n$ 的历史，分位数只取 $i < n$ 的误差样本。

其二，代价的不对称为风险定价提供了明确依据。多买 1 kWh 的确定成本是 p ，少买的期望成本是 $5p \Pr(N > g)$ 。令二者相等得 $F_N(g) = 0.8$ ：**最优计划量是净负荷分布的 80% 分位数**，该结论不依赖任何分布假设。更关键的是，它说明真正决定费用的是预测误差的**分位数结构**而非均值，因此本文不追求最小化 MAE，而在点预测上叠加历史误差的经验分位数作为风险余量。加入跨时段储能后 80% 不再具有全局最优性，故它只作为参数搜索的起点，最终由实际连续运行的总费用标定。

其三，跨日状态必须连续。问题二不再要求 $E_0 = E_{144}$ ，这不是简单的“放宽”，而**改变了问题结构**：电池成为真正的跨期资源。其后果有三：(1) 不能固定用 6000 kWh 作每日初值，必须传递当天**实际执行后**的日末电量；(2) 问题一的强校验关系不再成立，应改用状态累计关系 $\Delta E = \eta_c \sum_t c_t - \eta_d^{-1} \sum_t d_t$ ；(3) 单日模型倾向于日末放空电池，故可在目标中加入日末储备软惩罚 $\lambda \xi$ 抑制短视——标定结果为 $\lambda = 0$ ，即在该评价口径下数据认为不需要它，且无论取值如何该惩罚**只用于制定策略，不计入实际上报的账单**。

综合三点，问题二应建成“**预测—风险修正—日前优化—实时执行—跨日滚动**”的多层结构；整条链路的优劣只能由**按时间顺序的实际运行回放**评价，评价口径必须是包含 5 倍紧急购电费**的实际总费用**，而非预测误差。

2.3 问题三分析

问题三把信息结构与计费规则同时改动，二者的耦合是本题难点。

其一，信息从“一个点预测”变成“一串被反复修订的版本”。每天有 0:00、6:00、12:00、18:00 四个版本，各覆盖其后 24 个整点。这带来三个新要求：(1) 每条预报必须**同时携带“发布时间”与“目标时间”**——附件 3 的“预报 1 小时”指发布后 1 小时而非当天 1:00，混用会造成整体错位；(2) 首端没有 F_0 ，必须显式约定一个**不引入未来信息**的锚点；(3) 更晚发布的版本未必更准，其价值必须用实际运行费用衡量，不能由“更新”本身推出。

其二，计费规则的改变直接改写了风险定价的阈值。问题二中多买 1 kWh 的确定成本是 p 、少买的期望成本是 $5p \Pr(N > g)$ ，临界比给出 80% 分位数。问题三中调增 1 kWh 的确定成本是 $1.5p$ ，于是追加是否值得的条件变为 $5p \Pr(N > x) > 1.5p$ ，即

$$\Pr(N > x) > 0.3 \iff x^* = \max\{g^0, Q_{0.7}(N)\},$$

这种“由边际成本比给出最优分位数”的做法与库存管理中的临界比规则同源 [7]。反过来，若采用“取消部分退款”的解释，调减 1 kWh 净省 $0.5p$ 、代价是 $5p \Pr(N > x)$ ，得 $\Pr(N > x) = 0.1$ ，即**缺口概率低于 0.1 时才值得调减**。两种口径合起来给出 $[Q_{0.7}, Q_{0.9}]$ 的“**不必调整区间**”——这正是“有新预报不等于一定要改计划”的定量表述，第 7.8 节用八种预报组合的实际费用对它作了检验。该推导忽略储能与跨时段价值，只用于解释机制，实际参数仍由连续回放的**实际总费用**标定。

其三，决策由“一次冻结”变成“分区段依次正式提交”。问题三中 6:00、12:00、18:00 都可修改计划，于是必须回答：同一时段被修改多次按哪一次结算？本文采用**交付块策略**，让每个六小时交付块只在其块首正式调整一次（块范围见第 7.1 节）：每个十分钟时段只与自己的 g_t^0 结算一次，“最终净偏差结算一次”与“逐次变更累加”的歧义被彻底消除；更远时段的取值只作为**展望值**参与，不提交、不收费、不写入结果文件。

由此，问题三应建成**多次预报驱动的滚动购电与调整模型**。回答”是否需要其他时刻的预报”不能只看四个指定日期，必须把 8 种预报组合逐一完整回放全年，并另设一个在**同一时刻重新优化、但仍沿用 0:00 预报版本**的对照，以分离”新信息带来的收益”与”重新优化带来的收益”。

三、 模型假设

1. **数据准确假设**：附件 1 的电价、小区负载与光伏功率准确反映实际，不计预测误差（其影响留待问题三）。
2. **分段恒定假设**：每个 10 分钟时段内各功率恒定，时段电量等于功率乘以 $\Delta t = 1/6 \text{ h}$ ；供电保障对应这一离散模型。
3. **母线侧计量假设**： c_t 为微网送入储能的电量、 d_t 为储能向微网交付的电量，均按**微网母线侧**口径计量，额定功率同侧解释。
4. **储能理想假设**：单程效率恒为 $\eta_c = \eta_d = 0.90$ （往返 0.81），不随功率、储电量、温度或设备老化变化，无自放电与漏电损失。
5. **单向购电假设**：题目未给出售电价格， $g_t \geq 0$ ，不计余电上网收益。
6. **弃光可行假设**：光伏出力超过负载且储能无法完全吸纳时允许弃光， $0 \leq s_t \leq v_t$ ，富余光伏总有确定去向。
7. **不外加上网限值假设**：附录中 5000 kW 限制的是**储能设备**的充放电功率，本文不另设微网购电功率上限。
8. **无紧急购电假设**：问题一中计划购电得到完全执行，不触发因计划不足而生的紧急购电（该情形在问题二中讨论）。
9. **日循环假设**：储能每日回到当日 0:00 的水平，即 $E_{144} = E_0$ ，该计划可逐日重复执行。
10. **忽略衰减假设**：不计容量衰减与寿命损耗成本。

问题二改用以下假设（与问题一冲突处以本节为准）：

11. **电价逐日重复假设**：附件 1 的日内电价在 2025 年每天重复生效，不使用附件 4 的波动电价（留待问题四）。
12. **实际值可观测假设**：当天运行时，各时段的实际负载与光伏在该时段结束时可被观测并用于该时段记账；执行层以 10 分钟为记账步长。
13. **预测信息受限假设**：0:00 制定计划时只能使用当天之前的负载与光伏实际值、已知的日内电价、日期与星期，以及当前实际储电量；当天实际曲线与附件 3 的外部光伏预报均不可使用。
14. **计划购电量冻结假设**： $g_{n,t}$ 一经 0:00 确定便不再修改，不足部分只能按 5 倍电价紧急补购，无计划调整交易（留待问题三）。
15. **紧急购电不退款假设**：已按计划购入的电量无论用完与否均全额收费，用不完的部分可转为充电，其余按无收益弃用处理，即**允许弃用已购电**。
16. **储备惩罚不计入账单假设**：软惩罚项 $\lambda \xi_n$ 只用于制定策略，绝不进入实际上报的购电费。

四、 符号说明

符号	含义	单位
t	时段编号, $t = 1, 2, \dots, T$	—

符号	含义	单位
T	一天划分的时段总数, $T = 144$	个
Δt	单个时段时长, $\Delta t = 10/60 = 1/6$	h
p_t	第 t 时段的外网电价 (附件 1)	元/kWh
L_t	第 t 时段的小区负载功率 (附件 1)	kW
V_t	第 t 时段的光伏发电预测功率 (附件 1)	kW
ℓ_t	第 t 时段的负载电量, $\ell_t = L_t \Delta t$	kWh
v_t	第 t 时段的光伏发电量, $v_t = V_t \Delta t$	kWh
g_t	第 t 时段从外网购入的电量 (决策变量)	kWh
c_t	第 t 时段向储能设备输入的充电量 (决策变量)	kWh
d_t	第 t 时段储能设备向微网输出的放电量 (决策变量)	kWh
s_t	第 t 时段弃用的光伏电量 (决策变量)	kWh
E_t	第 t 时段末的储电量 (决策变量), $t = 0, \dots, T$	kWh
z_t	充放电模式变量, $z_t \in \{0, 1\}$ (决策变量)	—
E^{init}	0:00 时刻的初始储电量, 取 6000	kWh
M	单时段最大充电量或放电量, $M = 5000\Delta t = 2500/3$	kWh
η_c, η_d	储能充电效率、放电效率, 均为 0.90	—
$P_{\max}$	储能最大充放电功率, $P_{\max} = 5000$	kW
$E_{\min}, E_{\max}$	储电量运行下界与上界, 1200 与 10800	kWh
J	全天购电费用 (目标函数)	元

问题二新增或改变含义的符号如下 (其余沿用上表; 带横线的变量表示**日前参考计划**, 不带横线的为**实际执行值**):

符号	含义	单位
n	日期编号, $n = 0, 1, \dots, 364$ ($n = 0$ 为 2025-01-01)	日
$L_{n,t}, V_{n,t}$	第 n 天第 t 时段的实际负载、光伏功率 (附件 2)	kW
$\ell_{n,t}, v_{n,t}$	第 n 天第 t 时段的实际负载电量、光伏发电量	kWh
$\hat{\ell}_{n,t}, \hat{v}_{n,t}$	由 n 之前历史得到的负载、光伏预测电量	kWh
$\hat{N}_{n,t}$	净负荷预测, $\hat{N}_{n,t} = \hat{\ell}_{n,t} - \hat{v}_{n,t}$	kWh
$\varepsilon_{n,t}$	净负荷预测误差, $\varepsilon_{n,t} = (\ell_{n,t} - v_{n,t}) - \hat{N}_{n,t}$	kWh
$Q_{\alpha,n,t}$	此前误差在对应时段的经验 α 分位数	kWh
$\tilde{N}_{n,t}$	风险修正净负荷, $\tilde{N}_{n,t} = \hat{N}_{n,t} + Q_{\alpha,n,t}$	kWh
α	风险分位数 (待标定)	—
$\bar{E}_{n,t}$	日前参考储电量	kWh
$\bar{c}_{n,t}, \bar{d}_{n,t}, \bar{w}_{n,t}$	日前参考充电量、放电量、剩余电量	kWh
E^{tar}	希望为次日保留的参考储电量	kWh
λ	日末储备不足的规划代价 (待标定)	元/kWh
ξ_n	日末储备不足量, $\xi_n \geq E^{\text{tar}} - \bar{E}_{n,144}$	kWh
$b_{n,t}$	使用储能前的电量盈余, $b_{n,t} = g_{n,t} + v_{n,t} - \ell_{n,t}$	kWh
$R_{n,t}$	储备线, $R_{n,t} = E_{\min} + \rho(\bar{E}_{n,t} - E_{\min})$	kWh
ρ	储备线系数, $0 \leq \rho \leq 1$ (待标定)	—
$c_{n,t}, d_{n,t}$	实际充电量、实际放电量	kWh
$r_{n,t}$	紧急购电量 (按 $5p_t$ 计价)	kWh
$w_{n,t}$	实际弃用电量 (不退款)	kWh
$E_{n,t}$	实际储电量, $t = 0, \dots, 144$	kWh
J_n^{plan}	全天计划购电费, $\sum_t p_t g_{n,t}$	元
J_n^{emg}	全天紧急购电费, $\sum_t 5p_t r_{n,t}$	元
$\mathcal{H}_n$	第 n 天预测所用的历史日集合, $\mathcal{H}_n \subseteq \{i : i < n\}$	—

五、 问题一：单日计划购电优化模型

5.1 数据预处理与时段划分

附件 1 共给出 144 组数据, 涵盖一天 24 小时, 故将一天等分为 $T = 144$ 个长度为 $\Delta t = 10 \text{ min} = 1/6 \text{ h}$ 的时段。

需要特别说明时段标签的对应关系: 附件 1 的时间列为区间**结束时刻** (首行 0:10、末行 0:00+1), 其中 0:00+1 按题目附录 2 的规定表示当天 24:00。因此附件 1 的第 t 行

对应时段 $[(t-1)\Delta t, t\Delta t)$ ，即第 t 时段为 $(t-1) \times 10$ 分钟至 $t \times 10$ 分钟，该行的电价与功率作为这一区间的代表值。

这一理解与“储能设备在 0:00 和 24:00 的储电量相同”的日循环约束相互印证：只有末行的 0:00+1 代表 24:00，24:00 才是可约束的时刻。需要指出，附件 5 中 `result1.xlsx` 的“计划购电量”表首段标签为 0:10–0:20、末段为 0:00+1–0:10+1，其末段已越过当天，与正文要求的 0:00–24:00 口径整体错位一格。本文统一采用正文口径，在输出副本中将购电时间标签改写为 0:00–0:10 至 23:50–0:00+1，行序与模板一一对应，原始模板文件保持不变。若后续有题目勘误或补充说明，应据此调整。

单位换算：附件 1 中电价为元/kWh，负载与光伏功率为 kW。为便于与题目结果表逐项对应，本文不直接以功率作决策变量，而是先把负载与光伏换算为时段电量，再以**电量**为决策变量单位：

$$\ell_t = L_t \Delta t, \quad v_t = V_t \Delta t. \quad (1)$$

图 1a 给出附件 1 三类输入数据的全天变化特征：电价呈明显的双峰结构，早高峰位于 7:50–9:10（峰值 1.1683 元/kWh）、晚高峰位于 17:40–20:50，最高电价 1.3952 元/kWh（出现在 20:30–20:40），而夜间低谷约 0.42 元/kWh、全天最低 0.3713 元/kWh（出现在 5:30–5:40），峰谷价差达 3.76 倍，为峰谷套利提供了充足空间；小区负载在 8:00–18:00 维持在 4208 kW 以上、峰值 5958.97 kW（出现在 9:00–9:10），夜间回落至约 3400 kW；光伏出力呈标准钟形分布，4:30 前后开始发电，正午达到峰值 7612.32 kW，19:20 后归零，全天有 55 个时段出力为零。三者共同决定了优化问题的基本格局。

需要强调，题目所说的“每天的电价相同”指的是**每天重复同一条电价曲线**，而非一天内电价恒定。附件 1 的电价每 10 分钟都有具体数值，故本文不作峰谷电价粗化，直接在 10 分钟粒度上建模。

5.2 模型建立

5.2.1 决策变量

对每个时段 $t = 1, 2, \dots, 144$ ，定义如下决策变量（前四个以电量 kWh 计）：

- $g_t \geq 0$ ：从外网购入的电量（kWh）；
- $c_t \geq 0$ ：向储能设备输入的充电量（kWh）；
- $d_t \geq 0$ ：储能设备向微网输出的放电量（kWh）；
- $s_t \geq 0$ ：弃用的光伏电量（kWh）；
- E_t ：第 t 时段末的储电量（kWh）， $t = 0, 1, \dots, 144$ ；
- $z_t \in \{0, 1\}$ ：充放电模式变量， $z_t = 1$ 表示该时段允许充电、 $z_t = 0$ 表示允许放电。

注意 E_0 是 0:00 电量、 E_{144} 是 24:00 电量，因此储电量序列共有 **145** 个状态点。

5.2.2 目标函数：全天购电费最小

微网的购电费用仅来自从外网购入的电量，故目标为最小化全天购电费：

$$\min J = \sum_{t=1}^T p_t g_t. \quad (2)$$

由于 g_t 已经是电量（kWh），此处不再乘 Δt 。应注意目标是购电**费用**最低而非购电**总量**最低：低价充电带来的能量损耗可能使全天购电量上升，但仍能降低总费用。

5.2.3 约束条件

(1) 供需平衡约束（等式加显式弃光）

微网在第 t 时段可用电量由外网购电、扣除弃光后的光伏出力与储能放电三部分构成，须恰好等于小区负载耗电与储能充电之和：

$$g_t + (v_t - s_t) + d_t = \ell_t + c_t, \quad t = 1, 2, \dots, T \quad (3)$$

(2) 光伏与购电的非负及弃光上界

$$g_t \geq 0, \quad 0 \leq s_t \leq v_t, \quad t = 1, 2, \dots, T \quad (4)$$

式 (3)–(4) 完整满足负载，并通过显式弃光变量处理多余光伏。相比把供电条件写成不等式 $g_t + v_t + d_t - c_t \geq \ell_t$ ，等式加弃光变量的写法可以直接追踪富余电量的去向，更方便解释和检查。 s_t 的上界取 v_t 表明弃光量不可能超过光伏发电量本身。

(3) 储能电量递推约束

第 t 时段末的储电量由上一时段末的储电量、本时段的充电量与放电量决定：

$$E_t = E_{t-1} + \eta_c c_t - \frac{d_t}{\eta_d}, \quad t = 1, 2, \dots, T \quad (5)$$

其中 $\eta_c = \eta_d = 0.90$ 。这里不能把充电项与放电项都乘以 0.90——两者都是母线侧电量，但一个流入电池、一个流出电池：充入 1 kWh 电量，电池内部增加 0.9 kWh；向微网放出 1 kWh 电量，电池内部需减少 1/0.9 kWh。该设定对应储能往返效率 $\eta_c \eta_d = 0.81$ 。当 $t = 1$ 时 $E_{t-1} = E_0$ ，即 0:00 的初始储电量。

(4) 储电量运行区间约束

为避免过度充电或放电，储电量须全程保持在给定区间内，145 个状态点逐一受限：

$$E_{\min} \leq E_t \leq E_{\max}, \quad t = 0, 1, \dots, T \quad (6)$$

其中 $E_{\min} = 1200$ kWh, $E_{\max} = 10800$ kWh。

(5) 功率上限与充放电互斥约束

受逆变器与电池倍率限制，单个 10 分钟内最大充电量与最大放电量均为

$$M = P_{\max} \Delta t = 5000 \times \frac{1}{6} = \frac{2500}{3} \text{ kWh}, \quad (7)$$

并利用二元变量约束同一时段不能同时充放电：

$$0 \leq c_t \leq M z_t, \quad 0 \leq d_t \leq M(1 - z_t), \quad z_t \in \{0, 1\}, \quad t = 1, 2, \dots, T \quad (8)$$

$z_t = 1$ 时允许充电、放电量被压为零； $z_t = 0$ 时允许放电、充电量被压为零；两种模式下都可以令充放电量同时取零，表示待机。

(6) 日循环约束

题目要求储能设备在 0:00 与 24:00 的储电量相同，结合附录 1 给出的 2025 年 1 月 1 日 0:00 电量为 6000 kWh，有：

$$E_0 = E^{\text{init}} = 6000, \quad E_T = E_0 \quad (9)$$

该约束保证计划可逐日重复执行：若 24:00 储电量与 0:00 不同，则次日初始条件将发生漂移，单日模型无法描述长期运行。同时它也使得模型无法通过额外消耗初始库存来获取不公平的费用优势。

由状态递推式 (5) 对全天求和，结合日循环条件 (9)，可推出一个很有用的校验公式：

$$\eta_c \sum_{t=1}^T c_t = \frac{1}{\eta_d} \sum_{t=1}^T d_t \implies \sum_{t=1}^T d_t = \eta_c \eta_d \sum_{t=1}^T c_t = 0.81 \sum_{t=1}^T c_t. \quad (10)$$

因此，日初日末储电量相同并不意味着表中的全天充电量等于全天放电量，二者之比应恰为往返效率 0.81。

5.2.4 完整模型

综合以上各式，问题一的数学模型为：

$$\begin{aligned} \min_{g, c, d, s, E, z} \quad & J = \sum_{t=1}^{144} p_t g_t \\ \text{s.t.} \quad & g_t + (v_t - s_t) + d_t = \ell_t + c_t, \quad t = 1, \dots, 144 \\ & E_t = E_{t-1} + \eta_c c_t - \frac{d_t}{\eta_d}, \quad t = 1, \dots, 144 \\ & E_{\min} \leq E_t \leq E_{\max}, \quad t = 0, \dots, 144 \\ & 0 \leq c_t \leq M z_t, \quad 0 \leq d_t \leq M(1 - z_t), \quad t = 1, \dots, 144 \\ & 0 \leq s_t \leq v_t, \quad g_t \geq 0, \quad t = 1, \dots, 144 \\ & z_t \in \{0, 1\}, \quad t = 1, \dots, 144 \\ & E_0 = E_{144} = 6000 \end{aligned} \quad (11)$$

5.2.5 模型类别：MILP 及其 LP 松弛

模型 (11) 中，目标函数与全部约束关于决策变量均为线性，唯一的非线性来源是 $z_t \in \{0, 1\}$ 这一整数要求，因此这是一个 **混合整数线性规划 (MILP)**，可以直接表达充放电互斥，适合作问题一的完整主模型。

按变量排列 $x = [g_{1:T}, c_{1:T}, d_{1:T}, s_{1:T}, E_{1:T}]^T \in \mathbb{R}^{5T}$ ，并记 $z = [z_1, \dots, z_T]^T$ ，模型 (11) 可写成如下标准形式：

$$\min_{x, z} \bar{c}^T x \quad \text{s.t.} \quad A_{\text{eq}} \begin{bmatrix} x \\ z \end{bmatrix} = b_{\text{eq}}, \quad A_{\text{ub}} \begin{bmatrix} x \\ z \end{bmatrix} \leq b_{\text{ub}}, \quad l \leq x \leq u, \quad z \in \{0, 1\}^T \quad (12)$$

其中目标系数 $\bar{c}$ 仅在购电变量位置取 p_t ；等式约束 A_{eq} （共 $144 + 144 + 1 = 289$ 条）对应供需平衡 (3)、状态递推 (5) 与日循环条件 (9)；不等式约束 A_{ub} （共 288 条）对应互斥约束 (8)；边界 l, u 对应 (4) 与 (6)。完整模型共 $5 \times 144 + 144 = 864$ 个决策变量、289 条等式约束与 288 条不等式约束，且所有非零系数均取自 $\{0, \pm 1, \pm \eta_c, \pm 1/\eta_d\}$ ，不含任何非线性项。

LP 松弛的作用。去掉整数要求 $z_t \in \{0, 1\}$ （即令 $0 \leq z_t \leq 1$ ）、仅保留 $0 \leq c_t, d_t \leq M$ ，模型退化为一个普通的 LP。该 LP 的最优值是 MILP 最优值的下界，可用于核对数据、检查约束并提供费用下界的参照。更重要的是：**如果 LP 最优解本身不含同时充放电，则它满足完整模型，便已经是 MILP 的最优解。**这一判定由程序自动完成——先求解 LP，统计 c_t^* 与 d_t^* 同时为正的时段数，再求解 MILP 与之比较。需要强调的是，**不能仅凭电价为正就假定求解器一定不会返回同时充放电**：存在免费光伏富余、多重最优解或数值退化时，LP 松弛仍可能给出物理上不可执行的解。因此本文不依赖这一先验假设，而是以数值判定为准。

本模型在结构上还是一个**网络流型线性规划**：储能可视为一个连接各时段的“水库”， E_t 是水位， c_t 与 d_t 分别是入库与出库流量。求解器采用 HiGHS，其 LP 部分以并行对偶修正单纯形法为核心，MILP 部分以分支定界配合割平面求解，问题规模 (864 变

量)在该求解器下属小规模问题。

5.3 模型求解

5.3.1 求解方法与算法

本文使用 Python 科学计算栈中的 `scipy.optimize.milp` 接口调用 **HiGHS** 求解器(当前主流的开源 LP/MILP 求解器),以稀疏矩阵存储约束系数,对 LP 松弛与完整 MILP 分别求解。求解参数为默认的 10^{-7} 量级可行性容差。

求解流程为:

1. 读取附件 1 的电价、负载与光伏三类序列,校验时段标签严格递增、等间隔且恰好覆盖当天 144 个时段,确保时段口径成立;
2. 按式 (1) 将功率换算为时段电量,整理容量、功率、效率与日初电量等参数;
3. 按 (12) 装配目标向量、约束矩阵与变量边界,分别构造 LP 松弛与完整 MILP;
4. 调用 HiGHS 求解,记录求解状态、目标值与最优性间隙;
5. 对解进行独立复核(供需平衡、状态递推、物理边界、日循环、充放电互斥、费用复算),并判定 LP 松弛解是否已是 MILP 最优解;
6. 由通过校验的解汇总论文表格、绘制策略图,并按统一时间标签写出 `result1.xlsx`。

5.3.2 求解结果与 LP/MILP 一致性

LP 松弛与完整 MILP 的求解结果如下:

- **LP 松弛**: 最优费用 35126.9486 元;最优解中 c_t^* 与 d_t^* 同时为正的时段数为 0;
- **完整 MILP**: 最优费用 35126.9486 元,最优性间隙 0 (已证明全局最优);最优解同样不含同时充放电。

两模型的目标值相差 7.3×10^{-12} 元,仅为浮点舍入量级。由于 LP 松弛的最优解本身不含同时充放电,它自动满足互斥约束 (8),因而也是完整模型 (11) 的最优解。**该结论由数值判定独立得到,未使用“电价为正”这一先验假设。**本文后续所有结果均取自这一(同时是 LP 与 MILP 的)最优解。

5.3.3 求解结果的校验

为确保结果可信,对最优解独立复核了以下各项, **全部通过**:

1. **时段完整性**: 144 个时段完整覆盖当天 0:00–24:00,首尾与四小时分组均无错位;附件 1 时间标签经校验为 $\{10, 20, \dots, 1440\}$ 分钟且严格递增。
2. **逐段供需平衡**: $\max_t |g_t + v_t - s_t + d_t - \ell_t - c_t| < 10^{-6} \text{ kWh}$ 。
3. **状态递推**: 由 c^*, d^* 从 E_0 出发独立累计递推得到的储电量序列与求解器返回的 E^* 最大偏差小于 10^{-6} kWh 。
4. **物理边界**: E^* 全程落在 $[1200, 10800] \text{ kWh}$ 内; $c_t^*, d_t^* \leq M = 2500/3 \text{ kWh}$; $0 \leq s_t^* \leq v_t$; 且不存在同时充放电的时段。
5. **日循环**: $E_{144}^* = E_0^* = 6000 \text{ kWh}$,且按式 (10) 校验 $\sum_t d_t^* = 0.81 \sum_t c_t^*$ (实测 $16799.94/20740.67 = 0.810000$)。
6. **费用与汇总**: 用未四舍五入的数据重新计算 $\sum_t p_t g_t^*$,与求解器返回值一致;各分段汇总与全天明细一致。

数值复核仅设置极小的浮点容差,未通过删除负数、裁剪越界值或修改末段结果的方式掩盖任何约束违背。

5.4 求解结果

5.4.1 表 1：微网在指定时间段的购电量及全天的购电量和购电费

表 1 微网在指定时间段的购电量及全天的购电量和购电费

时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
10:00-10:10	0.00	12:00-12:10	480.41	14:00-14:10	0.00
16:00-16:10	445.43	18:00-18:10	531.89	20:00-20:10	0.00
全天购电量/kWh				59482.70	
全天购电费/元				35126.95	

5.4.2 表 2：储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量

表 2 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量

时间段	充电量/kWh	放电量/kWh	时间段	充电量/kWh	放电量/kWh
0:00-4:00	4500.00	0.00	4:00-8:00	833.33	6365.84
8:00-12:00	4787.96	1703.00	12:00-16:00	5286.04	91.10
16:00-20:00	0.00	5780.13	20:00-24:00	5333.33	2859.87
0:00 储电量/kWh				6000.00	
24:00 储电量/kWh				6000.00	

表 2 中每段的充电量与放电量**分别求和**，未用正负相抵后的净电量代替。若需另报电池内部的能量变化，对应为 $\eta_c C_k$ 与 D_k/η_d ，应与本文的母线侧报表口径明确区分。

完整的计划购电策略（144 个 10 分钟时段的逐时段购电量，以及各时段充放电量与储电量）已按要求写入文件 `result1.xlsx`。

5.5 结果分析

5.5.1 购电策略的时序特征

图 1b 给出全天逐时段的计划购电量与电价的对照关系。可以清晰地看到购电行为被**严格地**压缩到了低电价时段：

- 在 0:00–6:00 的夜间低谷（电价约 0.37–0.91 元/kWh），购电量维持在每 10 分钟 1400 kWh 上下，其中 0:00–0:40 因需同时供给负载并为储能充电而达到全天最大的 1406.6–1409.8 kWh/10 min；
- 在 5:30–5:40 电价降至全天最低（0.3713 元/kWh）时，购电量为 1383.66 kWh/10 min，仍处高位；此后随储能充满（5:40 触及上限 10800 kWh）迅速退出购电，5:50 起连续 31 个时段购电量降为零；
- 在 7:50–9:10 与 17:40–20:50 两个电价高峰段（电价最高达 1.3952 元/kWh，约为低谷的 3.76 倍），购电行为被显著抑制：早高峰 7:30–10:00 的购电量**恰好为零**，完全由储能放电支撑；晚高峰 17:00–21:00 购电量回落至 0–826.08 kWh/10 min（24 个时段中仅 12 个时段有购电），降幅虽大但仍保留了部分购电——因为此时储能储电量已接近下限，继续放电将破坏后续时段的供电能力；
- 在光伏出力超过小区负载的 9:30–14:20（共 30 个时段）内，购电量在 22 个时段**降为零**，微网完全由光伏供电并有盈余为储能充电；其余 8 个时段（11:00–12:30）仍有 179.28–575.17 kWh/10 min 的购电。这并非供电需要，而是模型捕捉到了正午时分电价出现的局部低谷（0.4113–0.5007 元/kWh，低于相邻时段）：此时储能尚有充足容量（储电量由 2971 kWh 升至 9144 kWh），故在光伏已经富余的情况下仍提

前购入低价电存入储能，留待傍晚 1.3952 元/kWh 的高价时段使用。这正是表 1 中 10:00、14:00、20:00 三个时段购电量为零、而 12:00、16:00、18:00 仍保留一部分购电的原因——16:00、18:00 的购电则是因储电量已达上限 10800 kWh、光伏出力已低于负载，只能由外网补足。

这一策略正是“谷充峰放”的定量体现：模型把所有可推迟的购电需求都转移到了电价最低的时段。用边际条件可以解释这一行为：忽略容量与功率限制时，在低价时段购入 1 kWh 电量用于充电，未来只能向负载交付 $\eta_c \eta_d$ kWh，故跨时段购电储能具有经济性的条件为 $p_{\text{低}}/\eta_c \eta_d < p_{\text{高}}$ 。代入 $\eta_c \eta_d = 0.81$ ，只要峰谷价差超过约 1.23 倍即值得套利，而附件 1 的实际峰谷比达 3.76，套利空间充裕。实际充电时段则由优化模型在综合考虑光伏富余、后续峰值价格、剩余容量与日末恢复电量后自动确定。

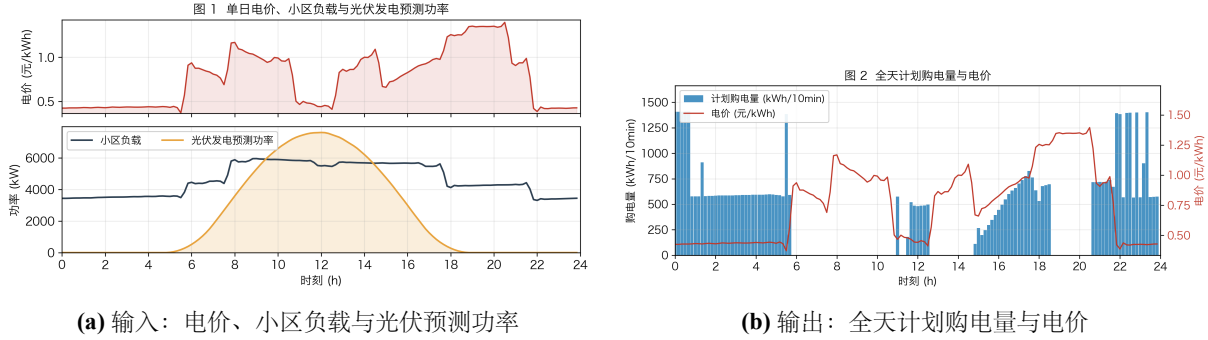


图 1 问题一的输入与计划购电结果

5.5.2 储能运行特征

图 2a 给出储能设备的充放电功率与储电量轨迹，呈现出典型的两充两放格局：

1. **第一充放循环：**0:00–5:40 利用夜间低谷电价充电，储电量由 6000 kWh 升至 10800 kWh（5:40 首次触及上界）；5:50–9:30 在早高峰放电，储电量一路降至全天次低点 1834.62 kWh；
2. **第二充放循环：**9:30–16:00 吸收光伏盈余并在正午电价局部低谷时补充购电充电（储电量自 1834.62 kWh 回升，13:50 突破 10500 kWh，16:00 再次达到上界 10800 kWh）；18:00–20:40 在晚高峰放电，储电量降至下界 1200 kWh（20:40 首次触及，并保持至 21:50）；
3. 21:50–24:00 再次充电，将储电量精确送回 6000 kWh，满足日循环条件。

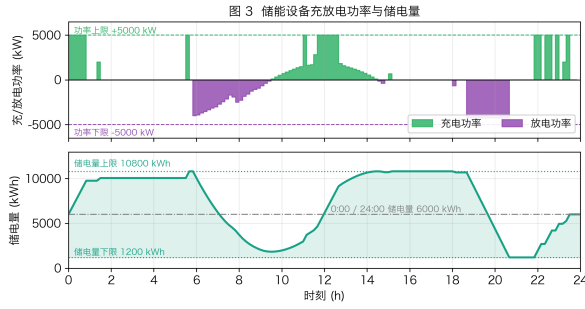
需要强调的是，储电量在 0:00–6:00 的充电过程中**精确触及上界 10800 kWh**，在 17:00–21:00 的放电过程中**精确触及下界 1200 kWh**。这说明储电量约束是**紧的 (binding)**：储能容量已成为制约进一步降低购电费的关键瓶颈。这一发现对实际工程具有直接意义——适当扩大储能容量或放宽储电量运行区间，可望带来额外的经济效益。

此外，全天充电量 20740.67 kWh 与放电量 16799.94 kWh 之比恰为 0.810000，与式 (10) 的理论值 $\eta_c \eta_d = 0.81$ 完全一致，从能量守恒角度佐证了求解结果的正确性，也印证了“日循环不等于充放电量相等”这一容易误判的结论。

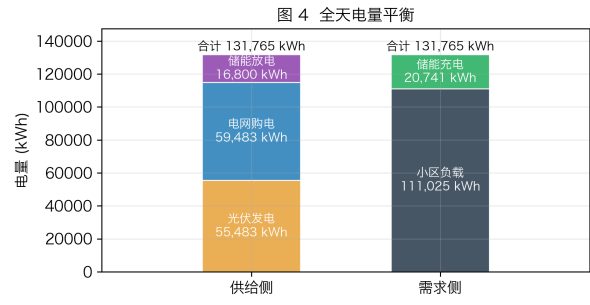
5.5.3 全天电量平衡与弃光

图 2b 给出全天电量的供给与需求构成。供给侧为光伏发电 55482.84 kWh、电网购电 59482.70 kWh 与储能放电 16799.94 kWh，合计 131765.48 kWh；需求侧为小区负载 111024.81 kWh 与储能充电 20740.67 kWh，合计亦为 131765.48 kWh，两侧严格相等。

一个值得注意的结果是：本模型的**弃光电量为零**，即 $s_t^* = 0$ 对所有 t 成立。储能设备将光伏出力超过负载的全部盈余电量（6247.96 kWh）尽数吸纳，未造成任何浪费。这是 10 分钟精细调度粒度带来的直接收益——若采用粗粒度调度，光伏盈余与储能充电能力在时间上难以精确匹配，将不可避免地产生弃光（详见第 5.6 节）。



(a) 储能设备充放电功率与储电量



(b) 全天电量平衡

图 2 问题一的储能运行与全天电量平衡

5.6 基准对照与灵敏度分析

为量化储能与光伏的经济价值，本文构造“光伏供电、缺口直接外购、电池保持不动”的基准方案（它是原问题的一个可行解，故给出同一口径下的费用上界）；针对题目只给出一个“充放电效率 90%”、其单程/往返分配方式存在解释空间的情况，又设计了效率分配、日循环水平与决策粒度共六组对照口径。两组对照的完整数据与逐项推理见附录 I，正文只保留结论。

1. **储能的价值不止于价差套利。**配置储能后全天购电费由 48052.05 元降至 35126.95 元，节省 12925.10 元（降幅 26.90%）；较“无光伏无储能”方案节省 53192.41 元（60.23%）。含储能方案的购电量反而更少（59482.70 对 61789.94 kWh），因为它把无储能时被迫弃掉的 6247.96 kWh 光伏盈余转移到了无光照时段，提升了光伏本地消纳率。
2. **效率约定影响有限，主要结论稳健。**单程与往返两种解释构成购电费区间 [33416.53, 35126.95] 元，跨度不足 5%；两种口径下“谷充峰放”的策略结构与零弃光结论均不变。
3. **决策粒度具有实质经济价值。**决策由 10 分钟粗化为 1 小时时，购电费上升 4.68% 并产生 768.44 kWh 弃光；粗化为 4 小时时上升 24.26%（43648.06 元），弃光增至 2293.67 kWh。因此采用 10 分钟粒度是必要的，这也印证了题目以 10 分钟为间隔要求购电量的合理性。
4. **给定的初始储电量已接近最优循环水平。**解除 $E_0 = 6000$ kWh 的固定约束后模型收敛到 $E_0 = 8550$ kWh，购电费仅下降 8.35 元（0.024%），无需为此偏离题目给定值。

六、 问题二：预测驱动的滚动日前计划与实时执行模型

6.1 问题二相对问题一的三个本质变化

问题二放宽了问题一的确定性设定：**电价仍逐日相同**（重复使用附件 1 的日内曲线），但**小区负载与光伏出力**改为附件 2 给出的全年实际值，且**计划购电不足时须按交易时刻电价的 5 倍紧急补购**。这带来三个必须在模型中显式处理的变化：

1. **信息约束：**每天 0:00 制定计划时，当天的负载与光伏实际值尚未发生，只能依据此前历史预测，不能提前读取待预测当天的实际曲线；
2. **代价不对称：**少买需要按 5 倍价格紧急补购，多买仍按计划量全额收费且**不予退款**。两侧代价不对称，意味着“平均预测误差最小”不再是正确的优化目标；
3. **跨日状态连续：**问题二没有再要求每天 0:00 与 24:00 的储电量相同，因此电池状态必须**跨日连续传递**，不能每天重置为 6000 kWh，也不能套用问题一“全天放电量为充电量的 0.81 倍”这一关系（该关系以首尾电量相等为前提）。

相应地，模型由问题一的**单日确定性 MILP** 扩展为**三层滚动结构**：预测层（逐日预测负载与光伏）→ 计划层（在风险修正净负荷下求解日 MILP，冻结当天 144 段计划购电量）→ 执行层（逐段观测实际值，因果地决定充放电、紧急购电与弃电，并把实际日未储电量传给次日）。

6.2 数据准备、时间口径与信息时点

附件 2 提供 2025 年 365 天、每天 144 个 10 分钟时段的负载功率与光伏实际功率，经校验无空值、无负值、日期严格递增且恰好覆盖 2025:01:01–2025:12:31。全天合计负载 40524.1 MWh、合计光伏 20251.2 MWh，功率范围分别为 1995.72–7978.88 kW 与 0–10216.20 kW。

时段口径与问题一完全一致：附件 2 的列标签同样采用区间**结束时刻**，故第 j 列对应区间 $[(j-1)\Delta t, j\Delta t)$ 。附件 5 中 `result2.xlsx` 的”计划购电量”表首段标签为 0:10–0:20、末段为 0:00–0:10+1，与问题一模板一样整体错位一格。本文在输出副本中按正文口径统一改写为 0:00–0:10 至 23:50–0:00+1，行序与模板一一对应，原模板文件保持不变。

信息时点纪律：2025 年 1 月被指定为训练与状态预热期，不进入正式结果；正式输出与评价区间为 2 月 1 日至 12 月 31 日，共 334 天。2 月 1 日的初始储电量由 1 月逐日实际运行自然递推得到，而非人为设定。每天的预测、误差分位数与标定参数一律只使用当天之前的已完成数据，计划电量 $g_{n,t}$ 在 0:00 一经确定便不再修改。

6.3 预测模型：先识别数据中的结构性规律

预测方法的选择不应先验假定，而应从数据出发。本文对附件 2 的 365 天数据做了两项结构性检验：

(1) 负载存在极强的星期结构。把全年按星期分组求日总电量均值，得到周一至周四及周日约为 123900–124144 kWh/日，而周五、周六仅约 78332–78407 kWh/日，两者之比达 1.58 倍。这一结构是**整体水平**上的：两族的日内归一化形状高度相似（相关系数 0.888，同样呈现凌晨低谷、上午高峰、午间回落、午后次高、入夜走低的形态），但并不完全一致——以周五、周六一族为基准，族间逐时段相对差的中位数为 8.9%，在傍晚 17:30 前后最大可达 38.5%（该族在傍晚的相对水平明显更低），而到日末 23:50 处又反向相差 30.9%（周五、周六的相对水平反而更高）。因此若把不同星期的历史日混在一起做加权平均，既会把两族水平”平均掉”、又无法刻画这一形状差异，产生系统性偏差。

(2) 光伏不存在星期结构，而以季节性为主。按星期分组的光伏日电量均值极差仅 1.80%，且与星期无明显对应关系，故光伏预测应侧重**最近**的历史曲线以跟踪季节变化。

据此采用”近期加权历史值”的预测形式：

$$\hat{\ell}_{n,t} = \sum_{i \in \mathcal{H}_n} a_{n,i} \ell_{i,t}, \quad \hat{v}_{n,t} = \sum_{i \in \mathcal{H}_n} b_{n,i} v_{i,t}, \quad a_{n,i}, b_{n,i} \geq 0, \quad \sum_i a_{n,i} = \sum_i b_{n,i} = 1 \quad (13)$$

其中权重按”周”衰减， $w_{n,i} = d^{(n-1-i)/7}$ 。负载预测取回看窗口 15 天、周衰减因子 $d = 0.30$ ，并只保留与待预测日同一星期的历史日；光伏预测取回看窗口 5 天、 $d = 0.50$ ，使用全部历史日。两组超参数只依据 1 月的数据选定，在正式区间开始前即行冻结，此后不再调整，从而不引入未来信息。

”是否按星期分组”这一结构选择可由 1 月数据**独立判定**：在 1 月的 30 天上，**不区分星期**与**区分星期**的净负荷日绝对误差分别为 20173.9 与 5903.7 kWh/日，相差 3.4 倍。在回看窗口取 7、14、21、28 天、衰减因子取 0.5 与 1.0 的 1 月网格上重复比较，不区分星期一族的误差落在 19304.8–20173.9 kWh/日，区分星期一族落在 5824.3–5903.7 kWh/日，**两族区间互不重叠**（比值 3.3–3.5 倍），故该结构结论不依赖窗口与衰减因子的具体取值。下

述对照与精度指标则在正式区间（2月1日–12月31日）上统计，属于对该结构选择的**事后评价**：**不区分星期**时误差为 19796.2 kWh/日，**区分星期**后降至 3116.9 kWh/日，改善约 6.4 倍。这一差距并非窗口长度等超参数造成的假象——在回看窗口取 7、14、21、28 天、衰减因子取 0.5 的一组设定下，不区分星期方案的误差始终稳定在 19498–19512 kWh/日；而**同样把这组窗口用于区分星期方案**，误差稳定在 3035–3302 kWh/日。换用逐时段口径复核结论一致：逐段绝对误差和由 20162.9 kWh/日降至 6372.4 kWh/日（3.2 倍）。正式区间上的预测精度见表 13：负载平均绝对误差 1823.4 kWh/日（相对 1.65%）、光伏 2077.9 kWh/日（3.64%）、净负荷 3116.9 kWh/日。

上面三个指标都是**逐日全天电量**口径的绝对误差，取绝对值后抹去了方向，而本题的代价恰恰是**不对称的**——低估触发 5 倍紧急购电，高估只造成弃电。因此另按**逐时段**口径统计两个方向，见表 13：实际净负荷高于点预测的时段占 49.1%、合计 3181.9 kWh/日（低估时段平均低估 45.0 kWh/段）；低于点预测的时段占 50.9%、合计 3190.5 kWh/日。两个方向在**频次与电量上都几乎对称**（49.1% 对 50.9%，3181.9 对 3190.5），说明点预测本身没有系统性的偏高或偏低，误差结构是“噪声型”而非“偏差型”。这意味着继续改进预测方法难以再获益，收益只能来自对**同一个误差分布**选取合适的分位数——这正是下一节的出发点。

令净负荷预测为 $\hat{N}_{n,t} = \hat{\ell}_{n,t} - \hat{v}_{n,t}$ 。注意**不把负的净负荷截为零**：负值表示光伏有余，是可以存入储能的信息，截断会白白丢失。

6.4 风险余量：为何可以从 80% 分位数出发

设某时段随机净负荷为 N ，计划购电 $g \geq 0$ 。忽略储能时，期望费用为

$$\min_{g \geq 0} pg + 5p \mathbb{E}[(N - g)_+], \quad (x)_+ = \max(x, 0). \quad (14)$$

若 N 的分布连续且最优解位于 $g > 0$ ，一阶条件给出

$$\frac{\partial}{\partial g} pg + 5p \Pr(N > g) = 0 \implies F_N(g) = \frac{5 - 1}{5} = 0.8. \quad (15)$$

即**最优计划量是被 5 倍紧急电价推高到 80% 分位数**的位置：多买 1 kWh 的确定成本是 p ，少买 1 kWh 的期望成本是 $5p \cdot \Pr(N > g)$ ，二者在 $F_N(g) = 0.8$ 处平衡。这一结果给出风险参数一个清楚的经济解释，但它**只在忽略储能与跨时段耦合时严格成立**，因此本文把它作为参数搜索的起点而非结论，最终由实际连续运行的总费用校准。

实践中无法直接得到 N 的分布，转而对**净负荷预测误差**建模。对每个已经结束的历史日期 i ，保存其当时利用更早数据作出的预测，计算误差

$$\varepsilon_{i,t} = (\ell_{i,t} - v_{i,t}) - \hat{N}_{i,t|\text{日期} < i}. \quad (16)$$

这里用的是**逐日预测后得到的真实预测误差**，而不是模型在训练数据上的拟合残差——后者会显著低估实际误差。取此前 28 天内对应时段误差的经验 α 分位数 $Q_{\alpha,n,t}$ ，构造风险修正净负荷：

$$\tilde{N}_{n,t} = \hat{N}_{n,t} + Q_{\alpha,n,t}. \quad (17)$$

第 n 天的分位数只取 $\varepsilon_{i,t}$ ($i < n$) 中已结束日期的样本；历史样本不足 10 个时改取该时段 ± 2 个相邻时段合并后的分位数，冷启动阶段仍有可用估计。直接对净负荷误差建模，还自动保留了负载误差与光伏误差之间的相关性。

6.5 日前计划模型：风险净负荷下的 MILP

每天 0:00 求解一次。决策变量以**电量** (kWh) 为单位：计划购电量 $g_{n,t}$ 、参考充电量 $\bar{c}_{n,t}$ 、参考放电量 $\bar{d}_{n,t}$ 、参考剩余电量 $\bar{w}_{n,t}$ 、参考储电量 $\bar{E}_{n,t}$ ，以及模式变量 $z_{n,t} \in \{0, 1\}$ 。带横线的变量表示**日前参考计划**，不是实际执行结果。

目标函数除购电费外，加入日末储备不足的软惩罚：

$$\min \sum_{t=1}^{144} p_t g_{n,t} + \lambda \xi_n, \quad \xi_n \geq E^{\text{tar}} - \bar{E}_{n,144}, \quad \xi_n \geq 0. \quad (18)$$

$E^{\text{tar}} = 6000$ kWh 是希望为次日保留的参考电量， λ (元/kWh) 是储备不足的规划代价。引入它的目的是减轻“只优化一天、日末过度放电”的短视倾向。需要强调，**该惩罚只用于制定策略，绝不计入实际上报的购电费**。本文并不预设 λ 的取值，而是把它与 α 、 ρ 一并交给数据标定（见 6.7 节）。

约束条件包括四组：

$$\begin{aligned} g_{n,t} + \bar{d}_{n,t} &= \tilde{N}_{n,t} + \bar{c}_{n,t} + \bar{w}_{n,t}, & g_{n,t} \geq 0, \bar{w}_{n,t} \geq 0 \\ \bar{E}_{n,t} &= \bar{E}_{n,t-1} + \eta_c \bar{c}_{n,t} - \frac{\bar{d}_{n,t}}{\eta_d}, & \bar{E}_{n,0} = E_{n,0}^{\text{actual}} \\ E_{\min} &\leq \bar{E}_{n,t} \leq E_{\max}, & E_{\min} = 1200, E_{\max} = 10800 \\ 0 \leq \bar{c}_{n,t} &\leq M z_{n,t}, \quad 0 \leq \bar{d}_{n,t} \leq M(1 - z_{n,t}), & z_{n,t} \in \{0, 1\} \end{aligned} \quad (19)$$

其中 $M = 5000\Delta t = 2500/3$ kWh， $\eta_c = \eta_d = 0.90$ ，与问题一保持同一套母线侧充放电口径。注意平衡式中使用的是**风险修正后**的净负荷，因此 $\bar{w}$ 只是规划意义上的剩余量，**实际弃电量必须在执行层重新计算**。 $\bar{c}$ 与 $\bar{w}$ 均不收费，即 $\bar{w}$ 允许“弃用已购电”，否则预测偏高的时段会人为变成不可行。

按变量排列，模型 (19) 共 $6 \times 144 + 1 = 865$ 个决策变量，其中 144 个为二元变量 $z_{n,t}$ ，其余为连续变量，故是**混合整数线性规划**。结构矩阵与问题一一样只依赖时段划分，可**预先装配一次**，此后每天只更新等式右端项，单次求解约 46 ms。

6.6 实时执行模型：因果反馈规则

执行层只使用当天 0:00 已冻结的计划、当前已观测的功率和当前储电量，不读取任何未来实际值。设使用储能前的电量盈余为

$$b_{n,t} = g_{n,t} + v_{n,t}^{\text{actual}} - \ell_{n,t}^{\text{actual}}, \quad (20)$$

$b > 0$ 表示计划购电与光伏有剩余， $b < 0$ 表示出现缺口。

(1) 有剩余时充电，只利用本段已有的剩余电量，不为追赶参考计划在负载已经缺电时机械充电：

$$c_{n,t}^{\text{actual}} = \min \left\{ (b_{n,t})_+, M, \frac{E_{\max} - E_{n,t-1}^{\text{actual}}}{\eta_c} \right\}. \quad (21)$$

(2) 有缺口时按储备线放电。在 0:00 即可算出当天的储备线

$$R_{n,t} = E_{\min} + \rho (\bar{E}_{n,t} - E_{\min}), \quad 0 \leq \rho \leq 1, \quad (22)$$

实际放电量为

$$d_{n,t}^{\text{actual}} = \min \left\{ (-b_{n,t})_+, M, \eta_d (E_{n,t-1}^{\text{actual}} - R_{n,t})_+ \right\}. \quad (23)$$

$\rho = 0$ 表示尽可能用现有储能补足当前缺口； $\rho = 1$ 表示不主动放到当段参考电量以下，为后续时段保留更多电量。注意**较高储备不一定更省钱**，故 ρ 同样交由标定决定。

(3) 紧急购电、弃电与真实电池状态。

$$r_{n,t} = (-b_{n,t} - d_{n,t}^{\text{actual}})_+, \quad w_{n,t}^{\text{actual}} = (b_{n,t} - c_{n,t}^{\text{actual}})_+, \quad (24)$$

$$E_{n,t}^{\text{actual}} = E_{n,t-1}^{\text{actual}} + \eta_c c_{n,t}^{\text{actual}} - \frac{d_{n,t}^{\text{actual}}}{\eta_d}. \quad (25)$$

该规则自动避免同时充放电，并保证实际逐段能量守恒：

$$g_{n,t} + r_{n,t} + v_{n,t}^{\text{actual}} + d_{n,t}^{\text{actual}} = \ell_{n,t}^{\text{actual}} + c_{n,t}^{\text{actual}} + w_{n,t}^{\text{actual}}. \quad (26)$$

$w_{n,t}^{\text{actual}}$ 允许来自光伏，也允许来自已经付费的计划购电，因此不设置“弃电量只能小于光伏电量”的限制。

(4) 跨日状态传递。次日初始电量严格取当天**实际执行后**的日末电量：

$$E_{n+1,0}^{\text{actual}} = E_{n,144}^{\text{actual}}. \quad (27)$$

不能用参考计划中的 $\bar{E}_{n,144}$ 替代。由式 (25) 对全天求和可得状态累计恒等式

$$E_{n,144}^{\text{actual}} - E_{n,0}^{\text{actual}} = \eta_c \sum_{t=1}^{144} c_{n,t}^{\text{actual}} - \frac{1}{\eta_d} \sum_{t=1}^{144} d_{n,t}^{\text{actual}}, \quad (28)$$

它是本文逐日校验的核心关系之一。只有当某日首尾电量恰好相同时， $\sum_t d_{n,t} = 0.81 \sum_t c_{n,t}$ 才成立；问题二中绝大多数日子并不满足，故**不能沿用问题一的这一关系**。

6.7 参数标定：用过去连续区间的实际总费用选参

模型含三个需要选择的参数：风险分位数 α 、储备线系数 ρ 与日末储备代价 λ 。本文采用如下**严格因果**的滚动标定：从正式区间第一天起，每隔 42 天重新标定一次；每次标定时，对参数网格中的每个组合，从**该日之前的 35 天窗口**起点出发，用主运行在**窗口起点日 0:00**的实际储电量作为窗口初值回放整个窗口，比较窗口内**包含 5 倍紧急购电费的实际总费用**，取最小者。需说明的是，首次标定（2 月 1 日）的窗口受数据起点 1 月 1 日限制，实际只有 31 天；其余 7 次均为完整的 35 天。参数网格为

$$\alpha \in \{0.50, 0.60, 0.70, 0.80, 0.90\}, \quad \rho \in \{0, 1\}, \quad \lambda \in \{0, 0.60\}, \quad (29)$$

共 $5 \times 2 \times 2 = 20$ 个组合。标定所用的历史、回放窗口与参数一律不越过标定日，因此完全满足信息时点要求。标定结果见表 15。

6.8 求解结果与校验

6.8.1 六项强制检查

按前述要求，对 365 天的完整运行逐日执行六项检查，表 14 给出结果，**全部通过**：

1. **信息时点**：程序逐日记录预测实际引用的历史日下标，其上界严格小于当天；误差分位数只取 $i < n$ 的已结束日期； $g_{n,t}$ 在 0:00 冻结，执行层结构中不存在修改 g 的入口。
2. **物理可行性**：365 天逐段供需平衡残差的**最大值**为 2.3×10^{-13} kWh（判定阈值为 1.3×10^{-3} kWh，实测值比阈值小 10 个数量级）；储电量全程落在 $[1200, 10800]$ kWh、越界量为 0； $c, d \leq M$ （越限量 0）且无同时充放电时段（0 段）。

3. 跨日连续性：逐日校验 $E_{n,0}^{\text{actual}} = E_{n-1,144}^{\text{actual}}$ ，365 天全部成立。
4. 状态累计：式 (28) 逐日成立， ΔE 与 $\eta_c \sum_t c_t - \eta_d^{-1} \sum_t d_t$ 的最大绝对偏差为 1.3×10^{-11} kWh。
5. 费用一致性：用未四舍五入的计划量与紧急量重新计算费用，与记录值的最大偏差为 0 元；弃电不退款；日末储备惩罚未混入实际账单。
6. 汇总一致性：日期完整、无跨日错位；紧急购电事件合并前后的电量总和严格相等（455 个事件全部核对）。

6.8.2 总体结果

表 13 给出正式区间的总体结果。334 天中计划购电量 21701732.99 kWh、计划购电费 13358737.95 元；紧急购电量 124045.19 kWh、紧急购电费 662827.17 元；**合计购电费 14021565.12 元**，平均购电单价约 0.642 元/kWh。紧急购电费仅占总费用的 4.73%，334 天中有 160 天发生紧急购电。全年弃用电量 2704833.51 kWh，期末储电量 1687.84 kWh，储电量全程位于 [1200, 10800] kWh 区间内。

表 3 微网在指定日期的计划购电量、全天购电量与购电费（问题二）

日期	10:00	12:00	14:00	16:00	18:00	20:00
2025.3.20	0.00	589.01	0.00	489.63	712.12	0.00
2025.6.21	0.00	0.00	0.00	343.48	550.29	0.00
2025.9.23	0.00	473.63	0.00	540.59	754.15	0.00
2025.12.21	0.00	991.46	0.00	850.06	742.70	0.00
全天汇总（单位：kWh，费用单位：元）						
日期	计划购电量		紧急购电量		合计购电费	
2025.3.20	68,165.47		0.00		41,546.86	
2025.6.21	41,410.82		0.00		25,342.18	
2025.9.23	66,665.55		1,012.68		47,837.19	
2025.12.21	100,405.38		0.00		65,004.18	

表 4 储能设备在指定日期的充放电量及 0:00 和 24:00 的储电量（问题二）

时段	2025.3.20		2025.6.21		2025.9.23	
	充电	放电	充电	放电	充电	放电
0:00-4:00	8,099.25	129.91	1,740.43	0.00	8,945.44	46.80
4:00-8:00	1,970.90	5,258.80	1,483.33	1,719.95	1,886.47	6,408.47
8:00-12:00	6,192.83	2,777.38	5,693.41	0.00	4,911.33	1,896.62
12:00-16:00	3,806.26	450.70	0.00	0.00	5,349.28	635.68
16:00-20:00	302.08	5,657.14	0.00	4,736.32	113.40	5,890.59
20:00-24:00	601.91	2,967.92	2,538.89	2,485.90	384.92	2,403.49
0:00 储电量	2,094.13		4,685.60		1,245.61	
24:00 储电量	1,812.43		5,060.32		1,475.54	

表 5 2025 年 12 月 21 日储能设备的充放电量及 0:00 和 24:00 的储电量

时段	充电量/kWh	放电量/kWh
0:00-4:00	8,789.13	60.85
4:00-8:00	1,862.99	1,410.66
8:00-12:00	5,955.61	6,659.77
12:00-16:00	5,982.69	2,220.11
16:00-20:00	0.00	5,545.22
20:00-24:00	635.34	2,843.90
0:00 储电量/kWh	1,970.17	
24:00 储电量/kWh	2,050.57	

6.8.3 指定日期的结果

表 3、表 4、表 5 与表 6 分别给出 3 月 20 日、6 月 21 日、9 月 23 日与 12 月 21 日四个指定日期的计划购电量、储能充放电量与紧急购电情况（其中 12 月 21 日的储能充放电量因列宽原因单列一表）。四个日期的运行特征差异明显，恰好覆盖了不同季节与不同星期的组合：

表 6 微网在指定日期的紧急购电量（问题二，按连续时段合并）

日期	紧急购电时间段	紧急购电量/kWh	该日紧急费/元
2025.3.20		无	0.00
2025.6.21		无	0.00
	20:20-20:40	806.60	
2025.9.23	20:50-22:00	185.65	6,522.02
	22:20-22:30	20.44	
2025.12.21		无	0.00

注：紧急购电量按框架要求逐段计算后，将同日连续发生紧急购电的时段合并为一个事件；合并后电量与逐段之和严格相等。

表 7 各策略在 2025 年 2 月 1 日-12 月 31 日的费用对照（同一预测器、同一实时执行规则、各自单独标定）

策略	计划购电费/元	紧急购电费/元	合计费用/元	期末储电量/kWh	相对本模型
本文风险修正策略（主模型）	13,358,738	662,827	14,021,565	1,687.8	—
预测均值策略（不加风险余量）	12,197,543	2,867,578	15,065,121	5,957.1	+7.44%
无储能（仍提前计划、五倍补缺）	17,503,774	1,052,131	18,555,905	6,000.0	+32.34%
事后理想（完全预知当天净负荷）	12,082,248	0	12,082,248	—	-13.83%

注：“事后理想”用当天实际净负荷替换预测值求解同一日前 MILP。它每天仍从主策略的实际日初储电量出发，且不要求日末保留电量，故日末储电量被放到下限附近（日均 1, 200.0 kWh），属于不可执行的乐观下界，仅作参考。

- 3 月 20 日（周四）与 9 月 23 日（周二）为工作日，负载处于高位（117648.38 kWh 与 121182.28 kWh），光伏出力中等（55439.42 kWh 与 59065.67 kWh），计划购电量分别为 68165.47 kWh 与 66665.55 kWh；
- 6 月 21 日（周六）属于低负载的周末族，全天负载仅 81408.06 kWh，而光伏出力却是四者中最高（62073.32 kWh），净负荷最低，故计划购电量仅 41410.82 kWh，为四个日期中最低；也是四者中弃电量最大的一天（19562.18 kWh）；
- 12 月 21 日（周日）适逢冬至，光伏出力处于全年低位（35338.66 kWh，在正式区间内为第 8 低），负载则属高负载族（124376.44 kWh，该族约 124000 kWh/日），净负荷在正式区间内列第 3，全天计划购电量高达 100405.38 kWh、购电费 65004.18 元，为四个日期中最高。

四个日期的这一组差异，恰好覆盖了负载的星期结构与光伏的季节变化两个维度，使得指定日期的结果具有代表性。

在紧急购电方面，四个指定日期中有三个**全天无紧急购电**，仅 9 月 23 日发生 3 个紧急购电事件（20:20-20:40、20:50-22:00、22:20-22:30），合计 1012.68 kWh、6522.02 元。这三个事件都出现在晚间 20 时以后：该日白天光伏出力偏低，储能到傍晚已接近下限，晚间高价时段出现预测未能覆盖的额外缺口，只能按 5 倍电价补购。事件按“同日连续发生紧急购电的时段合并为一个事件”的规则生成，合并后的电量与逐段之和严格相等。

6.9 结果分析

6.9.1 预测误差如何转化为紧急购电成本

图 3 给出四个指定日期的净负荷预测、风险余量与实际净负荷。三个层次的关系清晰可见：净负荷预测（虚线）给出基准，风险余量 Q_α （橙色阴影）在预测之上叠加一段风险修正量，得到用于日前求解的风险净负荷（橙实线），而实际净负荷（深蓝实线）则在其上下波动。

需要说明， Q_α 是历史误差的经验分位数，**并不保证为正**：正式区间 $334 \times 144 = 48096$ 个时段中有 1779 个（3.7%）为负，最小值 -30.9 kWh、最大值 241.1 kWh。这些负值出现在误差分布整体偏负（长期被高估）的时段，此时“分位数修正”正确地指示了**可以少买**——它与总体上“风险余量抬高计划量”的方向并不矛盾，因为决定费用的始终是分位数本身，而不是“加一个正数”这一形式。

关键在于**实际曲线落在风险净负荷之上的部分**。图 3 用倒三角标出了发生紧急购电的时段，9 月 23 日出现了 10 段紧急购电，另外三个日期则全天为零。但逐时段核对可

以发现，超出风险净负荷只是发生紧急购电的必要条件，而不是充分条件：

- 6月21日确实被风险净负荷**完整包住**（144段无一段超出）；
- 3月20日与12月21日分别有42段与14段超出风险净负荷，但最大超出仅85.2 kWh与80.4 kWh，且都出现在储能尚有余量的时段，被电池吸收，故**全天无紧急购电**；
- 9月23日有68段超出，最大超出137.8 kWh，且**当日储能在20:30已被放到下限1200 kWh，并在其后约8个时段钉在下限上**；电池一旦触底，其后每一段缺口都只能按5倍电价补购。

这解释了9月23日晚间的10段紧急购电：其中9段的实际净负荷高于风险净负荷，另1段（20:30）的实际值甚至**低于**风险净负荷34.0 kWh，却仍紧急购电784.9 kWh——因为电池已无电可放。可见真正的触发机制是”**实际净负荷超出风险余量的部分**”与”**储能剩余可放能力**”两者之差：前者为正只是故事的开始，只有当储能无法再兜底时，才转化为5倍电价的支出。这三个日期的差别正在于此——3月20日与12月21日的超出量小（超幅超过50 kWh的时段各仅2个），9月23日则达19个。

由此得到本文最重要的定性结论：**紧急购电成本来自”预测偏低”，而非”预测不准”**——预测偏高只会造成弃电（无直接费用），预测偏低才会在储能耗尽时触发5倍电价。这也是选择**分位数**而非均值作为计划依据的根本原因。

图1 指定日期的净负荷预测、风险余量与实际净负荷

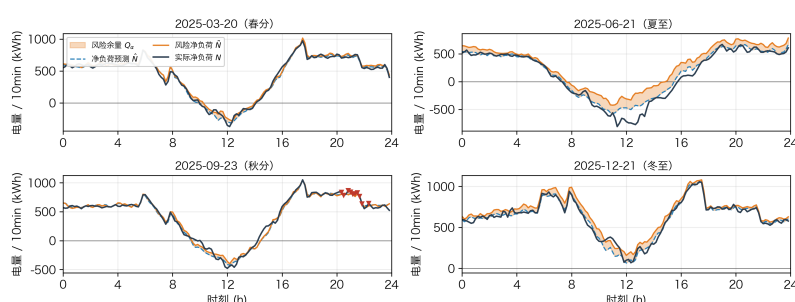


图3 指定日期的净负荷预测、风险余量与实际净负荷

6.9.2 计划购电与紧急购电的费用贡献

图4给出四个指定日期的计划购电量、紧急购电量与电价的对照。计划购电量（蓝色柱）呈现与电价高度相关的形态：在0:00–6:00与22:00–24:00的低价时段最大，在7:50–9:10的早高峰被压到零。而紧急购电量（红色斜纹）则以很细的柱体叠加在计划量之上，仅在个别时段出现，说明在风险余量的保护下绝大多数时段的计划量已经足够。

值得注意的是**计划购电量并不总是等于实际净负荷**：在中午光伏富余的时段，12:00附近仍保留数百 kWh 的计划购电。这与问题一中出现的现象同源——该时段电价处于局部低谷，提前购入留待傍晚高价时段使用更为经济，只不过问题二中这一决策必须在不知道当天实际光伏的情况下、基于风险净负荷作出。

图2 指定日期的计划购电量、紧急购电量与电价

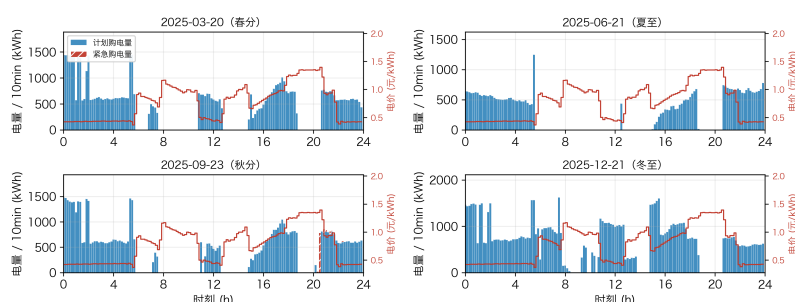


图4 指定日期的计划购电量、紧急购电量与电价

6.9.3 预测误差对储能轨迹的扰动

图 5 对比了四个指定日期的日前参考储电量 $\bar{E}$ 与实际执行后的储电量 E 。两者的差异正是预测误差在执行层的体现：参考轨迹是 0:00 基于风险净负荷算出的最优路径，而实际轨迹则因实际负载与光伏的偏离而上下浮动。图中同时标出了 $[1200, 10800]$ kWh 的运行边界与 $E^{\text{tar}} = 6000$ kWh 的参考水平。可以看到实际轨迹始终未越界，且四个日期的日初与日末储电量 **各不相同**（例如 3 月 20 日 $2094.13 \rightarrow 1812.43$ kWh，6 月 21 日 $4685.60 \rightarrow 5060.32$ kWh），这正是问题二**跨日连续、不要求日循环**的直接证据——若强行要求每天回到 6000 kWh，反而会人为限制模型的调节能力。

图 3 指定日期的日前参考储电量与实际储电量

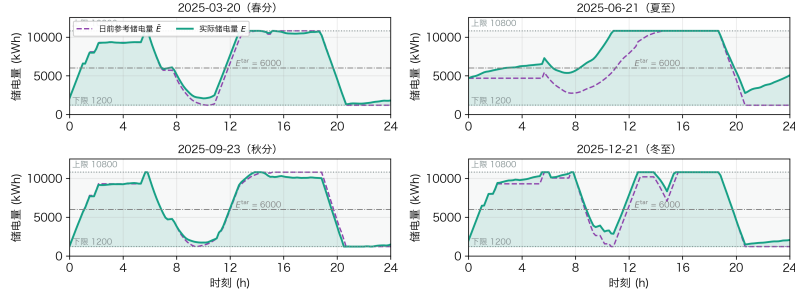


图 5 指定日期的日前参考储电量与实际储电量

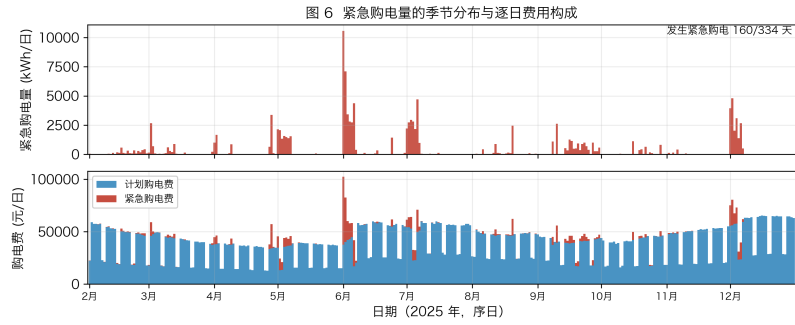


图 6 紧急购电量的季节分布与逐日费用构成

6.9.4 全年费用分解与对照策略

图 7 从全年尺度展示结果。图 7(a) 给出本文策略的逐月费用分解。月费用最高的三个月是 12 月（175.6 万元）、7 月（158.9 万元）与 6 月（155.6 万元），最低的三个月是 4 月（96.4 万元）、5 月（100.5 万元）与 10 月（111.8 万元），最高月约为最低月的 1.8 倍。这一形态主要由负载与光伏的季节错配决定：12 月光伏出力全年最弱而负载处于高位，6–7 月则因制冷负荷抬高用电水平，而春季与初秋光伏尚可、负载适中，故费用最低。

值得注意的是**月度总费用的排序与紧急购电费的排序并不一致**：紧急购电费在 6 月达到 190201 元，为全年最高，**占全年紧急购电费的 28.7%**，12 月（95615 元）与 9 月（84866 元）次之；而 11 月的紧急购电费仅 5028 元、为全年最低，其总费用却仍排在第 5 位。这说明**月费用的高低主要仍由负载水平决定，紧急购电费则取决于预测偏差恰好与储能容量紧张相遇的频率**，二者是两个不同的驱动因素。

为量化风险余量的价值，构造三组对照策略，全部使用**同一预测器、同一实时执行规则、同一评价区间（2–12 月共 334 天）**，并各自单独标定参数：

- **预测均值策略**：令 $\alpha = \text{None}$ ，即完全不加风险余量，直接用点预测制定日前计划，其余规则完全相同；
- **无储能策略**：仍提前制定购电计划、仍按 5 倍价格补缺，但不启用储能，用于单独评价电池的价值；
- **事后理想策略**：用当天**实际净负荷**替换预测值求解同一日前 MILP，代表”完全预

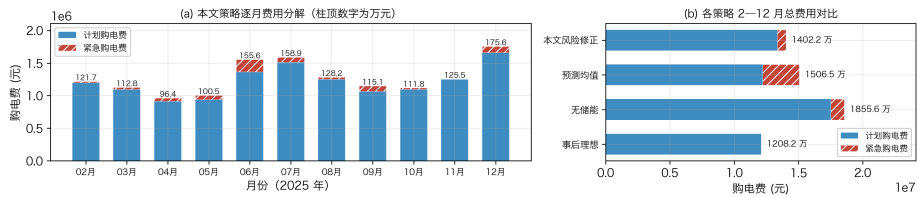
知当天净负荷”的费用参照。须注意它每天仍从主策略的实际日初储电量出发、且不要求日末保留电量，因此会把电池放到下限收手，是一个**不可执行的乐观下界**，而非可达方案。

表 7 给出四者的费用对比。本文风险修正策略全年合计 14021565 元，较预测均值策略的 15065121 元节省 1043556 元、较无储能策略的 18555905 元节省 4534340 元，分别相当于**本文策略费用**的 7.44% 与 32.34%（与表 7 末列同口径）。事后理想策略为 12082248 元，比本文策略低 1939317 元（13.8%）。但该值 **偏乐观**：理想策略每天直接动用主策略已按计划电价付过费的库存电量，且不要求日末留存。若改为让理想方案自行传递跨日状态、并与主策略一样扣住日末储电量，其费用为 12251972 元，差距收窄至 12.6%。按这一**自治口径**，预测不确定性带来的不可避免代价约为 12.6%（约 177 万元），本文策略由此仍有明确的改进空间。

更值得注意的是费用结构的变化：预测均值策略的**紧急购电费高达 2867578 元，是本文策略 662827 元的 4.3 倍**，而其计划购电费反而更低（12197543 元对 13358738 元）。这正是“代价不对称”的直接体现——不加风险余量时，日前计划买的电更少、看起来更省，但代价是运行时频繁触发 5 倍电价的紧急购电，总费用反而更高。**风险余量本质上是用“少量、分散、可预期的计划购电溢价”替换“大量、集中、代价高昂的紧急购电”**。

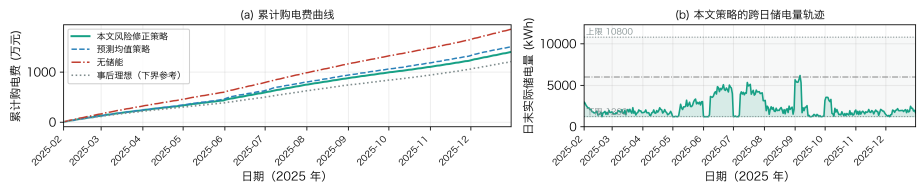
同时应注意，四个策略的**期末储电量并不相同**（本文策略 1687.8 kWh，预测均值策略 5957.1 kWh，无储能策略 6000.0 kWh）。期末储电量差异意味着库存并未被完全对等地消耗，因此本文同时报告该指标，避免把“多消耗库存”误当作算法收益。

图 4 逐月费用分解与策略费用对比



(a) 逐月费用分解与策略费用对比

图 5 各策略累计费用与跨日储电量



(b) 各策略累计费用与跨日储电量

图 7 全年费用分解与策略对照

6.9.5 标定结果对理论分析的验证

表 15 给出 8 次滚动标定的选中参数。结果非常集中： α 在 8 次标定中**全部落在 0.70–0.90 之间**，其中 0.80 出现 6 次、0.90 与 0.70 各 1 次。这一结果与式 (15) 的理论预测**高度吻合**：由 5 倍紧急电价导出的最优分位数恰为 0.80，而完全基于 2025 年实际数据、按真实总费用选出的参数也稳定落在 0.80 附近。**理论推导与实际数据标定相互印证**，这既支持了风险修正模型的合理性，也说明本文的 80% 分位数并非人为设定。

关于 λ ，全部 8 次标定都选择了 $\lambda = 0.00$ 。这说明在 **35 天窗口**的实际总费用这一评价口径下，日末储备惩罚并未带来净收益：窗口内的回放已经包含了储备不足在后续日期引发的紧急购电代价，而由于电池容量上限（10800 kWh）本身构成了自然的约束，额外的储备惩罚只会让模型在日末保留过多电量、减少可用的调峰空间。因此本文最终

报告的结果中 $\lambda = 0$ ；但需强调， ξ 项在模型中的存在是必要的——它使得“是否保留储备”成为一个由数据回答的问题，而不是一个未加检验的隐含假设。 ρ 的取值则在 0 与 1 两个端点之间跳变（6 次取 0、2 次取 1，且两次取 1 分别出现在 2 月与 6 月），同样说明较高的储备线并不必然更省钱。

6.9.6 弃用电量的成因

全年弃用电量 2704833.51 kWh，占同期光伏发电量 19068890.0 kWh 的 14.2%。题目未对弃电设置惩罚或收益，故弃电不带来额外扣款，但它仍是日前计划与实时运行之间偏差的**指示器**。逐时段分解显示弃电来自**两个性质不同的来源**：70.0% 是光伏富余超出储能吸纳能力（储能受 5000 kW 功率上限与 10800 kWh 容量上限双重约束）；另 30.0% 发生在光伏并无富余的时段，只能来自**已按计划电价付费、却因实际负载低于预期而用不掉的计划购电量**，其当初购电成本 544354 元约占全年计划购电费的 4.07%——即约三成弃电是“按 5 倍电价买保险”预付而未出险的保费。 α 敏感性验证表明这一取舍**经过权衡且最优**： α 由 0.80 降至 0.50 时弃电由 2662.5 降至 1500.1 MWh，但紧急购电量由 179.8 猛增至 642.9 MWh，**总费用反由 1407.2 万元升至 1513.1 万元**。此外，紧急购电负担在时间上**高度集中**：334 天中 160 天发生紧急购电，而 6 月 1 日单日即占全年紧急购电费的 9.8%，说明它由少数连续多日的高压时段主导。成因分解的完整数据与运营含义见附录 I。

6.9.7 模型的边界

本文的“风险余量 + 日前 MILP + 因果执行规则”是一条便于实现与解释的**近似策略**，其优劣由按时间顺序的实际回放判定，**并不等同于已求出所有随机控制策略中的全局最优解**；参数标定是过去 35 天窗口内的近视最优；计费口径的歧义会影响调减的方向。**适用边界的完整讨论见附录 I。**

七、 问题三：多版本预报驱动的滚动购电与调整模型

7.1 模型框架：三个购电量与交付块策略

问题三的核心困难不在物理模型——储能、平衡与效率约束与问题二完全一致——而在“计划”与“调整”这两个概念**必须被严格区分**。为此首先定义三个互不相同的量：

- g_t^0 ：当天 0:00 依据当时唯一可得的 0:00 预报确定的**初始计划购电量**。它一旦确定即**永久保存**，后续任何调整都不得覆盖它；
- x_t ：该时段**最终生效的完整购电量**（不含紧急购电）。注意它是一个完整量，**不是**相对于 g_t^0 的增量，**也不是**该时段被修订过的多个版本之和；
- r_t ：实际运行中额外发生的**紧急购电量**。

最终结算的常规电量为 $G^{\text{effective}} = \sum_t x_t$ ，紧急电量为 $G^{\text{emg}} = \sum_t r_t$ ，微网实际收到的电量为 $G^{\text{received}} = G^{\text{effective}} + G^{\text{emg}}$ ——这里**不能再把 g_t^0 加一遍**。

由于 6:00、12:00、18:00 都可以修改购电，同一个时段在理论上可能被修改多次，因而必须先约定结算方式。本文采用**交付块 (delivery block)** 策略：把一天切成四个六小时交付块 [0, 6), [6, 12), [12, 18), [18, 24)（对应时段索引 [0, 36), [36, 72), [72, 108), [108, 144)），每个块**只在它的块首被正式调整一次**：

决策时刻	正式提交的交付块	提交的时段索引
0:00	全天 144 段（即全部四个块）	0-143
6:00	6:00-12:00	36-71
12:00	12:00-18:00	72-107
18:00	18:00-24:00	108-143

每个十分钟时段因此只与自己的 g_t^0 结算一次，“最终净偏差结算一次”与“逐次变更累加收费”之间的歧义被彻底消除。求解时对更远时段的购电量仍会一并算出，但它

们只是**展望值**：不提交、不收费，也不写入结果文件中的调整购电量；到对应交付块开始时，再用当时的新信息重新决定。这是为消除结算歧义而选定的策略范围，而非题目强制每次只能调整六小时。

7.2 预报数据的时间口径

整点功率还是小时平均？附件 3 的每条记录给出”预报 1 小时”至”预报 24 小时”共 24 个**功率值**。它究竟是目标整点的瞬时功率，还是目标小时内的平均功率，决定了整点之间该如何取值。本文用对齐扫描判定：把预报值与附件 2 的实际光伏按不同时间平移对齐，比较全天平均绝对偏差，在正式区间内得到整点对齐时 191.51 kW、前移一小时 808.67 kW、按小时平均 268.02 kW（正式区间 2025-02-01 至 2025-12-31 的十分钟平均绝对偏差）。扫描脚本与输出见支撑材料 p3_alignment_scan.py 与 p3_alignment_scan.json。整点口径的偏差最小且差距显著，故**按瞬时整点功率处理**，在同一发布版本内部作分段线性插值。

设 $F_h^{(\tau)}$ 为发布时刻 τ 对目标时刻 $\tau + h$ 的预测功率。对相邻整点之间的时刻 $u = \tau + h + \theta$, $0 \leq \theta \leq 1$, 取

$$\widehat{V}^{(\tau)}(u) = (1 - \theta)F_h^{(\tau)} + \theta F_{h+1}^{(\tau)}.$$

十分钟区间的两端均与整点对齐，故该段预测电量为

$$\widehat{v}_t^{(\tau)} = \frac{\widehat{V}^{(\tau)}(a_t) + \widehat{V}^{(\tau)}(b_t)}{2} \Delta t, \quad \Delta t = \frac{1}{6} \text{ h},$$

即插值曲线在区间上的精确积分。

首端锚点。附件 3 从”预报 1 小时”开始，没有发布时刻自身的 $F_0^{(\tau)}$ 。本文约定：6:00、12:00、18:00 三个版本的首端取**同一时刻更早的发布版本**对该时刻的预报（例如 6:00 版的首端取当天 0:00 版对 6:00 的”预报 6 小时”，0:00 版的首端取前一天 18:00 版对该时刻的”预报 6 小时”。这一约定只使用**已经发布**的预报，不含任何未来信息。此外，附件 2 中全年每一天 0:00–0:10 的实际光伏恒为 0，因此 0:00 版本对该段取 0 与实际完全一致。

发布时间管理。代码把附件 3 展开为长表，每行同时保存发布时间、目标时间、提前小时数与预测功率，并按”发布时间不晚于当前决策时刻”筛选可用版本。这一纪律由数据结构本身保证：第 τ 次求解只读取 $F^{(\tau)}$ 这一张表，即只含 τ 时刻**已经发布**的预报。因此 0:00 绝不读取当天 6:00 才发布的预报，6:00 用的是当天 6:00 版，而 12:00、18:00 版要等到各自的发布时刻才进入求解——某个版本覆盖的目标时段越远（最多到发布后 24 小时），越要等到对应交付块开始时才被正式提交。

负载与误差。光伏直接采用附件 3 的当前已发布版本，负载仍沿用问题二的历史外推，并在 6:00 之后额外使用当天**已经完成**的时段作日内比例修正：取当日已观测段的实际负载与该段预测值之和的比值 $\kappa = \sum_{t < t_0} \ell_t^{\text{act}} / \sum_{t < t_0} \widehat{\ell}_t$ （分子分母取**同一时间窗**， t_0 为当前发布时刻对应的时段索引），用 κ 缩放当日剩余时段的负载预测。误差表按发布版本与目标时段维护：

$$\varepsilon_{n,t}^{(h)} = N_{n,t} - \widehat{N}_{n,t}^{(h)},$$

且只对 $t \geq 36h$ （即发布时刻之后的时段）**定义**——发布时刻之前的时段该版本本就不预测，强行填零会污染分位数。风险净负荷为 $\widetilde{N}_t^{(h)} = \widehat{N}_t^{(h)} + Q_\alpha(\varepsilon_{\cdot,t}^{(h)})$ ，分位数取发布时刻之前的历史样本；样本不足时不做修正。

图 8 把四个发布版本对同一个日期的光伏预报与事后实际值画在一起，直观显示了”多版本”的含义：0:00 版本只有一条正式预报，而到 18:00 时，早晨那次预报的偏差已经被三次修订逐步纠正。四个版本的精度差异将直接决定它们各自的边际价值（第 7.8 节）。

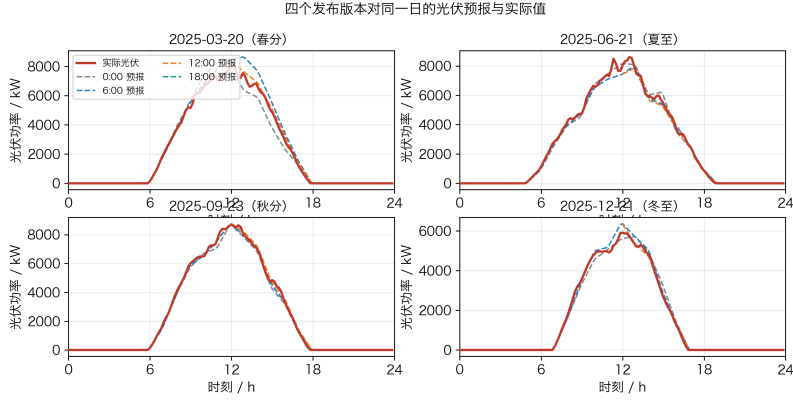


图 8 同一目标日期的多版本光伏预报与事后实际值

7.3 计费口径与调整模型

题目规定：“计划购电量高于调整购电量的部分，违约电价是交易时刻电价的 50%；调整购电量高于计划购电量的部分，超出部分的电价是交易时刻电价的 1.5 倍”，而“总的购电费用包括计划购电费用、紧急购电费用和调整购电量的相关费用”。这里存在一处必须**显式声明**的歧义：调减时被取消部分的**原价是否退回**，题面并未说明。本文以两种口径分别求解，并在论文中同时报告。

主口径（原计划照付、调减另计违约金）。计划费按 g_t^0 全额收取，调整费单独计算：

$$A_t(x_t; g_t^0) = 1.5 p_t(x_t - g_t^0)_+ + 0.5 p_t(g_t^0 - x_t)_+, \quad (y)_+ = \max(y, 0),$$

$$J^{\text{actual}} = \sum_t p_t g_t^0 + \sum_t A_t(x_t; g_t^0) + \sum_t 5 p_t r_t.$$

其中 $1.5 p_t$ 是上调部分的**总价**，不是“先付 p_t 再加 $1.5 p_t$ ”成为 $2.5 p_t$ 。该口径有一个直接结论：既然原计划已经付过费，保留原量并把多余电量无偿弃用，比减少购电再付违约金**至少不贵**，故可令 $x_t \geq g_t^0$ ， A_t 退化为 $1.5 p_t(x_t - g_t^0)$ 。本文以该口径为主模型。

退款口径（取消部分退回原价、再收 50% 违约金）。若把题面理解为被取消部分的原价退回，则费用为

$$J^{\text{refund}} = \sum_t \left[p_t g_t^0 + 1.5 p_t(x_t - g_t^0)_+ - 0.5 p_t(g_t^0 - x_t)_+ + 5 p_t r_t \right],$$

此时调减项为负，**调减才可能节约费用**，优化允许双向调整。本文把它作为一项独立的敏感性检查，与主口径使用同一套模型、同一套标定与执行规则，仅替换费用函数。

两种口径的分段线性化方式相同：引入非负的增量与减量变量 $a_t, b_t \geq 0$ 与

$$y_t - a_t + b_t = g_t^0, \quad a_t, b_t \geq 0,$$

则 $A_t = 1.5 p_t a_t - 0.5 p_t b_t$ ，对 a_t, b_t **无需引入二元变量**——因为把二者同时抬高 δ 只会平白多付 $(1.5 - 0.5) p_t \delta = p_t \delta$ ，最优解中二者不会同时为正。主口径再额外令 $b_t \equiv 0$ ， $y_t \geq g_t^0$ ，此时 $A_t = 1.5 p_t a_t$ 。

为何这是“调整”而不是“重新购电”。在调整时点，原计划费用已经确定，在目标函数中属于常数，因此不再重复计入；已经执行过的时段产生的调整费与紧急费同样不再优化，只保留在实际账本中。

7.4 两个 MILP 与执行层

0:00 初始计划。沿用问题二的日计划 MILP，只把历史外推的光伏点预测换成 0:00 已发布版本的光伏预报：

$$\min \sum_{t=0}^{143} p_t g_t^0 + \lambda \xi, \quad \xi \geq E^{\text{tar}} - \bar{E}_{144}, \quad \xi \geq 0,$$

$$g_t^0 + \bar{d}_t = \tilde{N}_t^{(0)} + \bar{c}_t + \bar{w}_t, \quad g_t^0, \bar{w}_t \geq 0,$$

$\bar{c}_t, \bar{d}_t, \bar{E}_t$ 为参考储能轨迹， $\bar{w}_t$ 为规划剩余，并满足 $1200 \leq \bar{E}_t \leq 10800$ 与充放电互斥，**这些都不是实际结果**。注意该模型**不含紧急购电变量**：0:00 只有该时点发布的预报版本可用，没有更新的预报、也没有当日的实际观测，模型只按风险净负荷 $\tilde{N}_t^{(0)}$ 安排一个“量入为出”的计划，缺口与盈余都由后续的执行层和调整层处理。

发布时点的调整模型。设 $\mathcal{T}_\tau$ 为当前时刻到当日 24:00 的尚未执行时段，以 y_t 为候选完整购电量：

$$\min \sum_{t \in \mathcal{T}_\tau} A_t(y_t; g_t^0) + \sum_{t \in \mathcal{T}_\tau} 5p_t \bar{r}_t + \lambda \xi,$$

$$y_t + \bar{r}_t + \bar{d}_t = \tilde{N}_t^{(\tau)} + \bar{c}_t + \bar{w}_t, \quad \bar{E}_t = \bar{E}_{t-1} + \eta_c \bar{c}_t - \frac{\bar{d}_t}{\eta_d}, \quad 1200 \leq \bar{E}_t \leq 10800,$$

$$0 \leq \bar{c}_t \leq M z_t, \quad 0 \leq \bar{d}_t \leq M(1 - z_t), \quad z_t \in \{0, 1\}, \quad M = 5000 \times \frac{1}{6} = \frac{2500}{3} \text{ kWh}.$$

这里有两个容易被忽略的实现要点。其一，每次求解必须以**当前实际储电量**为初值，而不是上一轮的预测电量——这是“滚动”的全部意义所在。其二， $\tilde{N}_t^{(\tau)}$ 与原计划 g_t^0 使用**不同的**信息集：前者是用最新版本预报和最新误差分位数算出的风险净负荷，后者是 0:00 就冻结的历史量。更新预报后，即便不改变 y_t ，也可以重排储能，使未来缺口下降——因此代码在每次求解后**整体刷新参考轨迹** $\bar{E}_t$ ，但**只把当前交付块**的 y_t 写回生效购电量 x_t ，其余全部丢弃。

执行层。每十分钟的实际充放电与紧急补购完全复用问题二的因果规则——这种“每个决策时刻只依据当前已实现状态重排未来轨迹”的做法正是模型预测控制的基本思想 [6]。令使用储能前的盈余为 $b_t = x_t + v_t^{\text{actual}} - \ell_t^{\text{actual}}$ ，用当前参考轨迹构造储备线 $R_t = 1200 + \rho(\bar{E}_t - 1200)$ ，则

$$c_t = \min \left\{ (b_t)_+, M, \frac{10800 - \bar{E}_{t-1}}{\eta_c} \right\}, \quad d_t = \min \left\{ (-b_t)_+, M, \eta_d (\bar{E}_{t-1} - R_t)_+ \right\},$$

$$r_t = (-b_t - d_t)_+, \quad w_t = (b_t - c_t)_+, \quad \bar{E}_t = \bar{E}_{t-1} + \eta_c c_t - \frac{d_t}{\eta_d},$$

并满足逐段守恒 $x_t + r_t + v_t^{\text{actual}} + d_t = \ell_t^{\text{actual}} + c_t + w_t$ 。规则**只依赖当前与历史信息**： b_t 用到当段实际负载与光伏， $\bar{E}_{t-1}$ 是上一段的实际状态。当日的实际日末电量直接作为次日初值，不要求日循环；1 月作为预热期，2 月 1 日的初值由 1 月的实际运行得到。

7.5 参数标定

需要标定的参数有三个：风险分位数 α 、储备线比例 ρ 与日末储备惩罚权重 λ 。 α 与 ρ 由**过去连续区间的实际总费用**选出，而不是照搬问题二的 0.80 或凭经验设定：每 42 天用此前 35 天作回放窗口，在 $\alpha \in \{0.30, \dots, 0.80\}$ 与 $\rho \in \{0, 0.25, 0.5, 1.0\}$ 的二维网格上逐一回放（共 24 组），取窗口内**含调整费与 5 倍紧急费用的实际总费用**最小者。 λ 固定为 0.60 并**不参与搜索**：它的作用只是防止日末储备被过度透支。问题二的标定网格本就含 $\lambda \in \{0, 0.60\}$ ，其 8 次标定全部选中 0，说明该项在问题二的账本中并未被激

活；把它一并入网格只会让回放次数成倍增加，代价与收益不成比例。窗口初值取主运行在该日 0:00 的实际储电量，不使用窗口末尾当天的状态，避免引入未来信息。

有两处公平性要求必须遵守：其一，**每种策略在它自己的预报组合上标定**，否则用”全套预报”选出的参数会被强加给”仅 0:00”的策略，比较结果失去意义；其二，各策略在评价区间、初始状态、负载模型、执行规则与计费口径上完全一致，不得利用四个展示日期倒选参数。由于不同策略的实际储电量会在跨日回放中逐渐分岔，这是策略的**后果**而非缺陷，不能每天人为重置来抹平，故比较费用时必须同时报告期末储电量。

题目未给出获取预报本身的费用，因此**不应**自行增加”预报购买费”；”是否需要其他时刻的预报”只能表达为：某次更新是否降低实际总费用、改善是否跨月份稳定、以及能否被其他更新替代。

表 8 指定日期六个时段的初始计划购电量 g_t^0 、最终生效购电量 x_t 、正式调整量与紧急购电量（单位：**kWh**）

日期	时段	初始计划 g_t^0	最终生效 x_t	调整量	紧急购电 r_t
2025.3.20	10:00-10:10	0.00	0.00	0.00	0.00
	12:00-12:10	509.51	509.51	0.00	0.00
	14:00-14:10	0.00	0.00	0.00	0.00
	16:00-16:10	647.39	647.39	0.00	0.00
	18:00-18:10	698.72	704.31	5.59	0.00
	20:00-20:10	0.00	0.00	0.00	0.00
2025.6.21	10:00-10:10	0.00	0.00	0.00	0.00
	12:00-12:10	0.00	0.00	0.00	0.00
	14:00-14:10	0.00	0.00	0.00	0.00
	16:00-16:10	241.46	241.46	0.00	0.00
	18:00-18:10	0.00	0.00	0.00	0.00
	20:00-20:10	0.00	0.00	0.00	0.00
2025.9.23	10:00-10:10	0.00	0.00	0.00	0.00
	12:00-12:10	309.90	309.90	0.00	0.00
	14:00-14:10	0.00	0.00	0.00	0.00
	16:00-16:10	526.15	526.15	0.00	0.00
	18:00-18:10	755.68	781.52	25.84	0.00
	20:00-20:10	0.00	0.00	0.00	0.00
2025.12.21	10:00-10:10	0.00	0.00	0.00	0.00
	12:00-12:10	979.36	979.36	0.00	0.00
	14:00-14:10	0.00	0.00	0.00	0.00
	16:00-16:10	852.80	852.80	0.00	0.00
	18:00-18:10	733.72	733.72	0.00	0.00
	20:00-20:10	0.00	0.00	0.00	0.00

续表：全天费用分解（单位：元）。计划费按 g_t^0 全额收取，调整费按 $1.5p_t(x_t - g_t^0)$ 另计，二者与紧急费之和即当日实际总费用。

日期	初始计划量	最终常规量	紧急购电量	计划费	调整费	当日合计费用
2025.3.20	65,739.8	67,183.3	371.0	40,069	1,744	43,632
2025.6.21	35,596.8	35,608.9	0.0	20,694	20	20,714
2025.9.23	65,479.3	66,526.2	673.8	40,323	1,512	45,683
2025.12.21	96,942.5	96,942.5	12.1	62,044	0	62,115

表 9 储能设备在指定日期的实际充放电量（按四小时汇总，单位：**kWh**）

时段	2025.3.20		2025.6.21		2025.9.23	
	充电	放电	充电	放电	充电	放电
0:00-4:00	4,183.11	436.34	604.87	1,022.39	4,716.66	324.79
4:00-8:00	907.49	5,900.81	469.00	4,396.07	1,018.72	6,581.69
8:00-12:00	4,695.33	2,406.36	9,512.78	0.00	5,294.08	1,812.49
12:00-16:00	6,285.80	254.72	0.00	0.00	5,892.83	465.86
16:00-20:00	9.23	5,667.02	168.09	5,393.94	99.06	5,870.41
20:00-24:00	5,562.75	3,026.13	6,097.43	2,504.35	5,325.43	2,811.44
0:00 储电量	6,333.5		7,292.5		5,722.1	
24:00 储电量	6,155.7		7,663.1		5,982.4	

7.6 求解结果与七项校验

以 2025 年 1 月为预热期、2 月 1 日-12 月 31 日共 334 天为正式区间，主策略（使用全部四个发布时刻）的结果为：**合计购电费 13457697.61 元**，其中计划购电费 12492137.14 元、调整费 675387.19 元、紧急购电费 290173.29 元。电量方面，初始计划量 $G^{\text{initial}} = 20527257.44$ kWh，最终生效常规量 $G^{\text{effective}} = 21124276.98$ kWh，紧急

表 10 2025 年 12 月 21 日储能设备的实际充放电量与 0:00、24:00 储电量

时段	充电量/kWh	放电量/kWh
0:00-4:00	4,763.94	182.24
4:00-8:00	1,462.17	1,495.25
8:00-12:00	5,881.35	7,168.08
12:00-16:00	6,820.49	2,287.09
16:00-20:00	9.57	5,752.84
20:00-24:00	5,548.98	2,848.52
0:00 储电量/kWh	6,134.5	
24:00 储电量/kWh	6,245.6	

表 11 微网在指定日期的紧急购电事件（问题三，按同日连续时段合并）

日期	紧急购电时间段	紧急购电量/kWh	该日紧急费/元
2025.3.20	9:00-10:00	371.02	1,819
2025.6.21	无		0.00
2025.9.23	9:20-9:50 20:20-21:50	84.13 589.71	3,847
2025.12.21	7:50-8:00	12.14	71

注：紧急购电量逐十分钟按 $5p_t r_t$ 计费后再把同日连续时段合并成事件，合并不改变费用；合并后电量与逐段之和严格相等。

表 12 八种预报组合在 2025 年 2 月 1 日–12 月 31 日的实际费用（各组合单独标定参数，评价区间、负载模型、执行规则与计费口径完全一致）

使用的预报时刻	初始计划费/元	调整费/元	紧急费/元	合计费用/元	相对 $\emptyset$	期末储电量/kWh
仅 0:00	13,460,229	0	747,173	14,207,402	—	6,290.4
6	13,184,766	211,122	580,165	13,976,054	-1.63%	6,290.4
12	13,027,382	253,395	625,467	13,906,244	-2.12%	6,057.1
18	12,922,577	369,782	440,657	13,733,016	-3.34%	6,057.1
6,12	12,748,432	469,452	460,787	13,678,671	-3.72%	5,891.2
6,18	12,628,812	625,335	220,381	13,474,527	-5.16%	5,891.2
12,18	12,819,205	403,719	498,346	13,721,269	-3.42%	6,057.1
6,12,18	12,492,137	675,387	290,173	13,457,698	-5.28%	5,891.2
对照：重新优化但仍用 0:00 版本	—	—	—	13,658,321	-3.86%	6,057.1
敏感性：退款口径 (x_t 可双向调整)	—	—	—	13,396,938	—	—

注：所有方案都使用 0:00 发布的预报；组合列出的时刻是额外使用的版本。费用按主计费口径计算，合“计费用已”经包含初始计划费，不能与计划费、调整费再相加。期末储电量用于说明各方案的跨日状态差异，不能只比较费用。未行为计费口径的敏感性检查：退款口径的目标函数另含调减收益，其总额与上表的相“对列”不可直接比较。

购电量 $G^{\text{emg}} = 51328.71 \text{ kWh}$ ，故微网实际收到 $G^{\text{received}} = 21175605.69 \text{ kWh}$ ；储能充电 6708959.29 kWh、放电 5435240.89 kWh、弃电 1962697.80 kWh，期初储电量 6984.40 kWh、期末储电量 5891.21 kWh。全年共正式提交调整 1002 次、求解 MILP 1336 次 (334×4)，与”四个交付块各求解一次”完全一致。

三项费用中调整费已超过紧急费，这与问题二的结论形成了鲜明对照：问题二中只能被动接受计划、缺口按 5 倍电价硬补，全年紧急费 662827.17 元、占合计费用的 4.73%；问题三中主动付费调整计划，紧急费降到 290173.29 元，仅为问题二的 43.8%，代价是付出 675387.19 元调整费。

对主策略的 334 个交付日逐日执行框架 8.2 节要求的七项检查，结果见表 17，七项检查的未通过天数全部为零。数值上，逐段供需平衡的最大残差为 $2.27 \times 10^{-13} \text{ kWh}$ ，状态累计的最大绝对偏差为 $1.91 \times 10^{-11} \text{ kWh}$ ，储电量越界与充放电越限均为 0，同时充放电时段数为 0，三项费用按独立路径复算的偏差不超过 $4.55 \times 10^{-13} \text{ 元}$ （其中计划费与紧急费恰为 0），主口径下下调量最大为 $1.02 \times 10^{-12} \text{ kWh}$ ——即 $x_t < g_t^0$ 在浮点零的意义下从未发生，与第 7.3 节的理论结论一致。这些残差都在双精度浮点的舍入量级，说明模型约束与账本记录是同一套代数关系的两种写法。

7.7 结果分析

(1) 信息的作用是”用调整费置换紧急费”。把八种组合的三项费用并列（表 12、图 11）可以看到两个沿组合合格严格单调的趋势：使用的预报版本越多，计划费越低（ $\emptyset$

的 13460229 元降至 $\{6, 12, 18\}$ 的 12492137 元)，**调整费越高**（0 元升至 675387.19 元）。这说明后发布的预报并没有让总购电量凭空变少，而是把原本要在执行阶段按 5 倍电价紧急补购的电量，提前识别出来并按 1.5 倍的价格调整。由于追加电价 $1.5p$ 远低于紧急电价 $5p$ ，只要追加不是盲目的，这一置换在总体上就有利：最完整的组合比只用 0:00 预报便宜 749704.62 元（5.28%）。

需要提醒的是，**紧急费本身并不随版本增多而单调下降**。它从 $\emptyset$ 的 747173.09 元降到 $\{6, 12, 18\}$ 的 290173.29 元（降幅 61.2%），但在 $\{18\}$ 之上再加 12:00 反而使紧急费由 440656.93 元回升到 $\{12, 18\}$ 的 498345.58 元。原因见第 7.8 节：12:00 版提供的信息与相邻两个版本高度重复，它重排了储能与购电，却把一部分缺口推到了电价更高的时段。真正随版本单调的是**总费用**（ $\emptyset \rightarrow \{18\} \rightarrow \{6, 18\} \rightarrow \{6, 12, 18\}$ 依次为 14207402 $\rightarrow$ 13733016 $\rightarrow$ 13474527 $\rightarrow$ 13457698 元，沿格单调不减），**计划费与调整费也各自沿格单调**（前者递减、后者递增，12 条覆盖边全部通过），**只有紧急费不单调**——上例中它先降后升。这说明“版本越多越省”是总费用层面的结论，不能简化为任一单项费用。

(2) 紧急购电量而非紧急费更能说明信息价值。 $\emptyset$ 组合全年紧急购电 126972.52 kWh， $\{6, 12, 18\}$ 组合仅 51328.71 kWh，减少 59.6%。值得注意的是，紧急**费**的降幅（61.2%）比**量**的降幅更大，这意味着剩下的 51328.71 kWh 紧急购电比原来的更集中在**低电价时段**——预报越准，越能把不可避免的缺口挪到便宜的时候补。

(3) 调整并非越多越好。 $\{6, 12, 18\}$ 的正式提交次数为 1002 次（334 天 $\times$ 三次更新），但其中相当一部分提交的结果是“保持不变”： $x_t = g_t^0$ 的时段占绝大多数。图 9a 给出四个指定日期的初始计划购电量与最终生效购电量，两条曲线的差异只在少数时段出现，且几乎都是 $x_t > g_t^0$ 的单向上调。这正是第 7.4 节所强调的——更新预报后重新求解，既可以改变购电量，也可以只重排储能；只有当预计减少的紧急购电收益超过调整费时，调价才真正发生。

(4) 储能轨迹被滚动修正。图 9b 给出四个指定日期的参考轨迹与实际储电量。可以看到每个发布时刻的参考轨迹都会重新拼接（竖虚线），而实际储电量始终连续。全年**日末**储电量落在 4836.9–9607.1 kWh 之间，没有出现日末被放空或充满的极端状态。这里必须区分**日末位置**与**逐时轨迹**：逐时段看，储能确实会在光伏富余时段触及 10800 kWh 的上界（334 天中有 324 天）、在连续缺口时被压至 1200 kWh 的下界（157 天），这正是设备容量对消纳能力的真实约束；而储备线 $R_t = 1200 + \rho(\bar{E}_t - 1200)$ 的作用，是防止执行层为省钱而在日末透支储能。四个指定日期的具体结果见表 8、表 9、表 10 与表 11。

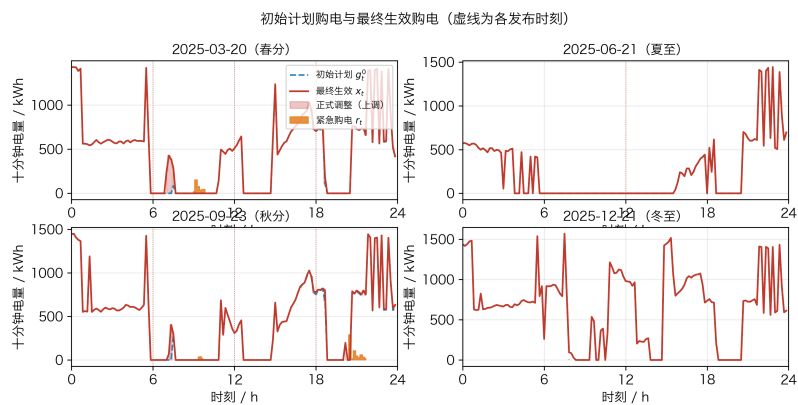
(5) 收益在月份间并不均匀。图 10 给出主策略逐日的调整费、紧急费与日末储电量。按月汇总，**调整费在 7 月最高**（13.57 万元），约为 2 月（3.04 万元）的 4.5 倍；**紧急费则在 5 月最高**（5.61 万元）。两项支出都集中在空调负荷与光伏波动最大的月份，这也说明分月评价预报收益是必要的，不能只看四个指定日期。

7.8 八种预报组合的比较：是否需要其他时刻的预报

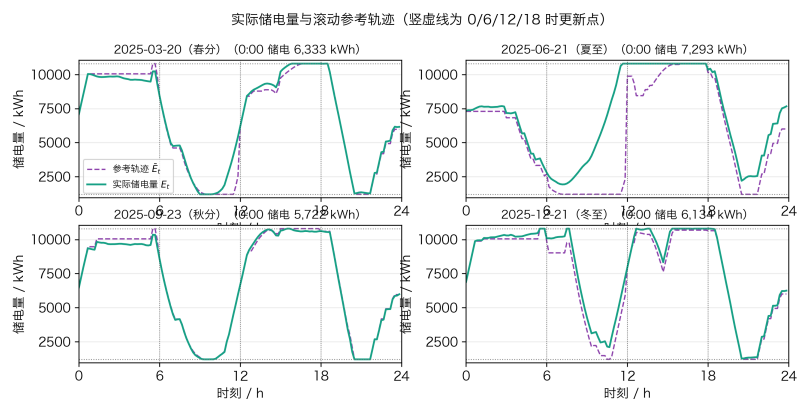
表 12 与图 11 给出八种组合的实际总费用。为回答“是否需要其他时刻的预报”，还要看**边际改善** $\Delta_h(S) = C(S) - C(S \cup \{h\})$ ，即已经用了 S 之后再加用时刻 h 能省多少。计算结果为（单位：万元）：

已有组合 S	$\Delta_6(S)$	$\Delta_{12}(S)$	$\Delta_{18}(S)$	参考
$\emptyset$	23.13	30.12	47.44	单独加入
$\{6\}$	—	29.74	50.15	
$\{12\}$	22.76	—	18.50	
$\{18\}$	25.85	1.17	—	
$\{6, 12\}$	—	—	22.10	
$\{6, 18\}$	—	1.68	—	
$\{12, 18\}$	26.36	—	—	

由此可以给出三个层次明确的结论。



(a) 初始计划购电量 g_t^0 与最终生效购电量 x_t



(b) 滚动参考储电量 $\bar{E}_t$ 与实际储电量 E_t

图 9 指定日期的购电量调整与储能轨迹

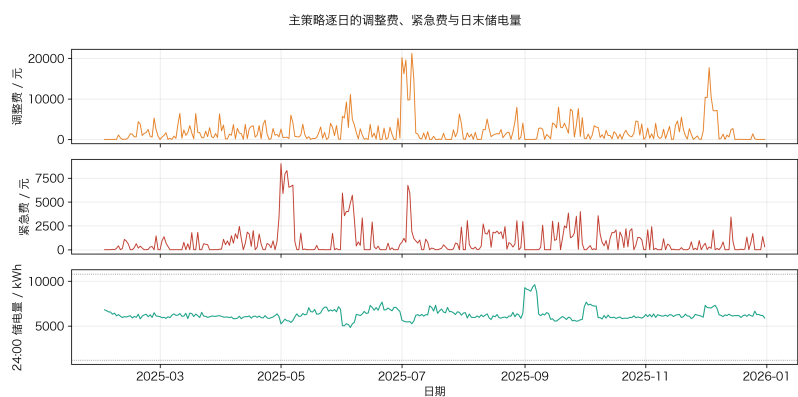


图 10 主策略逐日的调整费、紧急费与日末储电量

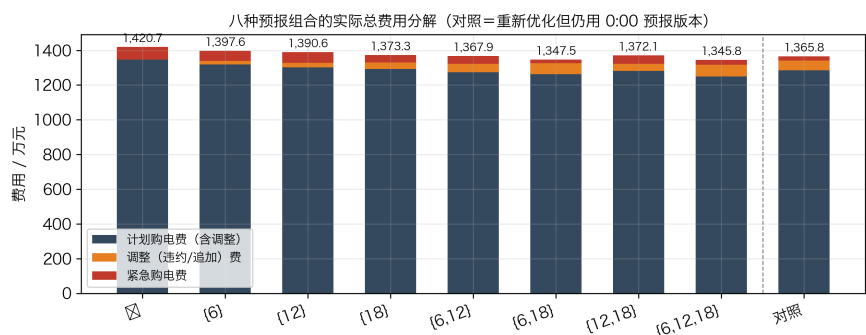


图 11 八种预报组合的实际总费用分解

其一，18:00 的预报最值得引入。单独加入时它是三者中改善最大的（省 47.44 万元），在 {6} 之上仍省 50.15 万元，在 {6, 12} 之上再加仍省 22.10 万元——即不会被其他时刻替代。这与表 16 的预报精度排序一致：18:00 版本的误差最小（全天电量偏差仅 461.3 kWh/日，相对日净负荷 0.86%），因为它覆盖的是发布后 1–24 小时，其中包含了次日凌晨的低谷时段。

需要留意一个例外：在已有 {12} 时，18:00 不是最有价值的那个——此时加入 6:00 可省 22.76 万元，而加入 18:00 只省 18.50 万元。这说明“哪个版本最值钱”取决于已经拥有哪些版本，边际价值不能脱离已有信息集来排序。

其二，6:00 的预报同样不可替代，在 {12, 18} 之上仍能省 26.36 万元；并且如上所述，它是补上 {12} 之后的首选。

其三，12:00 的预报边际价值很小。在 {18} 之上只省 1.17 万元、在 {6, 18} 之上只省 1.68 万元（约 0.12%），远小于它单独加入时的 30.12 万元。这说明 12:00 的信息与相邻两个时刻高度重复：它对当天下午的修正，6:00 已经预报过一次，18:00 又会覆盖相同目标时段。因此本文的建议是：应当引入 6:00 与 18:00 两个时刻的预报，12:00 可以省去——省去它全年只多花 1.68 万元，却少一次订阅与一次求解。

一个必须说明的反常现象：仅用 0:00 预报并不比问题二更省。表 12 第一行显示，只用 0:00 版本预报时的全年费用为 14207402.23 元，反而略高于问题二历史外推策略的 14021565.12 元。原因在表 16 中一目了然：附件 3 的 0:00 预报全天电量偏差为 3903.9 kWh/日，而问题二沿用的历史外推只有 3116.9 kWh/日——这一版预报本身就不如历史外推准。这说明“拿到了预报就更省”并不自动成立，只有发布的预报确实比已有的预测手段更准时它才有价值。这也界定了后文 5.28% 改善的归因：它由状态刷新与滚动重优化和新发布的预报版本两部分共同贡献（分解见本节末），而“仅仅拥有一份预报”本身并不自动带来任何改善。

信息收益与重新优化收益的分离。仅有“加入新预报更省钱”还不能说明是预报本身在起作用：在同样三个时刻重新求解、允许同样的调价动作、但继续沿用 0:00 版的光伏预报，得到对照策略的费用 13658321 元。于是

$$\underbrace{14207402 - 13658321}_{\text{重新优化收益}} = 549081 \text{ 元}, \quad \underbrace{13658321 - 13457698}_{\text{新信息收益}} = 200623 \text{ 元}.$$

对照策略的 α, ρ 同样由滚动标定选出，只是标定与执行都只用 0:00 版预报，即两条策略的参数各按自己的信息集定，对比才不掺入调参差异。还需说明对照并非“什么都不更新”：它同样在每个发布时刻以实际储电量为初值重排储能，也同样使用当天已经完成的时段对负载预测作日内比例修正——这两者都属于“用已有观测修正状态”，而不是“用新发布的预报”。因此上述 549081 元的对照差值应理解为“状态刷新与滚动重优化”的合计收益，200623 元才是新预报版本本身带来的增量。即总收益 749704.62 元中约 73.2% 来自“在每个时刻重新安排储能与购电”这一决策机制本身，26.8% 才来自新预报所提供的信息。这个分解很重要：它说明滚动重优化本身就有价值，不应把全部改善都归功于预报；同时也说明新预报的信息确实带来了可观的、额外的收益，引入它是合算的。

与理论阈值的对照。第 2.3 节由 1.5p 的调增价格推出临界分位数为 0.7，即只有当缺口概率超过 0.3 时才值得追加。上面的结果与之相容： $\emptyset$ 组合下只有 0:00 版预报，它的精度最差（全天电量偏差 3903.9 kWh/日，占日净负荷的 7.27%），缺口概率在多数时段都远高于 0.3，因此值得追加；而到了 18:00 时刻，预报误差已经很小（相对日净负荷 0.86%），多数时段的缺口概率降到 0.7 分位数以下，调整也就不再发生。这也解释了为什么版本越多、调整费越高，但总费用越低——追加只在它确实值得的时候才发生。

7.9 模型的边界

本文求解的是”风险余量 + 滚动调整 + 因果执行规则”的**可实现近似策略**，每个决策时点只使用当时已经发布的信息，因而不是未知未来下的全局最优解；参数标定是过去 35 天窗口内的近视最优；计费口径的歧义会影响调减的方向。**五条适用边界的完整讨论见附录 I。**

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于模型公式整理、求解代码编写与调试、结果校验脚本、论文部分文字的草拟与排版，详细使用情况见支撑材料。

参考文献

- [1] 全国大学生数学建模竞赛组委会. 2026 年高教社杯全国大学生数学建模竞赛 C 题：微网与外部电网电力调控策略 [Z]. 2026.
- [2] Virtanen P, Gommers R, Oliphant T E, et al. SciPy 1.0: fundamental algorithms for scientific computing in Python[J]. Nature Methods, 2020, 17(3): 261–272.
- [3] Huangfu Q, Hall J A J. Parallelizing the dual revised simplex method[J]. Mathematical Programming Computation, 2018, 10(1): 119–142.
- [4] 王成山, 武震, 李鹏. 微电网关键技术研究 [J]. 电工技术学报, 2014, 29(2): 1–12.
- [5] 李建林, 马会萌, 惠东. 储能技术融合分布式可再生能源的现状与发展趋势 [J]. 电工技术学报, 2016, 31(14): 1–10.
- [6] 席裕庚, 李德伟, 林姝. 模型预测控制——现状与挑战 [J]. 自动化学报, 2013, 39(3): 222–236.
- [7] Silver E A, Pyke D F, Peterson R. Inventory Management and Production Planning and Scheduling[M]. 3rd ed. New York: John Wiley & Sons, 1998.

A 问题一求解核心代码

1.1 参数、数据载入与时段校验

```
1 N = 144 # 一天 144 个 10 分钟时段
2 TAU = 10 / 60 # 每时段小时数 = 1/6 h
3 E_MIN, E_MAX = 1200.0, 10800.0 # 储电量运行下界/上界 kWh
4 E_INIT, P_MAX, ETA = 6000.0, 5000.0, 0.90
5 M_ENERGY = P_MAX * TAU # 单时段最大充放电量 M = 2500/3 kWh
6 ETA_SYM = float(np.sqrt(ETA)) # 若 90% 指往返效率, 单程对称取 sqrt(0.9)
7
8 def load_attach1():
9     """读取附件 1, 并校验时段标签的完整性与单调性。"""
10    df = pd.read_excel(ATTACH1, sheet_name="Sheet1")
11    df.columns = ["时间", "电价", "小区负载", "光伏发电预测功率"]
12    if len(df) != N:
13        raise ValueError("附件 1 应有 {N} 行, 实际 {len(df)} 行")
14    mins = np.array([to_minutes(v) for v in df["时间"]])
15    # 时间标签必须严格递增, 且恰为 {10,20,...,1440} 分钟
16    if not np.all(np.diff(mins) > 0):
17        raise ValueError("附件 1 时间标签非严格递增, 可能被重排或替换")
18    if list(mins) != list(range(10, 1441, 10)):
19        raise ValueError("附件 1 时间标签不是 {10,20,...,1440} 分钟")
20    # 第 t 行对应区间 [(t-1)*10min, t*10min)
21    df.insert(0, "区间序号", np.arange(N))
22    df["区间标签"] = [interval_label(i) for i in range(N)]
23    return df
```

1.2 模型装配与求解 (LP 松弛与完整 MILP)

```
1 # 变量分块: [g(0:144) | c(144:288) | d(288:432) | s(432:576) | E(576:720) | z]
2 iG, iC, iD, iS, iE, iZ = 0, N, 2 * N, 3 * N, 4 * N, 5 * N
3 nz = N if var.binary else 0
4 nvar = 5 * N + nz
5
6 # 目标: min J = sum_t p_t * g_t (g 已是电量, 不再乘 TAU)
7 cvec = np.zeros(nvar)
8 cvec[iG:iG + N] = price
9
10 # (1) 供需平衡 (等式 + 显式弃光):
11 # g_t + (v_t - s_t) + d_t = l_t + c_t
12 for i in range(N):
13     add_eq(i, [(iG + i, 1.0), (iD + i, 1.0),
14                (iC + i, -1.0), (iS + i, -1.0)], load_kwh[i] - pv_kwh[i])
15
16 # (2) 状态递推: E_t = E_{t-1} + eta_c*c_t - d_t/eta_d
17 for i in range(N):
18     coeffs = [(iE + i, 1.0)]
19     if i > 0:
20         coeffs.append((iE + i - 1, -1.0))
21         coeffs.append((iC + i, -ec))
22         coeffs.append((iD + i, 1.0 / ed))
23     add_eq(N + i, coeffs, E_INIT if i == 0 else 0.0)
24
25 # (3) 日循环: E_144 = E_0 = 6000
26 add_eq(2 * N, [(iE + N - 1, 1.0)], E_INIT)
27
28 # (4) 充放电互斥: c_t <= M*z_t, d_t <= M*(1-z_t) —— 仅 MILP
29 for i in range(N):
30     ub_rows += [2*i, 2*i]; ub_cols += [iC + i, iZ + i]
31     ub_vals += [1.0, -M_ENERGY]; ub_rhs.append(0.0)
```

```

32     ub_rows += [2*i+1, 2*i+1]; ub_cols += [iD + i, iZ + i]
33     ub_vals += [1.0, M_ENERGY];    ub_rhs.append(M_ENERGY)
34
35 # (5) 变量边界:  $g \geq 0$ ,  $0 \leq c, d \leq P\_MAX$ ,  $0 \leq s \leq pv$ ,  $E\_MIN \leq E \leq E\_MAX$ ,  $z \in \{0,1\}$ 
36 lb = np.concatenate([np.zeros(N), np.zeros(N), np.zeros(N),
37                      np.zeros(N), np.full(N, E_MIN), np.zeros(nz)])
38 ub = np.concatenate([np.full(N, np.inf), np.full(N, P_MAX), np.full(N, P_MAX),
39                      pv_kw.copy(), np.full(N, E_MAX), np.ones(nz)])
40 integrality = np.zeros(nvar)
41 if var.binary:
42     integrality[iZ:iZ + N] = 1          # 0 = 连续, 1 = 整数 (二元)
43
44 res = milp(cvec, constraints=[LinearConstraint(A_eq, b_eq, b_eq), ...],
45           integrality=integrality, bounds=Bounds(lb, ub))

```

1.3 解的正确性独立复核

```

1 def validate(sol, df, var, tol=1e-6):
2     """对最优解做独立复核, 返回问题清单 (空列表 = 全部通过)。"""
3     problems = []
4     # 1) 逐段供需平衡 (等式 + 显式弃光)
5     resid = sol.g + (pv_kwh - sol.s) + sol.d - load_kwh - sol.c
6     if np.abs(resid).max() > tol * max(1.0, load_kwh.max()):
7         problems.append(f"供需平衡残差过大: {np.abs(resid).max():.3e}")
8     # 2) 弃光量非负且不超过光伏出力
9     if sol.s.min() < -tol: problems.append("弃光量为负")
10    if (sol.s - pv_kwh).max() > tol: problems.append("弃光量超过光伏可用出力")
11    # 3) 充放电电量限值  $M = 2500/3$  kWh
12    if sol.c.max() > M_ENERGY + tol: problems.append("充电量超限")
13    if sol.d.max() > M_ENERGY + tol: problems.append("放电电量超限")
14    # 4) 储电量上下界 (145 个状态点)
15    Eseq = np.concatenate([sol.E0, sol.E])
16    if Eseq.min() < E_MIN - tol: problems.append("储电量低于下界")
17    if Eseq.max() > E_MAX + tol: problems.append("储电量高于上界")
18    # 5) 状态递推自洽
19    Erec = [sol.E0]
20    for i in range(N):
21        Erec.append(Erec[-1] + ec * sol.c[i] - sol.d[i] / ed)
22    if np.abs(np.array(Erec[1:]) - sol.E).max() > tol:
23        problems.append("储电量递推不自洽")
24    # 6) 日循环:  $E_{144} = E_0$ , 且  $\sum(d) = \eta_c \eta_d \sum(c)$ 
25    if abs(sol.E[-1] - sol.E0) > tol: problems.append("E144 != E0")
26    if abs(sol.d.sum() - ec * ed * sol.c.sum()) > tol * max(1.0, sol.d.sum()):
27        problems.append("sum(d) != eta_c*eta_d*sum(c)")
28    # 7) 充放电互斥 (题目未直接给出、需自证的物理约束)
29    if ((sol.c > tol) & (sol.d > tol)).any():
30        problems.append("存在区间同时充放电")
31    # 8) 费用复算
32    if abs(float(np.sum(price * sol.g)) - sol.obj) > tol * max(1.0, abs(sol.obj)):
33        problems.append("目标函数复算不符")
34    return problems
35
36 # LP 松弛与 MILP 的一致性判定
37 lp_both = int(((lp.c > 1e-9) & (lp.d > 1e-9)).sum())
38 gap = abs(milp_sol.obj - lp_sol.obj)
39 lp_is_optimal = (lp_both == 0) and (gap <= 1e-6 * max(1.0, abs(milp_sol.obj)))

```

B 问题一结果文件说明

result1.xlsx 按附件 5 模板的两个工作表组织:

- **计划购电量**：144 行，逐 10 分钟时段的购电量 (kWh)，时间标签统一为正文口径的”起始-结束”，自 0:00-0:10 至 23:50-0:00+1；
- **充放电量**：6 行 4 小时时段汇总的充电量、放电量 (kWh)，末段标签按题目表 2 记为 20:00-24:00，以及 0:00 与 24:00 的储电量 (kWh)，均为 6000.00。

此外，程序另输出内部结果明细表 `p1_detail.csv`，逐时段给出时段起止、电价、负载电量、光伏电量、购电量、充电量、放电量、弃光量、期初储电量、期末储电量与时段费用；论文表 1、表 2 与上述结果工作簿均由该明细表汇总得到，以避免多处分别计算导致口径不一致。

C 问题二、问题三的标定记录与校验明细

正文受篇幅限制，只保留了题目要求的表 1、表 2、表 3 格式结果与直接支撑结论的对照表；以下各表属于过程记录或汇总明细，结论均已在正文给出（问题二六项检查全部通过、问题三七项检查全部通过、正式区间总体结果已在正文逐项引用），故移到附录，数据一条不少。

表 13 问题二正式区间（2025 年 2 月 1 日–12 月 31 日）的总体结果与预测精度

项目	数值
正式区间	2025-02-01 至 2025-12-31（334 天）
计划购电量	21,701,732.99 kWh
计划购电费	13,358,737.95 元
紧急购电量	124,045.19 kWh
紧急购电费	662,827.17 元
合计购电量	21,825,778.18 kWh
合计购电费	14,021,565.12 元
紧急费占比	4.73%
弃用电量	2,704,833.51 kWh
期末储电量	1,687.84 kWh
负载预测 MAE	1,823.4 kWh/日（1.65%）
光伏预测 MAE	2,077.9 kWh/日（3.64%）
净负荷预测 MAE	3,116.9 kWh/日
净负荷被低估（逐时段）	49.1% 的时段，3,181.9 kWh/日
净负荷被高估（逐时段）	50.9% 的时段，3,190.5 kWh/日
风险余量为负的时段	3.7%（最小 -30.9、最大 241.1 kWh）

表 14 问题二六项强制检查的执行结果

检查项	内容	结果
信息时点	预测、误差分位数与参数仅用当天之前的数据；计划购电量在 0:00 后冻结	通过
物理可行性	逐段供需平衡；储电量在 [1200, 10800] kWh；充放电量 $\leq M$ 且同段互斥	通过
跨日连续性	次日 0:00 储电量严格等于前日 24:00 实际末值	通过
状态累计	$E_{n,144} - E_{n,0} = \eta_c \sum_t c_{n,t} - \frac{1}{\eta_d} \sum_t d_{n,t}$ 逐日成立	通过
费用一致性	用未舍入数据复算计划费与紧急费；弃电不退款；储备罚不计入账单	通过
汇总一致性	日期完整无错位；紧急事件合并前后电量一致	通过

表 15 滚动参数标定的选中结果（窗口 35 天，每 42 天重标定一次）

标定日	α	ρ	λ	窗口内选中策略
2025-02-01	0.80	1.0	0.00	$\alpha=0.80, \rho=1.0, \lambda=0.00$
2025-03-15	0.80	0.0	0.00	$\alpha=0.80, \rho=0.0, \lambda=0.00$
2025-04-26	0.80	0.0	0.00	$\alpha=0.80, \rho=0.0, \lambda=0.00$
2025-06-07	0.90	1.0	0.00	$\alpha=0.90, \rho=1.0, \lambda=0.00$
2025-07-19	0.80	0.0	0.00	$\alpha=0.80, \rho=0.0, \lambda=0.00$
2025-08-30	0.70	0.0	0.00	$\alpha=0.70, \rho=0.0, \lambda=0.00$
2025-10-11	0.80	0.0	0.00	$\alpha=0.80, \rho=0.0, \lambda=0.00$
2025-11-22	0.80	0.0	0.00	$\alpha=0.80, \rho=0.0, \lambda=0.00$
全年共标定 8 次： $\alpha = 0.70$ 1 次、 $\alpha = 0.80$ 6 次、 $\alpha = 0.90$ 1 次； λ 在所有标定中均取 0.00				

表 16 各发布版本在 2025 年 2 月 1 日–12 月 31 日的净负荷预测误差（只统计发布时刻之后的时段）

预报版本	逐时段绝对偏差之和/kWh	全天电量偏差/kWh	符号偏差/kWh	相对日净负荷
0:00 发布	7,462.0	3,903.9	-769.1	7.27%
6:00 发布	4,730.2	2,110.0	-215.6	3.93%
12:00 发布	2,557.4	1,197.7	+667.7	2.23%
18:00 发布	999.5	461.3	+100.5	0.86%
问题二历史外推对照	6,372.4	3,116.9	-8.5	5.80%

注：“逐时段绝对偏差之和”为 $\sum_t |N_t - \hat{N}_t|$ ，衡量十分钟尺度的错位；“全天电量偏差”为 $|\sum_t (N_t - \hat{N}_t)|$ ，衡量日总量口径，二者不可混用。符号偏差为正表示净负荷被低估。

表 17 框架要求的七项检查在主策略全部 334 个交付日上的实测结果

检查项	实测最坏值	未通过天数
信息可用性	结构上恒成立	0
时间转换	结构上恒成立	0
计划锁定	$1.02e - 12$ kWh	0
费用核对	$4.55e - 13$ 元	0
储能与供电	$2.27e - 13$ kWh	0
状态衔接	$1.91e - 11$ kWh	0
结果一致性	结构上恒成立	0

注：另有储电量越界最大量 $0.00e + 00$ kWh、充放电越限最大量 $0.00e + 00$ kWh、同时充放电时段数 0，均并入“储能与供电”一项。

D 问题二求解核心代码

问题二的主计算由 03_ 代码/p2_microgrid.py 完成，共约 1150 行；表格由 03_ 代码/p2_tables.py 从结果文件自动生成， α 敏感性验证由 03_ 代码/p2_alphasweep.py 完成，三者均为可独立运行的全部源码，随支撑材料一并提交。下面按“预测层 → 风险层 → 计划层 → 执行层 → 标定层 → 校验层”依次摘录 p2_microgrid.py 中的核心片段。

4.1 因果预测器与净负荷预测误差表

预测器只允许访问第 n 天之前的历史日，因此“信息时点”要求由数据结构本身保证，而不是靠调用约定。净负荷预测误差表 $\varepsilon_{n,t}$ 与 α, ρ, λ 无关，故全流程只预计算一次。

```

1 class ForecastParams:
2     """近期加权历史预测的超参数（框架文档 3.2 节）。"""
3     window: int = 15          # 回看窗口天数
4     decay: float = 0.30       # 每 7 天的权重衰减因子
5     weekday_only: bool = False # 是否只用同一星期的历史
6     name: str = "负荷"
7
8 # 在正式区间开始前选定并冻结的预测超参数，回放中不再调整。
9 LOAD_FORECAST = ForecastParams(window=15, decay=0.30, weekday_only=True, name="负荷")
10 PV_FORECAST = ForecastParams(window=5, decay=0.50, weekday_only=False, name="光伏")
11
12 def forecast_weights(n, fp):
13     """返回第 n 天预测所用的（历史日下标，归一化权重），只取 i < n。"""
14     lo = max(0, n - fp.window)
15     idx = np.arange(lo, n)
16     if len(idx) == 0:
17         return idx, np.zeros(0)
18     w = fp.decay ** ((n - 1 - idx) / 7.0)
19     if fp.weekday_only:
20         same = np.array([weekday_of(i) == weekday_of(n) for i in idx])
21         if same.any():
22             w = w * same
23     s = w.sum()
24     if s <= 0:
25         return idx, np.full(len(idx), 1.0 / len(idx))
26     return idx, w / s
27
28 def build_error_table(LOAD, PV, fo):
29     """预计算 eps[n, t] = N[n, t] - Nhat[n, t | 日期 < n]。
30
31     这是“逐日向前预测后得到的误差”，不是模型在训练数据上的拟合残差。
32     第 0 天无历史，记为 NaN 并永不使用。
33     """
34     eps = np.full((N_DAY, N), np.nan)
35     for n in range(1, N_DAY):
36         _, _, nhat = fo.energy(n)
37         eps[n, :] = (LOAD[n, :] - PV[n, :]) * TAU - nhat

```

4.2 风险余量：误差的经验分位数

```

1 def risk_margin(eps, n, rp):
2     """第 n 天各时段的风险余量  $Q_{\{\alpha,n,t\}}$  (kWh) 。
3
4     只取 n 之前已结束日期的误差；样本不足时合并相邻时段 ( $\pm 2$  个 10 分钟)，
5     仍然只用过去数据，绝不回填同日或未来信息。
6
7     if rp.alpha is None or n == 0:
8         return np.zeros(N)
9     lo = max(0, n - rp.window)
10    hist = eps[lo:n, :]
11    hist = hist[~np.isnan(hist).all(axis=1)]
12    k = hist.shape[0]
13    if k >= rp.min_samples:
14        return np.quantile(hist, rp.alpha, axis=0)
15    if k >= 3:
16        pooled = np.empty((k, N))
17        for t in range(N):
18            a, b = max(0, t - 2), min(N, t + 3)
19            pooled[:, t] = hist[:, a:b].mean(axis=1)
20        return np.quantile(pooled, rp.alpha, axis=0)
21    return np.zeros(N)

```

4.3 日前 MILP 的装配与求解

决策变量 $x = (g, c, d, w, \bar{E}, z, \xi)$ 共 $6N + 1 = 865$ 个，其中 $z \in \{0, 1\}$ ¹⁴⁴。系数矩阵与日期无关，故在模块加载时装配一次 (`_precomp`)，逐日只替换右端项，单日求解耗时约 46 ms。

```

1 def _precomp():
2     """装配与日期无关的 MILP 结构，逐日只更换右端项，显著加速。"""
3     nvar = 6 * N + 1 # g | c | d | w | E | z | xi
4     iG, iC, iD, iW, iE, iZ, iXi = 0, N, 2*N, 3*N, 4*N, 5*N, 6*N
5
6     # (1) 风险净负荷平衡:  $g_t + d_t - c_t - w_t = N_{\text{tilde}_t}$  行 0..N-1
7     # (2) 状态递推:  $E_t - E_{t-1} - \eta_c c_t + d_t / \eta_d = 0$  行 N..2N-1
8     # (3) 充放电互斥:  $c_i - M z_i \leq 0, d_i + M z_i \leq M$  2N 条
9     # (4) 日末储备:  $-xi - E_{N-1} \leq -E_{\text{tar}}$  1 条
10    A_eq = csr_matrix((vals, (rows, cols)), shape=(2 * N, nvar))
11    A_ub = csr_matrix((uvals, (urows, ucols)), shape=(2 * N + 1, nvar))
12
13    lb = np.concatenate([np.zeros(N), np.zeros(N), np.zeros(N), np.zeros(N),
14                          np.full(N, E_MIN), np.zeros(N), [0.0]])
15    ub = np.concatenate([np.full(N, np.inf), np.full(N, M_ENERGY),
16                          np.full(N, M_ENERGY), np.full(N, np.inf),
17                          np.full(N, E_MAX), np.ones(N), [np.inf]])
18
19    integ = np.zeros(nvar)
20    integ[iZ:iZ + N] = 1 # 只有 z 是整数
21    return {"nvar": nvar, "A_eq": A_eq, "A_ub": A_ub, "lb": lb, "ub": ub,
22           "integ": integ, "iG": iG, "iC": iC, "iD": iD, "iW": iW,
23           "iE": iE, "iXi": iXi}
24
25 _PC = _precomp()
26
27 def solve_dayahead(ntilde, e0, price, rp, etar=E_TAR):
28     """求解某一天的日前 MILP:
29         min  $\sum_t p_t g_t + \lambda * xi, \quad xi \geq E_{\text{tar}} - E_{\text{bar}}_{144}, \quad xi \geq 0$ 

```

```

29 """
30 b_eq = np.empty(2 * N)
31 b_eq[:N] = ntilde # Ntilde_t (风险净负荷)
32 b_eq[N] = e0 # E_0 = 当日实际日初储电量
33 b_eq[N + 1:] = 0.0
34
35 b_ub = np.empty(2 * N + 1)
36 b_ub[:2 * N] = np.tile([0.0, M_ENERGY], N)
37 b_ub[2 * N] = -etar # 日末储备约束的右端项
38
39 cvec = np.zeros(nvar)
40 cvec[iG:iG + N] = price # 计划购电按分时电价计费
41 cvec[iXi] = rp.lam # 储备不足的规划惩罚 (不计入账单)
42
43 res = milp(cvec,
44            constraints=[LinearConstraint(_PC["A_eq"], b_eq, b_eq),
45                          LinearConstraint(_PC["A_ub"], -np.inf, b_ub)],
46            integrality=_PC["integ"], bounds=Bounds(_PC["lb"], _PC["ub"]))
47 if not res.success:
48     raise RuntimeError(f"日前 MILP 求解失败: {res.message}")
49 ...

```

4.4 实时执行规则

执行层的核心特征是**因果性**：第 t 段的动作只依赖 t 时刻已经观测到的实际负载、实际光伏与当前储电量，购电量 g_t 由日前冻结、执行层无修改入口。

```

1 def execute_day(plan_g, Ebar, l_act, v_act, e0, price, rp,
2                 rule="greedy", allow_storage=True):
3     """实时执行：购电计划固定，储能按当前已观测情况因果响应。
4
5     b_t = g_t + v_t - l_t 为使用储能前的电量盈余 (kWh)。
6     b > 0 用盈余充电 (充不进去的弃用)；b < 0 在储备线之上放电，
7     剩余缺口按 5 倍电价紧急购电。
8     """
9     for t in range(N):
10         b = plan_g[t] + v_act[t] - l_act[t]
11         R = E_MIN + rp.rho * (Ebar[t] - E_MIN) # 0:00 即可算出的储备线
12
13         if b > 0:
14             bpos = min(b, M_ENERGY)
15             c[t] = min(bpos, (E_MAX - Ep) / ETA) # 盈余全部用于充电
16             w[t] = b - c[t] # 充不进去的弃用 (无收益)
17         else:
18             head = max(0.0, Ep - R) # 电量不得低于储备线
19             d[t] = min(-b, M_ENERGY, ETA * head)
20             r[t] = max(0.0, -b - d[t]) # 仍缺的部分紧急购电
21
22         Ep = Ep + ETA * c[t] - d[t] / ETA
23         E[t] = Ep
24
25     cost_plan = float(np.sum(price * plan_g))
26     cost_emg = float(np.sum(5.0 * price * r))
27     return DayExec(g=plan_g.copy(), c=c, d=d, r=r, w=w, E=E, E0=float(e0),
28                   cost_plan=cost_plan, cost_emg=cost_emg)

```

4.5 滚动参数标定

标定在第 n_0 天的 0:00 进行，只用窗口内**已经结束**的历史日回放，比较包含五倍紧急购电费的**实际总费用**，取最小者。

```

1 class CalConfig:
2     alphas: tuple = (0.50, 0.60, 0.70, 0.80, 0.90)
3     rhos: tuple = (0.0, 1.0)
4     lams: tuple = (0.0, 0.60)
5     window: int = 35 # 标定回放窗口天数
6     every: int = 42 # 每隔多少天重新标定一次
7     start: int = REPORT_START # 首次标定发生在正式区间第一天
8
9     def grid(self):
10         return [RiskParams(alpha=a, rho=r, lam=l)
11                 for a in self.alphas for r in self.rhos for l in self.lams]
12
13 def calibrate(sim, n0, e_at_window_start, cal):
14     """在第 n0 天 0:00 用**此前**数据选择参数。"""
15     lo = max(0, n0 - cal.window)
16     best, best_cost = None, np.inf
17     for rp in cal.grid():
18         cost, _, _ = sim.replay(lo, n0, rp, e_at_window_start, cal.rule,
19                                 cal.allow_storage)
20         if cost < best_cost - 1e-9:
21             best_cost, best = cost, rp
22     return best, rows
23
24 def run_strategy(sim, cal, verbose=True):
25     """从 2025-01-01 的 6000 kWh 出发跑完整年，期间滚动标定参数。"""
26     for n in range(N_DAY):
27         if n >= cal.start and (n - cal.start) % cal.every == 0:
28             params, _ = calibrate(
29                 sim, n, e_at_day_start[max(0, n - cal.window)], cal)
30             rec = sim.run_day(n, params, e, cal.rule, cal.allow_storage)
31             records.append(rec)
32             e = rec.E_end # 跨日连续: 次日 0:00 = 前日 24:00
33             e_at_day_start[n + 1] = e
34     return records, cal_rows

```

4.6 六项强制检查的独立复核

结果不依赖求解器自报的可行性，而是用未四舍五入的数据逐日独立复算。

```

1 def validate_day(rec, tol=1e-6):
2     """逐日复核实际运行结果，返回问题清单（空 = 通过）。框架文档 7.2 节。"""
3     p = []
4     ex, l, v = rec.exec, rec.l_act, rec.v_act
5
6     # 1) 实际逐段供需平衡:  $g + r + v + d = l + c + w$ 
7     resid = ex.g + ex.r + v + ex.d - l - ex.c - ex.w
8     scale = max(1.0, float(l.max()))
9     if np.abs(resid).max() > tol * scale:
10         p.append(f"供需平衡残差 {np.abs(resid).max():.3e} kWh")
11
12     # 2) 物理边界: 非负、不超 M、同段不同时充放电
13     if ((ex.c > tol) & (ex.d > tol)).any():
14         p.append("存在同时充放电的时段")
15
16     # 3) 储电量区间 (含日初, 共 145 个状态点)
17     Eseq = np.concatenate([[ex.E0], ex.E])
18     if Eseq.min() < E_MIN - tol or Eseq.max() > E_MAX + tol:
19         p.append("储电量越界")
20
21     # 4) 状态累计 (问题二不能套用问题一的  $\sum d = 0.81 \sum c$ )
22     lhs = ex.E[-1] - ex.E0
23     rhs = ETA * ex.c.sum() - ex.d.sum() / ETA

```

```

24 if abs(lhs - rhs) > tol * max(1.0, abs(rhs)):
25     p.append(f"状态累计不符: dE={lhs:.6f} vs {rhs:.6f}")
26
27 # 5) 费用一致性: 弃电不退款, 储备罚不混入实际账单
28 cp = float(np.sum(_PRICE_REF * ex.g))
29 ce = float(np.sum(5.0 * _PRICE_REF * ex.r))
30 if abs(cp - ex.cost_plan) > tol * max(1.0, abs(cp)):
31     p.append("计划费复算不符")
32 if abs(ce - ex.cost_emg) > tol * max(1.0, abs(ce)):
33     p.append("紧急费复算不符")
34
35 # 6) 计划购电量在 0:00 后冻结 (由架构保证, 此处做存在性校验)
36 if ex.g.shape != (N,) or ex.g.min() < -tol:
37     p.append("计划购电量非法")
38 return p

```

E 问题二结果文件说明

result2.xlsx 严格按附件 5 模板的三张表组织, 并额外增加一张全天汇总表:

- **表 1 (计划购电量):** 334 行 × 144 列, 逐 10 分钟时段的计划购电量 (kWh), 覆盖 2025 年 2 月 1 日至 12 月 31 日;
- **表 2 (充放电电量):** 334 行 × 6 个 4 小时时段, 给出充电量、放电量 (kWh) 以及 0:00 与 24:00 的储电量 (kWh);
- **表 3 (紧急购电量):** 按“同日连续发生紧急购电的时段合并为一个事件”的规则生成, 共 455 个事件行;
- **全天汇总 (额外):** 逐日给出计划购电量、紧急购电量、弃用电量、期初/期末储电量与计划费、紧急费, 便于核对正文表 13。

时间标签统一采用正文口径的“起始-结束”, 自 0:00-0:10 至 23:50-0:00+1。模板原始标签比该口径整体偏移一格, 故结果工作簿由程序从模板复制后**重写**标签列, 而非直接填充模板原列。

此外, 程序另输出四个内部结果文件, 论文中的每个数字均可追溯到其中之一:

- **p2_detail.csv:** 逐时段明细 (334 × 144 = 48096 行), 含电价、预测负载/光伏、净负荷预测、风险余量、风险净负荷、参考储电量、实际负载/光伏、计划购电、实际充放电、紧急购电、弃电、期初期末储电量与时段费用;
- **p2_daily.csv:** 逐日汇总 (334 行), 含全天电量、费用、储电量、紧急事件个数与当日生效参数;
- **p2_calibration.csv:** 8 次滚动标定 × 20 个参数组合的全部回放费用, 即表 15 的原始记录;
- **p2_results.json:** 正文全部汇总数字与六项检查结果的机器可读版本。表 13-15 由 p2_tables.py 直接读取该文件与 p2_detail.csv 自动生成 (p2_tables.tex), **不做任何手工转录**, 以保证论文与结果文件严格同源;
- **p2_alpha_sensitivity.json:** 正文 §6.9.6 弃电成因讨论中 α 敏感性验证的逐点结果 ($\alpha \in \{0.50, 0.70, 0.80, 0.90\}$ 各自的弃电量、紧急购电量与总费用), 由 p2_alphasweep.py 在与主策略完全相同的预测器、执行规则与评价区间下另行重跑得到。

F 问题三求解核心代码

问题三的主计算由 03_ 代码/p3_microgrid.py 完成，共约 1400 行，一次性产出 result3.xlsx 与 p3_results.json 等全部结果文件；论文表格由 03_ 代码/p3_tables.py 从结果文件自动生成，插图由 03_ 代码/p3_figures.py 生成，正文引用的附件 3 时间口径判定由 03_ 代码/p3_alignment_scan.py 独立复算。以上四个脚本均为可独立运行的全部源码，不含任何手工填数，随支撑材料一并提交。下面按”预报口径 → 调整模型 → 滚动回放 → 校验 → 组合比较”依次摘录核心片段。

6.1 附件 3 的载入：整点功率判定、首端锚点与长表

附件 3 的 24 个值是目标整点的瞬时功率，因此内部索引 0 表示发布时刻本身、1-24 表示发布后 1-24 小时。发布时刻自身的值附件没有给出，代码按”取更早发布版本对该时刻的预报”补齐，并记录来源以便审计。

```
1 F = np.zeros((N_RELEASE, N_DAY, 25))
2 F[:, :, 1:] = power.reshape(N_DAY, N_RELEASE, 24).transpose(1, 0, 2)
3
4 # ---- 首端锚点  $F^0(\tau)$ 
5 for n in range(N_DAY):
6     for hi in range(N_RELEASE):
7         if hi > 0:
8             F[hi, n, 0] = F[hi - 1, n, 6]      # 同一更早版本的"预报 6 小时"
9         elif n > 0:
10            F[0, n, 0] = F[3, n - 1, 6]        # 前一日 18:00 版本
11        else:
12            F[0, n, 0] = 0.0    # 2025-01-01 0:00 无更早版本；该时刻实际光伏为 0
```

6.2 时间转换：分段线性插值与十分钟电量

十分钟区间的两端都与整点对齐，因此区间上插值曲线的积分恰好等于两端点的算术平均乘区间长度，无需数值积分。

```
1 def _interp_hour(A: np.ndarray, rel_min: float) -> float:
2     # A[j] 是发布后 j 小时的预测功率 (kW)，节点间隔 60 min
3     if rel_min <= 0.0:
4         return float(A[0])
5     if rel_min >= 1440.0:
6         return float(A[24])
7     j = int(rel_min // 60)
8     th = (rel_min - 60.0 * j) / 60.0
9     return float((1.0 - th) * A[j] + th * A[j + 1])
10
11 def forecast_energy_slots(fp, hi: int, n: int) -> np.ndarray:
12     # 发布 hi 版对第 n 天各十分钟时段的光伏电量；发布前的时段为 NaN
13     t0 = 36 * hi
14     out = np.full(N, np.nan)
15     A = fp.F[hi, n, :]
16     rel0 = 60 * 6 * hi      # 发布时刻相对 0:00 的分钟数
17     for t in range(t0, N):
18         ra = t * 10.0 - rel0
19         rb = ra + 10.0
20         out[t] = 0.5 * (_interp_hour(A, ra) + _interp_hour(A, rb)) * TAU
21     return out
```

6.3 调整 MILP：两种计费口径的统一线性化

两种口径共用同一个目标函数，区别只在变量下界：主口径令 $x_t \geq g_t^0$ 并把下调量上界锁为 0，退款口径放开双向。由于同时抬高 a_t, b_t 只会多付费，最优解中二者不会同时为正，故不需要引入二元变量。

```
1 lb, ub = d["lb"].copy(), d["ub"].copy()
2 if convention == "main":
3     lb[iY:iY + m] = g0_rem      #  $x_t \geq g_t^0$ , a 自动等于上调量
4     ub[iB:iB + m] = 0.0        # 禁掉下调量
5 else:
6     lb[iY:iY + m] = 0.0        # 退款口径允许双向调整
7
8 cvec = np.zeros(d["nvar"])
9 cvec[iA:iA + m] = COEF_UP * price_rem      # 上调: 1.5 p
10 cvec[iB:iB + m] = -COEF_DOWN * price_rem   # 退款口径下这是收益: -0.5 p
11 cvec[iR:iR + m] = COEF_EMG * price_rem     # 紧急: 5 p
12 cvec[iXi] = rp.lam                        # 日末储备约束的惩罚
13
14 res = milp(cvec,
15            constraints=[LinearConstraint(d["A_eq"], b_eq, b_eq),
16                                   LinearConstraint(d["A_ub"], -np.inf, b_ub)],
17            integrality=d["integ"], bounds=Bounds(lb, ub))
```

6.4 滚动回放：只提交当前交付块

每个交付块的起点到达正式发布时刻时用当前实际储电量重新求解；只把本块的 y_t 写回生效购电量 x_t ，参考轨迹整体刷新，更远时段一律丢弃。

```
1 for hi, (a, b) in enumerate(BLOCK_BOUNDS):
2     if hi > 0 and RELEASE_HOURS[hi] in S:
3         efor = hi if use_new_forecast else 0      # 对照策略仍用 0:00 版本
4         l_hat_h = load_forecast_at(l_hat_day, l_act, hi)
5         v_hat_h = forecast_energy_slots(fp, efor, n)
6         n_risk_h = (l_hat_h - v_hat_h) + risk_margin3(eps, efor, n, params.alpha)
7         res = solve_adjust(n_risk_h[a:], g0[a:], runner.Ep, price[a:],
8                           params, convention)
9         x[a:b] = res.y[:b - a]      # 只正式提交当前交付块
10        Ebar[a:] = res.Ebar          # 参考轨迹整体刷新（含展望部分）
11        n_solve += 1
12        n_submit += 1
13        runner.run(a, b, x, Ebar)
```

6.5 七项强制检查的独立复核

校验函数不复用求解时的任何中间量，而是从最终账本重新按定义计算，逐日返回七类问题清单；只要任一天出现任一条，全年结果即视为不可用。

```
1 # 5. 储能与供电：逐段能量守恒、边界、互斥
2 resid = rec.x + rec.r + v + rec.d - l - rec.c - rec.w
3 if np.abs(resid).max() > tol * max(1.0, float(l.max())):
4     power.append("供需平衡残差 %.3e kWh" % np.abs(resid).max())
5 Eseq = np.concatenate([[rec.E0], rec.E])
6 if Eseq.min() < E_MIN - tol or Eseq.max() > E_MAX + tol:
7     power.append("储电量越界 [%.4f, %.4f]" % (Eseq.min(), Eseq.max()))
8
9 # 3. 计划锁定：主口径下必须恒有  $x_t \geq g_t^0$ 
10 if convention == "main" and (rec.x - rec.g0).min() < -tol:
11     lock.append("主口径下出现下调 %.3e kWh" % (rec.x - rec.g0).min())
12
```

```

13 # 6. 状态衔接: 日末 = 日初 + eta*sum(c) - sum(d)/eta
14 if abs((rec.E[-1] - rec.E0) - (ETA * rec.c.sum() - rec.d.sum() / ETA)) \
15     > tol * max(1.0, abs(ETA * rec.c.sum())):
16     chain.append("状态累计不符")
17
18 # 7. 结果一致性: 出现差异的时段必须落在当天正式提交过的交付块内
19 allowed = np.zeros(N, bool)
20 for h in rec.S:
21     allowed[36 * (h // 6):36 * (h // 6 + 1)] = True
22 stray = (np.abs(rec.x - rec.g0) > tol) & ~allowed

```

6.6 八种组合的比较与收益分解

十个策略并行回放，每个策略在它自己的组合上标定参数，保证比较公平。

```

1 tasks = [{"tag": combo_name(S), "S": S, "use_new": True,
2           "convention": "main"} for S in all_combos()]
3 tasks.append({"tag": CTRL_TAG, "S": tuple(ALL_S), "use_new": False,
4               "convention": "main"}) # 重新优化但沿用 0:00 预报版本
5 tasks.append({"tag": REFUND_TAG, "S": tuple(ALL_S), "use_new": True,
6               "convention": "refund"}) # 计费口径敏感性
7
8 info_gain = ctrl["合计费用_元"] - save(MAIN_TAG) # 新预报信息的收益
9 reopt_gain = base - ctrl["合计费用_元"] # 重新优化的收益

```

6.7 完整源程序

以上各段是为便于对照讲解而作的分段摘录；规范第五条要求附录给出“建模所用到的全部完整、可运行的源程序代码”。为免转录出错，下面用 `\lstinputlisting` 直接引入 03_ 代码/ 下的四个脚本，与随支撑材料提交的文件逐字节相同，不含任何手工改动；列出的是不经裁剪的全文，因此其中自然包含上面已讲解过的片段。问题一、问题二所用的源代码同样随支撑材料一并提交。

(1) p3_microgrid.py: 问题三主求解器

```
1 # -*- coding: utf-8 -*-
2 r"""
3 2026 年高教社杯全国大学生数学建模竞赛 C 题
4 微网与外部电网电力调控策略 —— 问题三
5
6 多次预报驱动的滚动购电与储能调度模型
7 -----
8 实现依据：《问题三_解题思路与实现框架》
9 01_题目/C题/问题三_解题思路与实现框架.md
10
11 与问题二的差别只在**信息来源**与**计费口径**两处，物理模型完全一致：
12
13 * 信息来源：光伏不再由历史外推预测，而是直接使用附件 3 中当日已发布的
14 预报版本；每天 0:00/6:00/12:00/18:00 各有一个版本，每个版本覆盖发布
15 时刻之后的 24 个整点。负载仍在历史外推的基础上叠加"当日已观测时段"的
16 日内比例修正。
17 * 计费口径（框架 3.2 节主口径）：计划费  $\Sigma p_g$  全额照付，调整费另计
18  $A_t = 1.5p(x_t - g_t)_+ + 0.5p(g_t - x_t)_+$ ，紧急费  $\Sigma 5p_r$ 。允许无偿弃电时
19 调减没有费用优势，因此主模型直接令  $x_t = g_t$ ， $A_t$  退化为  $1.5p(x_t - g_t)_+$ 。
20 框架 3.3 节的"退款口径"  $J^{\text{refund}}$  作为独立敏感性检查，由 solve_adjust /
21 run_day3 / replay3 的 convention 参数 ("main" / "refund") 切换；主流程
22 以 REFUND_TAG 追加一个 convention="refund" 的策略并行回放。
23
24 三个量必须分清（框架 3.1 节）：
25 g_t 0:00 冻结的初始计划购电量，永久保存，不被后续调整覆盖；
26 x_t 该时段**最终生效的完整购电量**（不是增量、不是多次版本之和）；
27 r_t 实际发生的紧急购电量。
28 最终收到的常规电量与紧急电量之和为  $G^{\text{received}} = G^{\text{effective}} + G^{\text{emg}}$ 。
29
30 交付块策略（框架 3.4 节）：每个六小时交付块只在块首正式调整一次，
31 0:00 提交全天 144 段，6:00 提交 6:00—12:00，12:00 提交 12:00—18:00，
32 18:00 提交 18:00—24:00。更远时段在优化里作为**展望值**参与，但不提交、
33 不收费、不写入最终调整结果。
34
35 时间纪律（框架 8.2 节第 1 项）：任何求解只使用"发布时刻不晚于决策时刻"的
36 预报版本，以及"目标时段已经过去"的实际观测；误差样本同样满足该条件。
37 -----
38 """
39
40 from __future__ import annotations
41
42 import argparse
43 import json
44 import sys
45 import time
46 from dataclasses import dataclass
47 from itertools import combinations
48 from pathlib import Path
49
50 import numpy as np
51 import pandas as pd
52 from scipy.optimize import Bounds, LinearConstraint, milp
```

```

53 from scipy.sparse import csr_matrix
54
55 sys.path.insert(0, str(Path(__file__).resolve().parent))
56 from p1_microgrid import ( # noqa: E402
57     N, TAU, E_MIN, E_MAX, E_INIT, M_ENERGY, ETA,
58     interval_label, fmt_time, block_label,
59 )
60 from p2_microgrid import ( # noqa: E402
61     ROOT, N_DAY, REPORT_START, REPORT_END, SPECIAL_DATES, WANT_LABELS,
62     EMG_EPS, E_TAR, Forecaster, RiskParams, load_attach2, solve_dayahead,
63     forecast_weights,
64 )
65
66 # ----- 路径
67 ATTACH3 = ROOT / "01_题目" / "C题" / "附件" / "附件3.xlsx"
68 TEMPLATE3 = ROOT / "01_题目" / "C题" / "附件" / "附件5" / "result3.xlsx"
69 OUT_DIR = ROOT / "06_支撑材料"
70 OUT_XLSX3 = OUT_DIR / "result3.xlsx"
71 OUT_JSON3 = OUT_DIR / "p3_results.json"
72 OUT_DETAIL3 = OUT_DIR / "p3_detail.csv"
73 OUT_DAILY3 = OUT_DIR / "p3_daily.csv"
74 OUT_COMBOS3 = OUT_DIR / "p3_combinations.csv"
75 # 每个策略求解完成后立刻把汇总结果落盘。整轮比较要跑四十多分钟（大头是
76 # 滚动标定的上万次 MILP），若不缓存，一旦后续写文件出错就全部作废。
77 CACHE_DIR = OUT_DIR / "p3_cache"
78
79 # ----- 常量
80 RELEASE_HOURS = (0, 6, 12, 18) # 每天四次发布的整点
81 N_RELEASE = 4
82 BLOCK_BOUNDS = [(0, 36), (36, 72), (72, 108), (108, 144)] # 四个六小时交付块
83 FOUR_HOUR_BLOCKS = [(0, 24), (24, 48), (48, 72), (72, 96), (96, 120),
84                     (120, 144)]
85 ALL_S = (6, 12, 18) # 0:00 之外可选的三个发布时刻
86
87 INTRADAY_LOAD = True # 负载是否启用日内比例修正
88 MIN_KAPPA_SAMPLES = 6 # 开启日内修正所需的最少已观测时段数
89
90 # 主计费口径下追加电量的价格系数（1.5 倍总价，不是先付原价再加 1.5 倍）
91 COEF_UP = 1.5
92 # 退款口径下调减的净效果：退回 p 再付 0.5p，故为 -0.5p
93 COEF_DOWN = 0.5
94 # 紧急购电的额外价格系数
95 COEF_EMG = 5.0
96
97 # 敏感性检查的计费口径： "main" = 框架 3.2 主口径； "refund" = 框架 3.3 退款口径
98 CONVENTIONS = ("main", "refund")
99
100 # ===== 模块 1：数据与版本整理
101 @dataclass
102 class ForecastPanel:
103     """附件 3 的长表化结果与按版本索引的预报值。
104
105     F[hi, n, j] 是第 n 天第 hi 次发布对"发布后 j 小时"的预测功率 (kW)。
106     j = 1..24 直接来自附件 3 的 24 列；j = 0 是发布时刻本身的锚点，附件
107     未提供，按框架 2.3 节的约定由**同一时刻早先版本**的预报补齐：
108         hi = 1,2,3 → 当天上一次发布的"预报 6 小时"（正好指向本次发布时刻）
109         hi = 0 → 前一日 18:00 发布的"预报 6 小时"（指向当日 0:00）
110         n = 0 → 无更早版本，取 0（全年 0:00—0:10 的实际光伏恒为 0）
111     """
112
113     F: np.ndarray # (4, N_DAY, 25)
114     long: pd.DataFrame # 长表：日期 / 发布时刻 / 目标时间 / 提前时长 / 功率
115     anchor_source: np.ndarray # (4, N_DAY) 每个版本的锚点来源，供核对

```

```

117
118
119 def load_attach3() -> ForecastPanel:
120     """读取附件 3，向下补齐日期，展开为长表并建立版本索引。"""
121     raw = pd.read_excel(ATTACH3, sheet_name=0)
122     if raw.shape != (N_DAY * N_RELEASE, 2 + 24):
123         raise ValueError(f"附件 3 形状应为 {(N_DAY * N_RELEASE, 26)}，实际 {raw.shape}")
124
125     # 日期列只有每天第一行有值，其余为空；只对**日期**列向下补齐，
126     # 功率列绝不做任何填充（框架 2.1 节的明确要求）。
127     day_col = raw.iloc[:, 0].astype("object").copy()
128     filled, cur = [], None
129     for v in day_col:
130         if isinstance(v, str) and v.strip() not in ("", "nan"):
131             cur = pd.Timestamp(v)
132         elif v is not None and not (isinstance(v, float) and np.isnan(v)):
133             cur = pd.Timestamp(v)
134         if cur is None:
135             raise ValueError("附件 3 首个数据行缺少日期")
136         filled.append(cur)
137
138     dates = pd.DatetimeIndex(filled)
139     if dates.nunique() != N_DAY:
140         raise ValueError(f"附件 3 日期应有 {N_DAY} 个不同值，实际 {dates.nunique()}")
141     if dates[0] != pd.Timestamp("2025-01-01") or dates[-1] != pd.Timestamp("2025-12-31"):
142         raise ValueError(f"附件 3 日期区间应为 2025-01-01~2025-12-31，实际 "
143             f"{dates[0].date()}~{dates[-1].date()}")
144
145     rel_col = raw.iloc[:, 1].astype(str).str.strip()
146     for k in range(N_RELEASE):
147         blk = rel_col.iloc[k:N_RELEASE].unique()
148         if list(blk) != [f"{RELEASE_HOURS[k]}:00"] and \
149             list(blk) != [f"{RELEASE_HOURS[k]:02d}:00"]:
150             raise ValueError(f"附件 3 第 {k} 行的发布时刻应恒为 "
151                 f"{RELEASE_HOURS[k]}:00，实际 {blk}")
152
153     power = raw.iloc[:, 2:].to_numpy(float)
154     if np.isnan(power).any():
155         raise ValueError("附件 3 功率列存在空值")
156     if power.min() < 0:
157         raise ValueError("附件 3 功率列存在负值")
158
159     F = np.zeros((N_RELEASE, N_DAY, 25))
160     F[:, :, 1:] = power.reshape(N_DAY, N_RELEASE, 24).transpose(1, 0, 2)
161
162     # ---- 首端锚点 F~0~{()}
163     anchor_source = np.empty((N_RELEASE, N_DAY), dtype=object)
164     for n in range(N_DAY):
165         for hi in range(N_RELEASE):
166             if hi > 0:
167                 F[hi, n, 0] = F[hi - 1, n, 6]
168                 anchor_source[hi, n] = f"{RELEASE_HOURS[hi-1]}:00 发布，预报 6 小时"
169             elif n > 0:
170                 F[0, n, 0] = F[3, n - 1, 6]
171                 anchor_source[0, n] = "前一日 18:00 发布，预报 6 小时"
172             else:
173                 # 2025-01-01 0:00 之前没有版本可用；该时刻的实际光伏为 0，
174                 # 且不能使用之后才得到的真实值，故取 0 并在此显式声明。
175                 F[0, n, 0] = 0.0
176                 anchor_source[0, n] = "无更早版本，取 0 (0:00 实际光伏为 0)"
177
178     # ---- 长表（模块 1 的交付物之一）
179     hi_of = np.repeat(np.arange(N_RELEASE), N_DAY)
180     recs = []

```

```

181     for hi in range(N_RELEASE):
182         base = pd.Timestamp("2025-01-01") + pd.to_timedelta(np.arange(N_DAY), "D")
183         rel = base + pd.Timedelta(hours=RELEASE_HOURS[hi])
184         for j in range(1, 25):
185             recs.append(pd.DataFrame({
186                 "发布日期": base,
187                 "发布时间": rel,
188                 "目标时间": rel + pd.Timedelta(hours=j),
189                 "提前时长_h": j,
190                 "预测功率_kW": F[hi, :, j],
191             }))
192     long = pd.concat(recs, ignore_index=True)
193
194     return ForecastPanel(F=F, long=long, anchor_source=anchor_source)
195
196 # ===== 模块 2: 预报时间转换
197 def _interp_hour(A: np.ndarray, rel_min: float) -> float:
198     """在同一发布版本内部做分段线性插值。
199
200     A[j] 是发布后 j 小时的预测功率 (kW), j = 0..24, 节点间隔 60 min。
201     rel_min 是相对发布时刻的分钟数, 取值 [0, 1440]。
202     """
203
204     if rel_min <= 0.0:
205         return float(A[0])
206     if rel_min >= 1440.0:
207         return float(A[24])
208     j = int(rel_min // 60)
209     th = (rel_min - 60.0 * j) / 60.0
210     return float((1.0 - th) * A[j] + th * A[j + 1])
211
212 def forecast_energy_slots(fp: ForecastPanel, hi: int, n: int) -> np.ndarray:
213     """把某次发布的整点功率预报积分成当天各十分钟时段的光伏电量 (kWh)。
214
215     返回长度 144 的数组; 发布时刻之前 (已经执行过) 的时段记为 NaN。
216     十分钟边界与整点对齐, 每个区间都落在同一段线性插值内, 故
217      $\hat{v}_t = (V(a_t) + V(b_t)) / 2 \cdot \Delta t$  精确等于该区间上插值曲线的积分。
218     """
219
220     t0 = 36 * hi
221     out = np.full(N, np.nan)
222     A = fp.F[hi, n, :]
223     rel0 = 60 * 6 * hi # 发布时刻相对当天 0:00 的分钟数
224     for t in range(t0, N):
225         ra = t * 10.0 - rel0
226         rb = ra + 10.0
227         out[t] = 0.5 * (_interp_hour(A, ra) + _interp_hour(A, rb)) * TAU
228     return out
229
230 def load_forecast_at(l_hat_day: np.ndarray, l_kwh: np.ndarray,
231                     hi: int) -> np.ndarray:
232     """决策时刻 hi 的负载预测 (kWh)。
233
234     hi = 0 用历史外推的日形状; hi > 0 时再用当天**已经观测完**的时段算一个
235     比例修正 kappa, 作用在尚未执行的时段上 (框架 4.1 节)。已观测时段过少
236     时不启用修正, 避免小样本放大噪声。
237     """
238
239     t0 = 36 * hi
240     out = l_hat_day.copy()
241     if hi == 0 or not INTRADAY_LOAD or t0 < MIN_KAPPA_SAMPLES:
242         return out
243     pred = float(l_hat_day[:t0].sum())
244     obs = float(l_kwh[:t0].sum())

```

```

245     if pred <= 1e-9:
246         return out
247     kappa = obs / pred
248     out[t0:] = l_hat_day[t0:] * kappa
249     return out
250
251
252 # ===== 模块 3: 负载与误差更新
253 def build_eps3(LOAD: np.ndarray, PV: np.ndarray, fp: ForecastPanel,
254               fo: Forecaster) -> np.ndarray:
255     """按发布版本维护可验证的净负荷预测残差。
256
257     eps3[hi, n, t] =  $N_{act}(n, t) - \hat{N}_t^{(hi, n)}$ , 只在  $t \geq 36 \cdot hi$  上有定义。
258     hi > 0 的样本使用当日的日内比例修正——该修正只取当天 0:00 至发布时刻
259     之间的实际负载, 样本被后续日期使用时整日早已观测完毕, 不会引入未来信息。
260     """
261     eps = np.full((N_RELEASE, N_DAY, N), np.nan)
262     l_kwh_all = LOAD * TAU
263     n_act_all = (LOAD - PV) * TAU
264     for n in range(1, N_DAY):
265         l_hat_day, _, _ = fo.energy(n)
266         l_kwh = l_kwh_all[n]
267         for hi in range(N_RELEASE):
268             t0 = 36 * hi
269             l_hat = load_forecast_at(l_hat_day, l_kwh, hi)
270             v_hat = forecast_energy_slots(fp, hi, n)
271             eps[hi, n, t0:] = n_act_all[n, t0:] - (l_hat - v_hat)[t0:]
272     return eps
273
274
275 def forecast_skill(LOAD: np.ndarray, PV: np.ndarray, fp: ForecastPanel,
276                  fo: Forecaster) -> dict:
277     """各发布版本在正式区间内的净负荷预测误差。
278
279     三个口径分开报, 避免把「逐时段绝对偏差之和」当成「全天电量偏差」:
280     * 逐时段绝对偏差之和 =  $\sum_t |N_t - \hat{N}_t|$  (kWh/日), 衡量十分钟尺度的错位;
281     * 全天电量偏差 =  $|\sum_t (N_t - \hat{N}_t)|$  (kWh/日), 衡量日总量口径;
282     * 符号偏差 =  $\sum_t (N_t - \hat{N}_t)$  (kWh/日), 正表示净负荷被低估。
283     另给出「对照_历史外推」, 即问题二那套光伏外推预测在同一区间的表现,
284     用于说明附件 3 的 0:00 版本并不比历史外推更准——这正是后续发布时刻
285     具有价值的原因。
286     """
287     n_days = REPORT_END - REPORT_START
288     out, acc = {}, {}
289     keys = list(RELEASE_HOURS) + ["hist"]
290     for k in keys:
291         acc[k] = [0.0, 0.0, 0.0] # |逐时段|之和 / |全天|之和 / 符号和
292     denom = 0.0
293     for n in range(REPORT_START, REPORT_END):
294         n_act = (LOAD[n] - PV[n]) * TAU
295         l_hat_day, _, n_hat_hist = fo.energy(n)
296         for hi, hr in enumerate(RELEASE_HOURS):
297             t0 = 36 * hi
298             l_hat = load_forecast_at(l_hat_day, LOAD[n] * TAU, hi)
299             v_hat = forecast_energy_slots(fp, hi, n)
300             d = n_act[t0:] - (l_hat - v_hat)[t0:]
301             acc[hr][0] += float(np.abs(d).sum())
302             acc[hr][1] += abs(float(d.sum()))
303             acc[hr][2] += float(d.sum())
304         d = n_act - n_hat_hist
305         acc["hist"][0] += float(np.abs(d).sum())
306         acc["hist"][1] += abs(float(d.sum()))
307         acc["hist"][2] += float(d.sum())
308     denom += float(n_act.sum())

```

```

309 mean_net = denom / n_days
310 for hr, (a, b, c) in acc.items():
311     name = "对照_历史外推" if hr == "hist" else hr
312     out[name] = {
313         "逐时段绝对偏差之和_kWh每日": a / n_days,
314         "全天电量偏差_kWh每日": b / n_days,
315         "符号偏差_kWh每日": c / n_days,
316         "相对日净负荷": (b / n_days) / mean_net,
317     }
318 out["平均日净负荷_kWh"] = mean_net
319 return out
320
321
322 def risk_margin3(eps: np.ndarray, hi: int, n: int, alpha: float | None,
323                 window: int = 28, min_samples: int = 10) -> np.ndarray:
324     """Q_{, ,t}: 只用第 n 天之前、且版本 hi 的历史残差。"""
325     t0 = 36 * hi
326     out = np.zeros(N)
327     if alpha is None or n == 0:
328         return out
329     lo = max(0, n - window)
330     hist = eps[hi, lo:n, t0:]
331     hist = hist[~np.isnan(hist).all(axis=1)]
332     k = hist.shape[0]
333     if k >= min_samples:
334         out[t0:] = np.quantile(hist, alpha, axis=0)
335         return out
336     if k >= 3:
337         # 样本不足时合并相邻时段 (±2 个十分钟)，仍然只用过去数据
338         m = hist.shape[1]
339         pooled = np.empty_like(hist)
340         for s in range(m):
341             a, b = max(0, s - 2), min(m, s + 3)
342             pooled[:, s] = hist[:, a:b].mean(axis=1)
343         out[t0:] = np.quantile(pooled, alpha, axis=0)
344     return out
345
346 # ===== 模块 4/5: 滚动规划 MILP
347 _PC3: dict[int, dict] = {}
348
349
350 def _precomp3(m: int) -> dict:
351     """为"剩余 m 个时段"的调整问题装配 MILP 结构 (与日期无关, 缓存复用)。
352
353     变量分块 (顺序即下标顺序):
354         y(m) 候选购电量 | a(m) 上调量 | b(m) 下调量 | c(m) 充电 | d(m) 放电
355         | w(m) 弃电 | r(m) 规划缺口 | E(m) 参考储电量 | z(m) 充放电互斥 |
356
357     y - a + b = g 把分段调整费用线性化:
358         主口径 → ub[b] = 0, lb[y] = g_t, 于是 a = (y-g)_+
359         退款口径 → lb[y] = 0, a, b = 0, 目标中 b 的系数为 -0.5p
360     最小化时不可能出现 a、b 同时为正 (同增 使费用增加 1.0p), 故无需整数
361     变量即可精确表达该分段函数。
362     """
363     if m in _PC3:
364         return _PC3[m]
365
366     nvar = 9 * m + 1
367     iY, iA, iB, iC, iD, iW, iR, iE, iZ, iXi = (
368         0, m, 2 * m, 3 * m, 4 * m, 5 * m, 6 * m, 7 * m, 8 * m, 9 * m)
369
370     rows, cols, vals = [], [], []
371
372

```

```

373 def add(r, c, v):
374     rows.append(r); cols.append(c); vals.append(v)
375
376 # (1) 风险净负荷平衡:  $y + r + d - c - w = \tilde{N}_t$  行 0..m-1
377 for i in range(m):
378     add(i, iY + i, 1.0); add(i, iR + i, 1.0); add(i, iD + i, 1.0)
379     add(i, iC + i, -1.0); add(i, iW + i, -1.0)
380
381 # (2) 状态递推:  $E_t - E_{t-1} - c_t + d_t = 0$  行 m..2m-1
382 for i in range(m):
383     r = m + i
384     add(r, iE + i, 1.0)
385     if i > 0:
386         add(r, iE + i - 1, -1.0)
387     add(r, iC + i, -ETA); add(r, iD + i, 1.0 / ETA)
388
389 # (3) 调整量分解:  $y - a + b = g_t$  行 2m..3m-1
390 for i in range(m):
391     r = 2 * m + i
392     add(r, iY + i, 1.0); add(r, iA + i, -1.0); add(r, iB + i, 1.0)
393
394 A_eq = csr_matrix((vals, (rows, cols)), shape=(3 * m, nvar))
395
396 ur, uc, uv = [], [], []
397
398 def uadd(r, c, v):
399     ur.append(r); uc.append(c); uv.append(v)
400
401 for i in range(m):
402     uadd(2 * i, iC + i, 1.0); uadd(2 * i, iZ + i, -M_ENERGY)
403     uadd(2 * i + 1, iD + i, 1.0); uadd(2 * i + 1, iZ + i, M_ENERGY)
404 uadd(2 * m, iXi, -1.0); uadd(2 * m, iE + m - 1, -1.0)
405
406 A_ub = csr_matrix((uv, (ur, uc)), shape=(2 * m + 1, nvar))
407
408 lb = np.concatenate([
409     np.zeros(m),           # y (求解时抬到 g_t 或保持 0)
410     np.zeros(m),           # a
411     np.zeros(m),           # b
412     np.zeros(m),           # c
413     np.zeros(m),           # d
414     np.zeros(m),           # w
415     np.zeros(m),           # r
416     np.full(m, E_MIN),     # E
417     np.zeros(m),           # z
418     [0.0],                 #
419 ])
420 ub = np.concatenate([
421     np.full(m, np.inf),     # y
422     np.full(m, np.inf),     # a
423     np.full(m, np.inf),     # b (主口径求解时压到 0)
424     np.full(m, M_ENERGY),   # c
425     np.full(m, M_ENERGY),   # d
426     np.full(m, np.inf),     # w
427     np.full(m, np.inf),     # r
428     np.full(m, E_MAX),      # E
429     np.ones(m),             # z
430     [np.inf],               #
431 ])
432 integ = np.zeros(nvar)
433 integ[iZ:iZ + m] = 1
434
435 d = {"nvar": nvar, "A_eq": A_eq, "A_ub": A_ub, "lb": lb, "ub": ub,
436     "integ": integ, "iY": iY, "iA": iA, "iB": iB, "iR": iR, "iE": iE,

```

```

437         "iXi": iXi, "m": m}
438     _PC3[m] = d
439     return d
440
441
442 @dataclass
443 class AdjustPlan:
444     y: np.ndarray          # 剩余时段最终生效的完整购电量（含展望部分）
445     up: np.ndarray         # 上调量 a_t
446     down: np.ndarray       # 下调量 b_t
447     Ebar: np.ndarray       # 参考储电量轨迹
448     r_plan: np.ndarray     # 规划缺口
449     fee: float             # 本次求解出的调整费（不含已确定的原计划费）
450
451
452 def solve_adjust(ntilde: np.ndarray, g0_rem: np.ndarray, e0: float,
453                 price_rem: np.ndarray, rp: RiskParams,
454                 convention: str = "main", etar: float = E_TAR) -> AdjustPlan:
455     """在剩余时段上求解调整模型（框架 4.3 节）。
456
457     目标 =  $\sum A_t(y_t; g_t) + \sum 5p \bar{r}_t +$  ，原计划费是常数，不重复计入。
458     """
459     if convention not in CONVENTIONS:
460         raise ValueError(f"未知计费口径 {convention!r}")
461     m = len(ntilde)
462     if not (E_MIN - 1e-9 <= e0 <= E_MAX + 1e-9):
463         raise ValueError(f"日初储电量 e0={e0:.6f} 超出 [{E_MIN}, {E_MAX}] kWh")
464
465     d = _precomp3(m)
466     iY, iA, iB, iR, iE, iXi = (d["iY"], d["iA"], d["iB"], d["iR"], d["iE"],
467                               d["iXi"])
468
469     b_eq = np.empty(3 * m)
470     b_eq[:m] = ntilde
471     b_eq[m] = e0
472     b_eq[m + 1:2 * m] = 0.0
473     b_eq[2 * m:] = g0_rem
474
475     b_ub = np.empty(2 * m + 1)
476     b_ub[:2 * m] = np.tile([0.0, M_ENERGY], m)
477     b_ub[2 * m] = -etar
478
479     lb = d["lb"].copy()
480     ub = d["ub"].copy()
481     if convention == "main":
482         lb[iY:iY + m] = g0_rem          # x_t g_t, a 自动等于上调量
483         ub[iB:iB + m] = 0.0            # 禁掉下调量
484     else:
485         lb[iY:iY + m] = 0.0            # 退款口径允许双向调整
486
487     cvec = np.zeros(d["nvar"])
488     cvec[iA:iA + m] = COEF_UP * price_rem
489     cvec[iB:iB + m] = -COEF_DOWN * price_rem
490     cvec[iR:iR + m] = COEF_EMG * price_rem
491     cvec[iXi] = rp.lam
492
493     res = milp(cvec,
494               constraints=[LinearConstraint(d["A_eq"], b_eq, b_eq),
495                               LinearConstraint(d["A_ub"], -np.inf, b_ub)],
496               integrality=d["integ"], bounds=Bounds(lb, ub))
497     if not res.success:
498         raise RuntimeError(f"调整 MILP 求解失败: {res.message}")
499
500     x = res.x

```

```

501 a, b = x[iA:iA + m], x[iB:iB + m]
502 fee = float(np.sum(COEF_UP * price_rem * a - COEF_DOWN * price_rem * b))
503 return AdjustPlan(y=x[iY:iY + m], up=a, down=b, Ebar=x[iE:iE + m],
504                  r_plan=x[iR:iR + m], fee=fee)
505
506
507 # ===== 模块 7: 实际执行回放
508 class DayRunner:
509     """按交付块推进的十分钟执行器, 规则与问题二 5.2 节完全一致。
510
511      $b_t = x_t + v_t - \_t$  为使用储能前的盈余; 储备线  $R_t = 1200 + (\bar{E}_t - 1200)$ 
512     取**当前最新一次求解**给出的参考轨迹。分块推进的原因是新预报到达时
513     参考轨迹会整体刷新, 后面的时段必须用新的  $R_t$ 。
514     """
515
516     def __init__(self, l_act: np.ndarray, v_act: np.ndarray,
517                  e0: float, rho: float):
518         self.l, self.v = l_act, v_act
519         self.rho = rho
520         self.E0 = float(e0)
521         self.Ep = float(e0)
522         self.c = np.zeros(N)
523         self.d = np.zeros(N)
524         self.r = np.zeros(N)
525         self.w = np.zeros(N)
526         self.E = np.zeros(N)
527
528     def run(self, a: int, b: int, x: np.ndarray, Ebar: np.ndarray) -> None:
529         for t in range(a, b):
530             bt = x[t] + self.v[t] - self.l[t]
531             R = E_MIN + self.rho * (Ebar[t] - E_MIN)
532             if bt > 0:
533                 self.c[t] = min(bt, M_ENERGY, (E_MAX - self.Ep) / ETA)
534                 self.w[t] = bt - self.c[t]
535             else:
536                 head = max(0.0, self.Ep - R)
537                 self.d[t] = min(-bt, M_ENERGY, ETA * head)
538                 self.r[t] = max(0.0, -bt - self.d[t])
539             self.Ep = self.Ep + ETA * self.c[t] - self.d[t] / ETA
540             self.E[t] = self.Ep
541
542     @property
543     def E_end(self) -> float:
544         return float(self.E[-1])
545
546
547 @dataclass
548 class DayRecord3:
549     n: int
550     date: str
551     g0: np.ndarray          # 0:00 冻结的初始计划 (永久保存)
552     x: np.ndarray          # 最终生效的完整购电量
553     c: np.ndarray
554     d: np.ndarray
555     r: np.ndarray
556     w: np.ndarray
557     E: np.ndarray
558     E0: float
559     Ebar: np.ndarray        # 拼接后的参考轨迹 (诊断用)
560     cost_plan: float        #  $\sum p g$ 
561     cost_adj: float         #  $\sum A_t(x; g)$ 
562     cost_emg: float         #  $\sum 5p r$ 
563     n_solve: int            # 当天实际执行的 MILP 求解次数
564     n_submit: int           # 当天正式提交调整的交付块数

```

```

565 S: tuple # 当天使用的预报组合（用于校验提交范围）
566 l_hat0: np.ndarray
567 v_hat0: np.ndarray
568 n_hat0: np.ndarray
569 margin0: np.ndarray
570 n_risk0: np.ndarray
571
572 @property
573 def E_end(self) -> float:
574     return float(self.E[-1])
575
576 @property
577 def cost_total(self) -> float:
578     return self.cost_plan + self.cost_adj + self.cost_emg
579
580 @property
581 def g_initial(self) -> float:
582     return float(self.g0.sum())
583
584 @property
585 def g_effective(self) -> float:
586     return float(self.x.sum())
587
588 @property
589 def g_emg(self) -> float:
590     return float(self.r.sum())
591
592
593 # ----- 初始计划缓存
594 _PLANO_CACHE: dict[tuple, tuple[np.ndarray, np.ndarray]] = {}
595
596
597 def initial_plan(n: int, ntilde0: np.ndarray, e0: float, price: np.ndarray,
598                 rp: RiskParams) -> tuple[np.ndarray, np.ndarray]:
599     """0:00 初始计划的带缓存求解。
600
601     计划只依赖 (n, , , e0) 与当天的 0:00 预报；在滚动标定的多次回放中
602     (n, , ) 会大量重复，而 e0 由回放起点决定、在同一窗口内唯一，故缓存
603     键取 (n, , , e0)。 只影响执行层，不进缓存键。
604     """
605     key = (n, rp.alpha, rp.lam, round(float(e0), 6))
606     hit = _PLANO_CACHE.get(key)
607     if hit is not None:
608         return hit[0].copy(), hit[1].copy()
609     if len(_PLANO_CACHE) > 40000: # 上界保护：约 92 MB，超出则整体清空
610         _PLANO_CACHE.clear()
611     plan = solve_dayahead(ntilde0, e0, price, rp)
612     _PLANO_CACHE[key] = (plan.g.copy(), plan.Ebar.copy())
613     return plan.g, plan.Ebar
614
615
616 # ===== 单日仿真
617 def run_day3(n: int, e0: float, S: tuple, params: RiskParams,
618             use_new_forecast: bool, LOAD: np.ndarray, PV: np.ndarray,
619             price: np.ndarray, fp: ForecastPanel, eps: np.ndarray,
620             fo: Forecaster, convention: str = "main") -> DayRecord3:
621     """回放第 n 天：0:00 定计划 → 逐块执行 → 到达发布时刻则滚动调整。"""
622     l_act = LOAD[n, :] * TAU
623     v_act = PV[n, :] * TAU
624     date = str(np.datetime64("2025-01-01") + np.timedelta64(n, "D"))
625
626     if n == 0:
627         # 冷启动：与问题二一致，无历史可用，计划购电为零，缺口全部紧急补购
628         runner = DayRunner(l_act, v_act, e0, params.rho)

```

```

629     z = np.zeros(N)
630     runner.run(0, N, z, np.full(N, E_INIT))
631     return DayRecord3(
632         n=n, date=date, g0=z.copy(), x=z.copy(), c=runner.c, d=runner.d,
633         r=runner.r, w=runner.w, E=runner.E, E0=float(e0),
634         Ebar=np.full(N, E_INIT), cost_plan=0.0, cost_adj=0.0,
635         cost_emg=float(np.sum(COEF_EMG * price * runner.r)), n_solve=0,
636         n_submit=0, S=(), l_hat0=z, v_hat0=z, n_hat0=z, margin0=z,
637         n_risk0=z)
638
639     # ----- 0:00: 全天初始计划 (永远使用 0:00 发布的预报)
640     l_hat_day, _, _ = fo.energy(n)
641     l_hat0 = load_forecast_at(l_hat_day, l_act, 0)
642     v_hat0 = forecast_energy_slots(fp, 0, n)
643     margin0 = risk_margin3(e0, 0, n, params.alpha)
644     n_hat0 = l_hat0 - v_hat0
645     n_risk0 = n_hat0 + margin0
646     g0, Ebar = initial_plan(n, n_risk0, e0, price, params)
647     x = g0.copy()
648
649     runner = DayRunner(l_act, v_act, e0, params.rho)
650     n_solve, n_submit = 1, 0
651
652     # ----- 六个小时一个交付块
653     for hi, (a, b) in enumerate(BLOCK_BOUNDS):
654         if hi > 0 and RELEASE_HOURS[hi] in S:
655             # 使用最新风险净负荷重新求解; 对照策略只换状态、不换预报版本
656             efor = hi if use_new_forecast else 0
657             l_hat_h = load_forecast_at(l_hat_day, l_act, hi)
658             v_hat_h = forecast_energy_slots(fp, efor, n)
659             margin_h = risk_margin3(e0, efor, n, params.alpha)
660             n_risk_h = (l_hat_h - v_hat_h) + margin_h
661             res = solve_adjust(n_risk_h[a:], g0[a:], runner.Ep, price[a:],
662                               params, convention)
663             x[a:b] = res.y[b - a:] # 只正式提交当前交付块
664             Ebar[a:] = res.Ebar # 参考轨迹整体刷新 (含展望部分)
665             n_solve += 1
666             n_submit += 1
667             runner.run(a, b, x, Ebar)
668
669     cost_plan = float(np.sum(price * g0))
670     if convention == "main":
671         cost_adj = float(np.sum(COEF_UP * price * np.maximum(x - g0, 0.0)))
672     else:
673         cost_adj = float(np.sum(COEF_UP * price * np.maximum(x - g0, 0.0)
674                               - COEF_DOWN * price * np.maximum(g0 - x, 0.0)))
675     cost_emg = float(np.sum(COEF_EMG * price * runner.r))
676
677     return DayRecord3(
678         n=n, date=date, g0=g0, x=x, c=runner.c, d=runner.d, r=runner.r,
679         w=runner.w, E=runner.E, E0=float(e0), Ebar=Ebar, cost_plan=cost_plan,
680         cost_adj=cost_adj, cost_emg=cost_emg, n_solve=n_solve,
681         n_submit=n_submit, S=tuple(S), l_hat0=l_hat0, v_hat0=v_hat0,
682         n_hat0=n_hat0, margin0=margin0, n_risk0=n_risk0)
683
684     # ===== 整段回放
685     _SIM: dict = {}
686
687
688
689     def setup(LOAD, PV, price, fp, fo) -> None:
690         _SIM.update(LOAD=LOAD, PV=PV, price=price, fp=fp, fo=fo,
691                     eps=build_eps3(LOAD, PV, fp, fo))
692

```

```

693
694 def replay3(n0: int, n1: int, e0: float, S: tuple, params: RiskParams,
695             use_new_forecast: bool = True,
696             convention: str = "main") -> tuple[list[DayRecord3], float]:
697     """从 e0 出发回放 [n0, n1) 天, 返回 (逐日记录, 末日储电量)。"""
698     d = _SIM
699     recs = []
700     e = float(e0)
701     for n in range(n0, n1):
702         rec = run_day3(n, e, S, params, use_new_forecast, d["LOAD"], d["PV"],
703                       d["price"], d["fp"], d["eps"], d["fo"], convention)
704         recs.append(rec)
705         e = rec.E_end
706     return recs, e
707
708
709 def totals3(recs: list[DayRecord3]) -> dict:
710     """按记录集合汇总费用与电量 (框架 8.1 节要求同时报告期末储电量)。"""
711     cp = sum(r.cost_plan for r in recs)
712     ca = sum(r.cost_adj for r in recs)
713     ce = sum(r.cost_emg for r in recs)
714     gi = sum(r.g_initial for r in recs)
715     ge = sum(r.g_effective for r in recs)
716     gem = sum(r.g_emg for r in recs)
717     return {
718         "天数": len(recs),
719         "初始计划量_kWh": gi,
720         "最终常规量_kWh": ge,
721         "紧急购电量_kWh": gem,
722         "收到总电量_kWh": ge + gem,
723         "计划费_元": cp,
724         "调整费_元": ca,
725         "紧急费_元": ce,
726         "合计费用_元": cp + ca + ce,
727         "弃电量_kWh": sum(float(r.w.sum()) for r in recs),
728         "充电量_kWh": sum(float(r.c.sum()) for r in recs),
729         "放电量_kWh": sum(float(r.d.sum()) for r in recs),
730         "期初储电量_kWh": float(recs[0].E0),
731         "期末储电量_kWh": float(recs[-1].E[-1]),
732         "调整提交次数": sum(r.n_submit for r in recs),
733         "MILP求解次数": sum(r.n_solve for r in recs),
734     }
735
736
737 # ===== 参数标定
738 @dataclass(frozen=True)
739 class CalConfig3:
740     # 网格下探到 =0.30: 主计费口径下追加电量的边际成本高于问题二,
741     # 最优风险余量本就偏小, 若只从 0.60 起扫描会把最优值压在边界上。
742     alphas: tuple = (0.30, 0.40, 0.50, 0.60, 0.70, 0.80)
743     rhos: tuple = (0.0, 0.25, 0.5, 1.0)
744     lams: tuple = (0.60,)
745     window: int = 35 # 标定回放窗口天数
746     every: int = 42 # 每隔多少天重新标定一次
747     start: int = REPORT_START # 首次标定发生在正式区间首日
748     S: tuple = ALL_S # 标定所用的预报组合
749
750     def grid(self) -> list[RiskParams]:
751         return [RiskParams(alpha=a, rho=r, lam=l)
752                 for a in self.alphas for r in self.rhos for l in self.lams]
753
754
755 def calibrate3(n0: int, e_at_window_start: float,
756               cal: CalConfig3, use_new_forecast: bool = True,

```

```

757         convention: str = "main") -> tuple[RiskParams, list[dict]]:
758     """在第 n0 天 0:00 用此前数据选择参数。
759
760     对网格中每个组合，从窗口起点回放窗口内的历史日期，比较**实际总费用**
761     （含调整费与五倍紧急费），取最小者。窗口初值取主运行在该日 0:00 的
762     实际储电量，不使用第 n0 天（窗口末尾）的状态，避免引入未来信息。
763
764     use_new_forecast 与 convention 必须由调用方传入并原样转交给 replay3:
765     标定是在**该策略自己的信息集与计费口径**下进行的，否则对照策略（不看
766     新预报）与退款口径敏感性会用另一套策略的参数，对照就失去意义。
767     """
768     lo = max(0, n0 - cal.window)
769     rows, best, best_cost = [], None, np.inf
770     for rp in cal.grid():
771         recs, _ = replay3(lo, n0, e_at_window_start, cal.S, rp,
772                           use_new_forecast, convention)
773         cost = sum(r.cost_total for r in recs)
774         rows.append({"标定日": _date_of(n0), "窗口起": _date_of(lo),
775                     "参数": rp.label(), "窗口实际总费用_元": cost})
776         if cost < best_cost - 1e-9:
777             best_cost, best = cost, rp
778     assert best is not None
779     for row in rows:
780         row["选中"] = row["参数"] == best.label()
781     return best, rows
782
783
784 def _date_of(n: int) -> str:
785     return str(np.datetime64("2025-01-01") + np.timedelta64(int(n), "D"))
786
787
788 def run_strategy3(cal: CalConfig3, SOFTS: tuple, e_init: float = E_INIT,
789                  use_new_forecast: bool = True, convention: str = "main",
790                  verbose: bool = False) -> tuple[list[DayRecord3], list[dict]]:
791     """整年滚动：预热 1 月，正式区间内每隔 cal.every 天重新标定一次参数。"""
792     recs, cal_rows = [], []
793     e = float(e_init)
794     e_at = {0: e}
795     params = RiskParams()
796     for n in range(N_DAY):
797         if n >= cal.start and (n - cal.start) % cal.every == 0:
798             params, rows = calibrate3(n, e_at[max(0, n - cal.window)], cal,
799                                       use_new_forecast, convention)
800             cal_rows.extend(rows)
801             if verbose:
802                 print(f"    [{_date_of(n)}] 选中 {params.label()}", flush=True)
803             rec = run_day3(n, e, SOFTS, params, use_new_forecast, _SIM["LOAD"],
804                           _SIM["PV"], _SIM["price"], _SIM["fp"], _SIM["eps"],
805                           _SIM["fo"], convention)
806             recs.append(rec)
807             e = rec.E_end
808             e_at[n + 1] = e
809     return recs, cal_rows
810
811
812 # ===== 校验
813 def validate_day3(rec: DayRecord3, price: np.ndarray,
814                  convention: str = "main", tol: float = 1e-6) -> dict:
815     """逐日复核，按框架 8.2 节的七项检查分组返回问题清单。"""
816     info, tm, lock, cost, power, chain, consist = [], [], [], [], [], [], []
817     l = _SIM["LOAD"][rec.n, :] * TAU
818     v = _SIM["PV"][rec.n, :] * TAU
819
820     # 5. 储能与供电：逐段能量守恒、边界、互斥

```

```

821 resid = rec.x + rec.r + v + rec.d - l - rec.c - rec.w
822 scale = max(1.0, float(l.max()))
823 if np.abs(resid).max() > tol * scale:
824     power.append(f"供需平衡残差 {np.abs(resid).max():.3e} kWh")
825 if min(rec.c.min(), rec.d.min(), rec.r.min(), rec.w.min()) < -tol:
826     power.append("存在负的充电/放电/紧急购电/弃电")
827 if rec.c.max() > M_ENERGY + tol or rec.d.max() > M_ENERGY + tol:
828     power.append("充放电越限")
829 if ((rec.c > tol) & (rec.d > tol)).any():
830     power.append("存在同时充放电时段")
831 Eseq = np.concatenate([[rec.E0], rec.E])
832 if Eseq.min() < E_MIN - tol or Eseq.max() > E_MAX + tol:
833     power.append(f"储电量越界 [{Eseq.min():.4f}, {Eseq.max():.4f}]")
834
835 # 3. 计划锁定: x g (主口径) 且两者均为 144 维非负
836 if rec.g0.shape != (N,) or rec.x.shape != (N,):
837     lock.append("计划/调整数组维度非法")
838 if rec.g0.min() < -tol or rec.x.min() < -tol:
839     lock.append("存在负的购电量")
840 if convention == "main" and (rec.x - rec.g0).min() < -tol:
841     lock.append(f"主口径下出现下调 {(rec.x - rec.g0).min():.3e} kWh")
842
843 # 4. 费用核对: 按定义独立复算
844 cp = float(np.sum(price * rec.g0))
845 if convention == "main":
846     ca = float(np.sum(COEF_UP * price * np.maximum(rec.x - rec.g0, 0.0)))
847 else:
848     ca = float(np.sum(COEF_UP * price * np.maximum(rec.x - rec.g0, 0.0)
849                    - COEF_DOWN * price * np.maximum(rec.g0 - rec.x, 0.0)))
850 ce = float(np.sum(COEF_EMG * price * rec.r))
851 for name, a, b in (("计划费", cp, rec.cost_plan), ("调整费", ca, rec.cost_adj),
852                  ("紧急费", ce, rec.cost_emg)):
853     if abs(a - b) > tol * max(1.0, abs(a)):
854         cost.append(f"{name}复算不符 {a:.6f} vs {b:.6f}")
855
856 # 6. 状态衔接: 日末电量 = 日初 + c · Σc - Σd/
857 lhs = rec.E[-1] - rec.E0
858 rhs = ETA * rec.c.sum() - rec.d.sum() / ETA
859 if abs(lhs - rhs) > tol * max(1.0, abs(rhs)):
860     chain.append(f"状态累计不符: ΔE={lhs:.6f} vs {rhs:.6f}")
861
862 # 7. 结果一致性: 初始计划不被覆盖——出现差异的时段必须落在当天
863 # 正式提交过的交付块内 (第 0 块永远按初始计划执行), 且提交次数
864 # 必须与组合 S 一致。
865 diff = np.abs(rec.x - rec.g0) > tol
866 if rec.n != 0 and rec.n_submit != len(rec.S):
867     consist.append(f"提交次数 {rec.n_submit} 与组合 {rec.S} 不符")
868 allowed = np.zeros(N, bool)
869 for h in rec.S:
870     b = h // 6
871     allowed[36 * b:36 * (b + 1)] = True
872 stray = diff & ~allowed
873 if stray.any():
874     consist.append(f"存在 {int(stray.sum())} 段未提交却生效的调整")
875
876 # 1. 信息可用性 / 2. 时间转换: 结构由实现保证, 此处校验可核对的量
877 if np.isnan(rec.v_hat0[36:]).any() or np.isnan(rec.l_hat0).any():
878     info.append("0:00 计划的输入含空值")
879 if not np.all(np.diff(rec.E) > -1e9):
880     tm.append("储电量序列异常")
881
882 return {"信息可用性": info, "时间转换": tm, "计划锁定": lock,
883        "费用核对": cost, "储能与供电": power, "状态衔接": chain,
884        "结果一致性": consist}

```

```

885
886
887 def validation_metrics3(recs: list[DayRecord3], price: np.ndarray,
888                        convention: str = "main") -> dict:
889     """汇总七项校验的**数值**口径，供论文按实测值陈述。"""
890     m = {"供需平衡最大残差_kWh": 0.0, "储电量越界最大量_kWh": 0.0,
891          "充放电越限最大量_kWh": 0.0, "同时充放电时段数": 0,
892          "状态累计最大绝对偏差_kWh": 0.0, "计划费最大复算偏差_元": 0.0,
893          "调整费最大复算偏差_元": 0.0, "紧急费最大复算偏差_元": 0.0,
894          "主口径最大下调量_kWh": 0.0}
895     for rec in recs:
896         l = _SIM["LOAD"][rec.n, :] * TAU
897         v = _SIM["PV"][rec.n, :] * TAU
898         resid = rec.x + rec.r + v + rec.d - l - rec.c - rec.w
899         m["供需平衡最大残差_kWh"] = max(m["供需平衡最大残差_kWh"],
900                                          float(np.abs(resid).max()))
901         Eseq = np.concatenate([rec.E0, rec.E])
902         m["储电量越界最大量_kWh"] = max(m["储电量越界最大量_kWh"],
903                                          float(max(0.0, E_MIN - Eseq.min()))),
904                                          float(max(0.0, Eseq.max() - E_MAX)))
905         m["充放电越限最大量_kWh"] = max(m["充放电越限最大量_kWh"],
906                                          float(max(0.0, rec.c.max() - M_ENERGY)),
907                                          float(max(0.0, rec.d.max() - M_ENERGY)))
908         m["同时充放电时段数"] += int(((rec.c > 1e-6) & (rec.d > 1e-6)).sum())
909         m["状态累计最大绝对偏差_kWh"] = max(
910             m["状态累计最大绝对偏差_kWh"],
911             abs(float(rec.E[-1] - rec.E0 - (ETA * rec.c.sum()
912                                                - rec.d.sum() / ETA))))
913         cp = float(np.sum(price * rec.g0))
914         ca = float(np.sum(COEF_UP * price * np.maximum(rec.x - rec.g0, 0.0)
915                    - COEF_DOWN * price * np.maximum(rec.g0 - rec.x, 0.0)))
916         ce = float(np.sum(COEF_EMG * price * rec.r))
917         m["计划费最大复算偏差_元"] = max(m["计划费最大复算偏差_元"],
918                                          abs(cp - rec.cost_plan))
919         m["调整费最大复算偏差_元"] = max(m["调整费最大复算偏差_元"],
920                                          abs(ca - rec.cost_adj))
921         m["紧急费最大复算偏差_元"] = max(m["紧急费最大复算偏差_元"],
922                                          abs(ce - rec.cost_emg))
923         m["主口径最大下调量_kWh"] = max(m["主口径最大下调量_kWh"],
924                                          float(max(0.0, (rec.g0 - rec.x).max())))
925     return m
926
927
928 def check_all3(recs: list[DayRecord3], price: np.ndarray,
929               convention: str = "main") -> dict[str, int]:
930     """返回七项检查中各项的**失败天数**。"""
931     counts = {"信息可用性": 0, "时间转换": 0, "计划锁定": 0, "费用核对": 0,
932              "储能与供电": 0, "状态衔接": 0, "结果一致性": 0}
933     for rec in recs:
934         for k, v in validate_day3(rec, price, convention).items():
935             if v:
936                 counts[k] += 1
937     return counts
938
939
940 # ===== 结果整理
941 def emergency_events3(r: np.ndarray, eps: float = EMG_EPS) -> list[dict]:
942     """同日相邻的非零紧急购电合并为一个事件；跨午夜天然按日拆分。"""
943     out, t = [], 0
944     while t < N:
945         if r[t] <= eps:
946             t += 1
947             continue
948         a = t

```

```

949     while t < N and r[t] > eps:
950         t += 1
951     out.append({"时段起": fmt_time(a * 10), "时段止": fmt_time(t * 10),
952               "区间标签": f"{fmt_time(a * 10)}-{fmt_time(t * 10)}",
953               "段数": t - a, "购电量_kWh": float(r[a:t].sum())})
954     return out
955
956
957 def day_summary3(rec: DayRecord3, price: np.ndarray) -> dict:
958     idx = {interval_label(i): i for i in range(N)}
959     table1 = [{"时间段": lab, "计划购电量_kWh": float(rec.g0[idx[lab]]),
960              "调整购电量_kWh": float(rec.x[idx[lab]]),
961              "增量_kWh": float(rec.x[idx[lab]] - rec.g0[idx[lab]]),
962              "紧急购电量_kWh": float(rec.r[idx[lab]])}]
963     for lab in WANT_LABELS]
964     # 末段必须是 "20:00-24:00"。这里不能用 fmt_time: 它是给十分钟区间标签用的
965     # 约定, fmt_time(1440) 会返回 "0:00+1", 于是末段被写成 "20:00-0:00+1", 与
966     # 题目表 2 及附件 5 模板都不一致。block_label 正是为此实现的 (见问题一),
967     # 问题二的 table2 也走它。
968     table2 = [{"时间段": block_label(a, z),
969              "充电量_kWh": float(rec.c[a:z].sum()),
970              "放电量_kWh": float(rec.d[a:z].sum())}]
971     for a, z in FOUR_HOUR_BLOCKS]
972     return {
973         "date": rec.date, "table1": table1, "table2": table2,
974         "events": emergency_events3(rec.r),
975         "g_initial": rec.g_initial, "g_effective": rec.g_effective,
976         "g_emg": rec.g_emg, "g_received": rec.g_effective + rec.g_emg,
977         "cost_plan": rec.cost_plan, "cost_adj": rec.cost_adj,
978         "cost_emg": rec.cost_emg, "cost_total": rec.cost_total,
979         "charge_kwh": float(rec.c.sum()), "dischg_kwh": float(rec.d.sum()),
980         "dump_kwh": float(rec.w.sum()), "E0": rec.E0, "E144": float(rec.E[-1]),
981         "E_min": float(np.concatenate([rec.E0, rec.E]).min()),
982         "E_max": float(np.concatenate([rec.E0, rec.E]).max()),
983         "n_submit": rec.n_submit, "n_solve": rec.n_solve,
984         "emg_segments": int((rec.r > EMG_EPS).sum()),
985         "emg_events": len(emergency_events3(rec.r)),
986         "load_kwh": float((_SIM["LOAD"][rec.n] * TAU).sum()),
987         "pv_kwh": float((_SIM["PV"][rec.n] * TAU).sum()),
988     }
989
990
991 def detail_frame3(rec: DayRecord3, price: np.ndarray) -> pd.DataFrame:
992     E_prev = np.concatenate([rec.E0, rec.E[:-1]])
993     return pd.DataFrame({
994         "日期": rec.date,
995         "时段起": [fmt_time(i * 10) for i in range(N)],
996         "时段止": [fmt_time((i + 1) * 10) for i in range(N)],
997         "电价_元每kWh": price,
998         "预测负载_kWh": rec.l_hat0,
999         "预测光伏_0点版本_kWh": rec.v_hat0,
1000         "净负荷预测_kWh": rec.n_hat0,
1001         "风险余量_kWh": rec.margin0,
1002         "风险净负荷_kWh": rec.n_risk0,
1003         "参考储电量_kWh": rec.Ebar,
1004         "实际负载_kWh": _SIM["LOAD"][rec.n] * TAU,
1005         "实际光伏_kWh": _SIM["PV"][rec.n] * TAU,
1006         "初始计划购电_kWh": rec.g0,
1007         "最终购电_kWh": rec.x,
1008         "上调量_kWh": np.maximum(rec.x - rec.g0, 0.0),
1009         "下调量_kWh": np.maximum(rec.g0 - rec.x, 0.0),
1010         "实际充电_kWh": rec.c,
1011         "实际放电_kWh": rec.d,
1012         "紧急购电_kWh": rec.r,

```

```

1013     "弃电_kWh": rec.w,
1014     "期初储电量_kWh": E_prev,
1015     "期末储电量_kWh": rec.E,
1016     "计划费_元": price * rec.g0,
1017     "调整费_元": COEF_UP * price * np.maximum(rec.x - rec.g0, 0.0)
1018                 - COEF_DOWN * price * np.maximum(rec.g0 - rec.x, 0.0),
1019     "紧急费_元": COEF_EMG * price * rec.r,
1020 })
1021
1022
1023 def write_result3(summaries: list[dict], recs: list[DayRecord3] | None,
1024                  dates: list[str], path: Path) -> None:
1025     """按附件 5 的 result3.xlsx 模板写出四张表。
1026
1027     模板前两张表的时间标签整体错位一格（首列 '0:10-0:20'、末列 '0:00-0:10+1'），
1028     这里按正文口径统一改写为 '0:00-0:10' ... '23:50-0:00+1'，行序与模板一一对应，
1029     数值不会错位（与问题二 result2.xlsx 的处理一致）。
1030     「计划购电量」写 0:00 冻结的 g 与初始计划费；「调整购电量」写最终生效的 x
1031     与**已包含初始计划费**的完整实际总费用——两表费用不能再次相加。
1032     """
1033     import openpyxl
1034     import shutil
1035
1036     # 模板必须先落到输出路径。此前漏了这一步，导致整轮比较在最后一步崩在
1037     # FileNotFoundError 上，把四十多分钟的求解结果全部丢掉。
1038     if not path.exists():
1039         if not TEMPLATE3.exists():
1040             raise FileNotFoundError(f"缺少 result3.xlsx 模板: {TEMPLATE3}")
1041         path.parent.mkdir(parents=True, exist_ok=True)
1042         shutil.copyfile(TEMPLATE3, path)
1043
1044     wb = openpyxl.load_workbook(path)
1045
1046     def fill_slot_sheet(name: str, series_key: str, total_key: str,
1047                       fee_key: str) -> None:
1048         ws = wb[name]
1049         ws.cell(row=1, column=1, value="日期\\时间")
1050         for i in range(N):
1051             ws.cell(row=1, column=2 + i, value=interval_label(i))
1052         ws.cell(row=1, column=2 + N, value="全天购电量")
1053         ws.cell(row=1, column=3 + N, value="全天购电费")
1054         for k, s in enumerate(summaries):
1055             r = 2 + k
1056             ws.cell(row=r, column=1, value=dates[REPORT_START + k])
1057             for i in range(N):
1058                 ws.cell(row=r, column=2 + i,
1059                       value=round(float(s[series_key][i]), 6))
1060             ws.cell(row=r, column=2 + N, value=round(s[total_key], 6))
1061             ws.cell(row=r, column=3 + N, value=round(s[fee_key], 6))
1062
1063     if recs is not None:
1064         for k, rec in enumerate(recs):
1065             summaries[k]["list_g0"] = rec.g0
1066             summaries[k]["list_x"] = rec.x
1067     else:
1068         # 只读缓存重建时，逐段数组由 p3_detail.csv 提供，已挂在 summaries 上
1069         for k, s in enumerate(summaries):
1070             for key in ("list_g0", "list_x"):
1071                 if key not in s:
1072                     raise KeyError(f"第 {k} 天的汇总缺少 {key}, "
1073                                   "无法重建 result3.xlsx")
1074     fill_slot_sheet("计划购电量", "list_g0", "g_initial", "cost_plan")
1075     fill_slot_sheet("调整购电量", "list_x", "g_effective", "cost_total")
1076

```

```

1077 # ---- 充放电量：每天六段，全区间展开
1078 ws2 = wb["充放电量"]
1079 for row in range(ws2.max_row, 1, -1):
1080     ws2.delete_rows(row)
1081     ws2.cell(row=1, column=1, value="日期")
1082     ws2.cell(row=1, column=2, value="时间段")
1083     ws2.cell(row=1, column=3, value="充电量")
1084     ws2.cell(row=1, column=4, value="放电量")
1085     ws2.cell(row=1, column=5, value="时刻")
1086     ws2.cell(row=1, column=6, value="储电量")
1087     r = 2
1088     for k, s in enumerate(summaries):
1089         a0 = r
1090         for blk in s["table2"]:
1091             ws2.cell(row=r, column=2, value=blk["时间段"])
1092             ws2.cell(row=r, column=3, value=round(blk["充电量_kWh"], 6))
1093             ws2.cell(row=r, column=4, value=round(blk["放电量_kWh"], 6))
1094             r += 1
1095             ws2.cell(row=a0, column=1, value=dates[REPORT_START + k])
1096             ws2.cell(row=a0, column=5, value="0:00")
1097             ws2.cell(row=a0, column=6, value=round(s["E0"], 6))
1098             ws2.cell(row=a0 + 5, column=5, value="24:00")
1099             ws2.cell(row=a0 + 5, column=6, value=round(s["E144"], 6))
1100
1101 # ---- 紧急购电量：按实际事件展开
1102 ws3 = wb["紧急购电量"]
1103 for row in range(ws3.max_row, 1, -1):
1104     ws3.delete_rows(row)
1105     ws3.cell(row=1, column=1, value="日期")
1106     ws3.cell(row=1, column=2, value="购电时间段")
1107     ws3.cell(row=1, column=3, value="购电量")
1108     r, n_ev = 2, 0
1109     for k, s in enumerate(summaries):
1110         for j, e in enumerate(s["events"]):
1111             ws3.cell(row=r, column=1,
1112                     value=dates[REPORT_START + k] if j == 0 else None)
1113             ws3.cell(row=r, column=2, value=e["区间标签"])
1114             ws3.cell(row=r, column=3, value=round(e["购电量_kWh"], 6))
1115             r += 1
1116             n_ev += 1
1117
1118 # ---- 全天汇总（新增表，不改变原模板）
1119 if "全天汇总" in wb.sheetnames:
1120     del wb["全天汇总"]
1121 ws4 = wb.create_sheet("全天汇总")
1122 hdr = ["日期", "初始计划量_kWh", "最终常规量_kWh", "紧急购电量_kWh",
1123        "收到总电量_kWh", "计划费_元", "调整费_元", "紧急费_元",
1124        "合计费用_元", "弃电量_kWh", "0:00储电量_kWh", "24:00储电量_kWh",
1125        "调整提交次数"]
1126 for j, h in enumerate(hdr, start=1):
1127     ws4.cell(row=1, column=j, value=h)
1128 for k, s in enumerate(summaries):
1129     vals = [dates[REPORT_START + k], s["g_initial"], s["g_effective"],
1130            s["g_emg"], s["g_received"], s["cost_plan"], s["cost_adj"],
1131            s["cost_emg"], s["cost_total"], s["dump_kwh"], s["E0"],
1132            s["E144"], s["n_submit"]]
1133     for j, val in enumerate(vals, start=1):
1134         ws4.cell(row=2 + k, column=j,
1135                 value=round(val, 6) if isinstance(val, float) else val)
1136
1137 path.parent.mkdir(parents=True, exist_ok=True)
1138 wb.save(path)
1139 print(f" 计划购电量 / 调整购电量: {len(summaries)} 天 × {N} 段")
1140 print(f" 充放电量 : {len(summaries) * 6} 行")

```

```

1141     print(f"  紧急购电量: {n_ev} 个事件行")
1142
1143
1144 # ===== 主流程
1145 def combo_name(S: tuple) -> str:
1146     return " " if not S else "{" + ",".join(str(h) for h in S) + "}"
1147
1148
1149 def all_combos() -> list[tuple]:
1150     out = []
1151     for k in range(4):
1152         out.extend(combinations(ALL_S, k))
1153     return out
1154
1155
1156 MAIN_TAG = "{6,12,18}"
1157 CTRL_TAG = "对照: 沿用0:00预报版本"
1158 REFUND_TAG = "敏感性: 退款口径"
1159
1160
1161 def load_caches() -> dict[str, dict]:
1162     """读回 p3_cache/ 下每个策略的汇总结果 (--report-only 用)。"""
1163     if not CACHE_DIR.exists():
1164         raise FileNotFoundError(
1165             f"没有缓存目录 {CACHE_DIR}, 请先运行一次完整求解")
1166     out = {}
1167     for p in sorted(CACHE_DIR.glob("*.json")):
1168         c = json.loads(p.read_text(encoding="utf-8"))
1169         out[c["tag"]] = c
1170     return out
1171
1172
1173 def main_arrays_from_detail() -> tuple[np.ndarray, np.ndarray]:
1174     """从 p3_detail.csv 还原主策略逐日逐段的 g 与 x (供重建 13 表用)。
1175
1176     detail_frame3 按"日期、时段起"排序输出, reshape 前必须再排一次序,
1177     否则跨日回放中偶发的乱序会让整张表错行。
1178     """
1179     if not OUT_DETAIL3.exists():
1180         raise FileNotFoundError(f"缺少 {OUT_DETAIL3}, 无法重建 result3.xlsx")
1181     det = pd.read_csv(OUT_DETAIL3, encoding="utf-8-sig")
1182     det = det.sort_values(["日期", "时段起"], kind="stable")
1183     g0 = det["初始计划购电_kWh"].to_numpy(float).reshape(-1, N)
1184     xs = det["最终购电_kWh"].to_numpy(float).reshape(-1, N)
1185     return g0, xs
1186
1187
1188 def run_task(task: dict) -> dict:
1189     """一个完整策略: 载入数据 → 滚动标定 → 全年回放 → 汇总。
1190
1191     每个进程各自持有一份数据副本与独立的计划缓存; 主策略额外回传逐日记录,
1192     供写出 result3.xlsx 与明细文件使用。
1193     """
1194     from p1_microgrid import load_attach1
1195     price = load_attach1()["电价"].to_numpy(float)
1196     LOAD, PV, dates = load_attach2()
1197     fp = load_attach3()
1198     fo = Forecaster(LOAD, PV)
1199     _SIM.clear()
1200     _PLANO_CACHE.clear()
1201     setup(LOAD, PV, price, fp, fo)
1202
1203     # 标定必须用**该策略自己**的组合回放, 否则参数是按另一套信息集选的
1204     cal = CalConfig3(S=task["S"], **task["cal"])

```

```

1205 recs, cal_rows = run_strategy3(cal, task["S"], E_INIT,
1206                               task["use_new"], task["convention"])
1207 rep = [r for r in recs if REPORT_START <= r.n < REPORT_END]
1208 t = totals3(rep)
1209 out = {"tag": task["tag"], "S": task["S"], "total": t,
1210        "cal_rows": cal_rows, "records": rep if task["keep"] else None}
1211 if task["convention"] == "main":
1212     out["failures"] = check_all3(rep, price)
1213     out["vmetrics"] = validation_metrics3(rep, price)
1214
1215 # 汇总结果立刻落盘：后续的报告阶段只读这些文件，出任何错都不必重算
1216 # 四十多分钟的滚动标定。records 体积很大，单独走 p3_detail.csv。
1217 CACHE_DIR.mkdir(parents=True, exist_ok=True)
1218 cache = {
1219     "tag": task["tag"], "S": list(task["S"]),
1220     "use_new": task["use_new"], "convention": task["convention"],
1221     "total": t, "cal_rows": cal_rows,
1222     "failures": out.get("failures"), "vmetrics": out.get("vmetrics"),
1223     "combo_row": combo_row(task["tag"], t, task["S"]),
1224     # 逐日汇总只对主策略留档（论文的指定日期表由它生成），其余策略
1225     # 的体积不值得占用磁盘
1226     "summaries": ([day_summary3(r, price) for r in rep]
1227                   if task["keep"] else None),
1228 }
1229 tmp = CACHE_DIR / f"_{safe_tag(task['tag'])}.json.tmp"
1230 tmp.write_text(json.dumps(cache, ensure_ascii=False, default=float),
1231                encoding="utf-8")
1232 tmp.replace(CACHE_DIR / f"_{safe_tag(task['tag'])}.json")
1233 return out
1234
1235
1236 def _safe_tag(tag: str) -> str:
1237     """把 "{6,12}" 这类标签变成安全的文件名。"""
1238     return (tag.replace("{", "").replace("}", "").replace(",", "_")
1239            .replace(":", "").replace(" ", "") or "empty")
1240
1241
1242 def combo_row(tag: str, t: dict, S: tuple) -> dict:
1243     """组合比较表的一行：标签、使用到的发布时刻、以及全部汇总指标。"""
1244     return {"组合": tag, "使用预报": ",".join(str(h) for h in (0,) + tuple(S)),
1245            **t}
1246
1247
1248 def main() -> None:
1249     ap = argparse.ArgumentParser(description="2026 CUMCM C题 问题三 求解")
1250     ap.add_argument("--quick", action="store_true", help="只跑主策略，不写交付文件")
1251     ap.add_argument("--no-cal", action="store_true", help="跳过标定，用默认参数")
1252     ap.add_argument("--jobs", type=int, default=1, help="组合比较的并行进程数")
1253     ap.add_argument("--report-only", action="store_true",
1254                     help="不求解，直接读 06_支撑材料/p3_cache/ 重建全部交付文件")
1255     args = ap.parse_args()
1256
1257     t_start = time.time()
1258     from pl_microgrid import load_attach1
1259     price = load_attach1()["电价"].to_numpy(float)
1260     LOAD, PV, dates = load_attach2()
1261     fp = load_attach3()
1262     fo = Forecaster(LOAD, PV)
1263     setup(LOAD, PV, price, fp, fo)
1264     eps = _SIM["eps"]
1265
1266     print("=" * 78)
1267     print(f"附件 3 载入：{fp.long.shape[0]} 条预报 "
1268           f"= {N_DAY} 天 × {N_RELEASE} 次发布")

```

```

1269 print(f" 锚点约定示例: 2025-03-20 6:00 版本首端取自 "
1270       f"{fp.anchor_source[1, 78]}")
1271 print(f" 预报功率 {fp.F[:, :, 1:].min():.2f} ~ {fp.F[:, :, 1:].max():.2f} kW; "
1272       f"残差表可用天数 {int(np.sum(~np.isnan(eps[0]).all(axis=1)))}")
1273
1274 # ---- 预报精度: 各发布版本在正式区间内的净负荷预测误差
1275 prec = forecast_skill(Load, PV, fp, fo)
1276 print(f"\n各发布版本的净负荷预测精度 (正式区间 "
1277       f"{dates[REPORT_START]} 至 {dates[REPORT_END-1]}, "
1278       f"仅统计发布时刻之后的时段): ")
1279 print(f" {'版本':>6s} {'逐时段绝对偏差之和':>18s} {'全天电量偏差':>14s} "
1280       f"{'符号偏差':>12s} {'相对日净负荷':>12s}")
1281 for hr in RELEASE_HOURS:
1282     v = prec[hr]
1283     print(f" {hr:4d}:00 {v['逐时段绝对偏差之和_kWh每日']:18,.1f} "
1284           f"{v['全天电量偏差_kWh每日']:14,.1f} "
1285           f"{v['符号偏差_kWh每日']:12,.1f} "
1286           f"{v['相对日净负荷']*100:11.2f}%")
1287 print(f" 对照: 若改用问题二 的历史外推光伏预测, 全天电量偏差 "
1288       f"{prec['对照_历史外推']['全天电量偏差_kWh每日']:12,.1f} kWh/日")
1289
1290 if args.report_only:
1291     caches = load_caches()
1292     print(f"\n--report-only: 从 {CACHE_DIR.name}/ 读到 {len(caches)} 个策略")
1293     for c in caches.values():
1294         print(f" {c['tag']:>22s} {c['total']['合计费用_元']:12,.0f} 元")
1295     by_tag = caches
1296     report = None
1297 else:
1298     cal_kwargs = {} if not args.no_cal else {
1299         "alphas": (None,), "rhos": (1.0,), "lams": (0.60,)}
1300     tasks = [{"tag": combo_name(S), "S": S, "use_new": True,
1301              "convention": "main", "cal": cal_kwargs,
1302              "keep": combo_name(S) == MAIN_TAG}
1303              for S in all_combos()]
1304     tasks.append({"tag": CTRL_TAG, "S": tuple(ALL_S), "use_new": False,
1305                  "convention": "main", "cal": cal_kwargs, "keep": False})
1306     tasks.append({"tag": REFUND_TAG, "S": tuple(ALL_S), "use_new": True,
1307                  "convention": "refund", "cal": cal_kwargs, "keep": False})
1308 if args.quick:
1309     tasks = [t for t in tasks if t["tag"] == MAIN_TAG]
1310
1311 print(f"\n开始比较 {len(tasks)} 个策略 (并行进程数 {args.jobs}): ",
1312       flush=True)
1313 t0 = time.time()
1314 if args.jobs > 1:
1315     from concurrent.futures import ProcessPoolExecutor, as_completed
1316     # 每个策略含 8 次滚动标定 (每次 24 组参数 × 35 天回放), 单个
1317     # 策略就要跑十几分钟; 用 as_completed 逐个汇报才能判断进度。
1318     done = {}
1319     with ProcessPoolExecutor(max_workers=args.jobs) as ex:
1320         futs = {ex.submit(run_task, t): t for t in tasks}
1321         for fu in as_completed(futs):
1322             r = fu.result()
1323             done[r["tag"]] = r
1324             print(f" [{time.time()-t0:6.0f}s] {r['tag']} 完成"
1325                   f"(合计费用 {r['total']['合计费用_元']:12,.0f} 元)",
1326                   flush=True)
1327     results = [done[t["tag"]] for t in tasks]
1328 else:
1329     results = []
1330     for task in tasks:
1331         if task["tag"] != MAIN_TAG:
1332             _SIM.clear()

```

```

1333         _PLANO_CACHE.clear()
1334         setup(LOAD, PV, price, fp, fo)
1335         results.append(run_task(task))
1336         print(f"    {task['tag']} 完成 (累计 {time.time()-t0:.0f}s) ",
1337               flush=True)
1338
1339     by_tag = {r["tag"]: r for r in results}
1340     report = by_tag[MAIN_TAG]["records"]
1341
1342     main_res = by_tag[MAIN_TAG]
1343     main_tot = main_res["total"]
1344     failures, vmetrics = main_res["failures"], main_res["vmetrics"]
1345     cal_rows = main_res["cal_rows"]
1346
1347     print(f"\n主策略 {MAIN_TAG} (使用全部发布时刻): ")
1348     for k, v in main_tot.items():
1349         print(f"    {k}: {v:,.2f}" if isinstance(v, float) else f"    {k}: {v}")
1350     print("\n逐日校验 (各检查项的失败天数与实测最坏值): ")
1351     for k, v in failures.items():
1352         print(f"    {k}: {v} 天")
1353     for k, v in vmetrics.items():
1354         print(f"    {k}: {v:.3e}")
1355
1356     if args.quick:
1357         print(f"\n--quick 模式结束, 总用时 {time.time() - t_start:.0f}s")
1358         return
1359
1360     def save(tag: str) -> float:
1361         """组合比较表使用的一行; 缺了某个策略就报错, 不用假数据顶替。"""
1362         if tag not in by_tag:
1363             raise KeyError(f"缺少策略 {tag} 的结果, 无法生成完整比较表")
1364         return by_tag[tag]["total"]["合计费用_元"]
1365
1366     base = save(combo_name())
1367     print("\n八种预报组合: ")
1368     combo_rows = []
1369     for S in all_combos():
1370         t = by_tag[combo_name(S)]["total"]
1371         combo_rows.append(combo_row(combo_name(S), t, S))
1372         print(f"    {combo_name(S):>10s} 合计 {t['合计费用_元']/1e4:10.2f} 万元 "
1373               f"计划 {t['计划费_元']/1e4:10.2f} 调整 {t['调整费_元']/1e4:8.2f} "
1374               f"紧急 {t['紧急费_元']/1e4:9.2f} 期末储电 "
1375               f"{t['期末储电量_kWh']:8.1f}")
1376     print("\n各发布时刻在已有组合下的边际费用改善 (正值为加入后更省): ")
1377     for h in ALL_S:
1378         for S in all_combos():
1379             if h in S:
1380                 continue
1381             d = save(combo_name(S)) - save(combo_name(tuple(sorted(S + (h,)))))
1382             print(f"    Δ_{h}({combo_name(S) or ' '}) = {d/1e4:8.2f} 万元")
1383
1384     ctrl = by_tag[CTRL_TAG]["total"]
1385     rf_tot = by_tag[REFUND_TAG]["total"]
1386     info_gain = ctrl["合计费用_元"] - save(MAIN_TAG)
1387     reopt_gain = base - ctrl["合计费用_元"]
1388     print(f"\n{CTRL_TAG}: 合计 {ctrl['合计费用_元']/1e4:.2f} 万元")
1389     print(f"    重新优化收益 = {reopt_gain/1e4:.2f} 万元; "
1390           f"    新预报信息收益 = {info_gain/1e4:.2f} 万元")
1391     print(f"{REFUND_TAG}: 合计 {rf_tot['合计费用_元']/1e4:.2f} 万元; "
1392           f"    调整费 {rf_tot['调整费_元']/1e4:.2f} 万元")
1393
1394     # ---- 交付物
1395     if report is not None:
1396         _SIM.clear()

```

```

1397     _PLANO_CACHE.clear()
1398     setup(LOAD, PV, price, fp, fo)
1399     summaries = [day_summary3(r, price) for r in report]
1400 else:
1401     summaries = main_res["summaries"]
1402     if summaries is None:
1403         raise KeyError(f"缓存 {MAIN_TAG} 中没有逐日汇总")
1404     g0, xs = main_arrays_from_detail()
1405     if len(g0) != len(summaries):
1406         raise ValueError(f"p3_detail.csv 有 {len(g0)} 天, "
1407                          f"缓存有 {len(summaries)} 天")
1408     for k, s in enumerate(summaries):
1409         s["list_g0"], s["list_x"] = g0[k].tolist(), xs[k].tolist()
1410
1411 # 指定日期的三张表 (论文表 1—表 3 的数据源) 一并落进 JSON
1412 special = {s["date"]: s for s in summaries}
1413 special_dates = {}
1414 for d in SPECIAL_DATES:
1415     s = special[d]
1416     special_dates[d] = {
1417         "table1": s["table1"], "table2": s["table2"], "events": s["events"],
1418         "初始计划量_kWh": s["g_initial"], "最终常规量_kWh": s["g_effective"],
1419         "紧急购电量_kWh": s["g_emg"],
1420         "计划费_元": s["cost_plan"], "调整费_元": s["cost_adj"],
1421         "紧急费_元": s["cost_emg"], "合计费用_元": s["cost_total"],
1422         "弃电量_kWh": s["dump_kwh"], "期初储电量_kWh": s["EO"],
1423         "期末储电量_kWh": s["E144"], "调整提交次数": s["n_submit"],
1424     }
1425 pd.DataFrame(summaries).drop(
1426     columns=["table1", "table2", "events"]).to_csv(
1427     OUT_DAILY3, index=False, encoding="utf-8-sig")
1428 if report is not None:
1429     pd.concat([detail_frame3(r, price) for r in report],
1430               ignore_index=True).to_csv(OUT_DETAIL3, index=False,
1431                                          encoding="utf-8-sig")
1432 pd.DataFrame(combo_rows).to_csv(OUT_COMBOS3, index=False,
1433                                 encoding="utf-8-sig")
1434
1435 payload = {
1436     "口径": "主计费口径:  $J = \Sigma p \cdot g + \Sigma 1.5p(x-g)_+ + \Sigma 5p \cdot r$ ",
1437     "评价区间": f"{dates[REPORT_START]} 至 {dates[REPORT_END-1]}",
1438     "预报精度_净负荷MAE_kWh每日": prec,
1439     "主策略": {"组合": MAIN_TAG, **main_tot},
1440     "指定日期": special_dates,
1441     "逐日校验失败天数": failures,
1442     "校验数值口径": vmetrics,
1443     "八种组合": combo_rows,
1444     "对照_沿用0点预报版本": ctrl,
1445     "敏感性_退款口径": rf_tot,
1446     "标定记录": cal_rows,
1447 }
1448 OUT_JSON3.write_text(json.dumps(payload, ensure_ascii=False, indent=2,
1449                                 default=float), encoding="utf-8")
1450 # 工作簿最后写: 它依赖 openpyxl, 是最容易出错的一步, 放在 JSON 之后
1451 # 可以保证即使写表失败, 论文要用的全部数字也已经落盘。
1452 write_result3(summaries, report, dates, OUT_XLSX3)
1453 print(f"\n写入 {OUT_XLSX3.name} / {OUT_JSON3.name} / "
1454       f"{OUT_DAILY3.name} / {OUT_DETAIL3.name} / {OUT_COMBOS3.name}")
1455 print(f"总用时 {time.time() - t_start:.0f}s")
1456
1457 if __name__ == "__main__":
1458     main()
1459

```

(2) p3_tables.py: 论文表格生成

```
1 # -*- coding: utf-8 -*-
2 r"""
3 2026 CUMCM C 题 问题三 —— 由求解结果直接生成论文表格片段。
4
5 写出 05_论文/CUMCMThesis/p3_tables.tex, 由 main.tex 以 \input 引入,
6 使论文中的每个数字都与 result3/p3_results.json 严格同源, 避免手工转录错误。
7
8 运行: python3 03_代码/p3_tables.py (需先运行 p3_microgrid.py)
9 """
10
11 from __future__ import annotations
12
13 import json
14 import sys
15 from pathlib import Path
16
17 import numpy as np
18 import pandas as pd
19
20 sys.path.insert(0, str(Path(__file__).resolve().parent))
21 from p1_microgrid import N # noqa: E402
22
23 ROOT = Path(__file__).resolve().parents[1]
24 SUP = ROOT / "06_支撑材料"
25 OUT_TEX = ROOT / "05_论文" / "CUMCMThesis" / "p3_tables.tex"
26 # TEX 规范第四条: 正文(含摘要页)不超过 30 页, 附录页数不限。故把"支撑性"
27 # 的两张表放进附录: 预报精度表与七项校验表。它们只是论据, 正文用一句话给出
28 # 结论并 \ref 过去即可。
29 #
30 # 注意\textbf{不能}移到附录的是什么: 题目问题二、问题三都明确要求"在论文中按
31 # 表 1、表 2 和表 3 的格式给出表 3 中指定日期的结果", 即 tab_buy (表 1 格式)、
32 # tab_storage/tab_storage-dec (表 2 格式) 与 tab_emg (表 3 格式) 必须留在正文;
33 # tab_combo 是"是否需要引入其他时刻预报"这一提问的直接答案, 也必须留在正文。
34 OUT_TEX_TAIL = ROOT / "05_论文" / "CUMCMThesis" / "p3_tables_appendix.tex"
35
36 SPECIAL = ["2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21"]
37 DISP = {"2025-03-20": "2025.3.20", "2025-06-21": "2025.6.21",
38         "2025-09-23": "2025.9.23", "2025-12-21": "2025.12.21"}
39 BLOCK_LAB = ["0:00-4:00", "4:00-8:00", "8:00-12:00", "12:00-16:00",
40             "16:00-20:00", "20:00-24:00"]
41
42 SIX = ["10:00-10:10", "12:00-12:10", "14:00-14:10",
43       "16:00-16:10", "18:00-18:10", "20:00-20:10"]
44
45
46 def fmt(v: float, nd: int = 2) -> str:
47     return f"{v:.{nd}f}"
48
49
50 def texlab(s: str) -> str:
51     return s.replace(":", "{:}")
52
53
54 def signed(pct: float) -> str:
55     return f"{pct:+.2f}\\%".replace("+-", "-")
56
57
58 # ----- 表: 预报精度
59 def tab_skill(res: dict) -> str:
60     """表: 各发布版本的净负荷预测误差, 以及问题二历史外推的对照。"""
61     prec = res["预报精度_净负荷MAE_kWh每日"]
62     lines = [
```

```

63     r"\begin{table}[htbp]",
64     r" \centering",
65     r" \caption{各发布版本在 2025 年 2 月 1 日--12 月 31 日的净负荷预测"
66     r"误差（只统计发布时刻之后的时段）}",
67     r" \label{tab:p3-skill}",
68     r" \footnotesize",
69     r" \begin{tabular}{lrrrrr}",
70     r" \toprule",
71     r" 预报版本 & 逐时段绝对偏差之和/kWh & 全天电量偏差/kWh & "
72     r"符号偏差/kWh & 相对日净负荷 \\",
73     r" \midrule",
74 ]
75 for hr in (0, 6, 12, 18):
76     v = prec[str(hr)] if str(hr) in prec else prec[hr]
77     lines.append(
78         f" {hr}{{:}}00 发布 & {fmt(v['逐时段绝对偏差之和_kWh每日'], 1)}"
79         f" & {fmt(v['全天电量偏差_kWh每日'], 1)}"
80         f" & {v['符号偏差_kWh每日']:+.1f}"
81         f" & {v['相对日净负荷']*100:.2f}\\\% \\\\"
82     h = prec["对照_历史外推"]
83     lines += [
84         r" \midrule",
85         f" 问题二历史外推对照 & "
86         f"{fmt(h['逐时段绝对偏差之和_kWh每日'], 1)}"
87         f" & {fmt(h['全天电量偏差_kWh每日'], 1)}"
88         f" & {h['符号偏差_kWh每日']:+.1f}"
89         f" & {h['相对日净负荷']*100:.2f}\\\% \\\\",
90         r" \bottomrule",
91         r" \end{tabular}",
92         r" \par\vspace{2pt}",
93         r" \begin{minipage}{.92\textwidth}\footnotesize",
94         r" 注：“逐时段绝对偏差之和”为  $\sum_t |N_t - \widehat{N}_t|$ ，衡量十分钟"
95         r"尺度的错位；“全天电量偏差”为  $|\sum_t (N_t - \widehat{N}_t)|$ ，衡量日总量"
96         r"口径，二者不可混用。符号偏差为正表示净负荷被\textbf{低估}。",
97         r" \end{minipage}",
98         r"\end{table}",
99         "",
100     ]
101     return "\n".join(lines)
102
103
104 # ----- 表 1: 购电结果
105 def tab_buy(res: dict) -> str:
106     """论文表 1: 六个指定时段的初始计划、最终购电与全天费用分解。"""
107     sd = res["指定日期"]
108     lines = [
109         r"\begin{table}[htbp]",
110         r" \centering",
111         r" \caption{指定日期六个时段的初始计划购电量  $g^0_t$ 、最终生效购电量 "
112         r" $x_t$ 、正式调整量与紧急购电量（单位：kWh）}",
113         r" \label{tab:p3-buy}",
114         r" \scriptsize",
115         r" \begin{tabular}{llrrrr}",
116         r" \toprule",
117         r" 日期 & 时段 & 初始计划  $g^0_t$  & 最终生效  $x_t$  & 调整量 "
118         r"& 紧急购电  $r_t$  \\",
119         r" \midrule",
120     ]
121     for d in SPECIAL:
122         t1 = {r["时间段"]: r for r in sd[d]["table1"]}
123         lines.append(
124             f" \\\multirow{{6}}{{*}}{{{DISP[d]}}}"
125             f" & {\texlab(SIX[0])} & {fmt(t1[SIX[0]]['计划购电量_kWh'])}"
126             f" & {fmt(t1[SIX[0]]['调整购电量_kWh'])}"

```

```

127         f" & {fmt(t1[SIX[0]]['增量_kWh'])}"
128         f" & {fmt(t1[SIX[0]]['紧急购电量_kWh'])} \\\\"
129     for k in SIX[1:]:
130         lines.append(
131             f" & {texlab(k)} & {fmt(t1[k]['计划购电量_kWh'])}"
132             f" & {fmt(t1[k]['调整购电量_kWh'])}"
133             f" & {fmt(t1[k]['增量_kWh'])}"
134             f" & {fmt(t1[k]['紧急购电量_kWh'])} \\\\"
135         lines.append(r" \midrule")
136     if lines[-1] == r" \midrule":
137         lines.pop()
138     lines += [
139         r" \bottomrule",
140         r" \end{tabular}",
141         r" \par\vspace{2pt}",
142         r" \begin{minipage}{.92\textwidth}\footnotesize",
143         r" 续表：全天费用分解（单位：元）。计划费按  $g^0_t$  全额收取，"
144         r" 调整费按  $1.5p_t(x_t - g^0_t)$  另计，二者与紧急费之和即当日实际总费用。",
145         r" \end{minipage}",
146         r" \par\vspace{4pt}",
147         r" \begin{tabular}{lrrrrrr}",
148         r" \toprule",
149         r" 日期 & 初始计划量 & 最终常规量 & 紧急购电量 & 计划费 & 调整费 "
150         r"& 当日合计费用 \\",
151         r" \midrule",
152     ]
153     for d in SPECIAL:
154         s = sd[d]
155         lines.append(
156             f" {DISP[d]} & {fmt(s['初始计划量_kWh'], 1)}"
157             f" & {fmt(s['最终常规量_kWh'], 1)}"
158             f" & {fmt(s['紧急购电量_kWh'], 1)}"
159             f" & {fmt(s['计划费_元'], 0)} & {fmt(s['调整费_元'], 0)}"
160             f" & \textbf{{{fmt(s['合计费用_元'], 0)}}} \\\\"
161         lines += [
162             r" \bottomrule",
163             r" \end{tabular}",
164             r" \end{table}",
165             "",
166         ]
167     return "\n".join(lines)
168
169
170 # ----- 表 2：储能
171 def tab_storage(res: dict) -> str:
172     """论文表 2：指定日期的实际充放电电量与 0:00、24:00 储电量。"""
173     sd = res["指定日期"]
174     lines = [
175         r"\begin{table}[htbp]",
176         r" \centering",
177         r" \caption{储能设备在指定日期的实际充放电电量（按四小时汇总，"
178         r" 单位：kWh）}",
179         r" \label{tab:p3-storage}",
180         r" \footnotesize",
181         r" \begin{tabular}{lrrrrrr}",
182         r" \toprule",
183         r" 时段 & \multicolumn{2}{c}{2025.3.20} & \multicolumn{2}{c}{2025.6.21}"
184         r" & \multicolumn{2}{c}{2025.9.23} \\",
185         r" & 充电 & 放电 & 充电 & 放电 & 充电 & 放电 \\",
186         r" \midrule",
187     ]
188     # 按位置取，不按标签字符串取：求解器把末块标为 "20:00-0:00+1"（即次日 0 点，
189     # 与 24:00 是同一时刻），论文里统一显示为 "20:00-24:00"。两者语义相同，且
190     # 六个四小时块一一对应，故用位置索引最稳妥。

```

```

191 per = {d: list(sd[d]["table2"]) for d in SPECIAL}
192 for d in SPECIAL:
193     assert len(per[d]) == len(BLOCK_LAB), (d, len(per[d]))
194 for i, lab in enumerate(BLOCK_LAB):
195     cells = " & ".join(
196         f"{fmt(per[d][i]['充电量_kWh'])} & {fmt(per[d][i]['放电量_kWh'])}"
197         for d in SPECIAL[:3])
198     lines.append(f"    {texlab(lab)} & {cells} \\\\"
199 lines += [r"    \midrule"]
200 for lab, key in [(r"$0{:}00$ 储电量", "期初储电量_kWh"),
201                 (r"$24{:}00$ 储电量", "期末储电量_kWh")]:
202     cells = " & ".join(
203         f"\multicolumn{{2}}{{c}}{{{fmt(sd[d][key], 1)}}}"
204         for d in SPECIAL[:3])
205     lines.append(f"    {lab} & {cells} \\\\"
206 lines += [r"    \end{tabular}", r"\end{table}", ""]
207
208 d = "2025-12-21"
209 s = sd[d]
210 lines += [
211     r"\begin{table}[htbp]",
212     r"    \centering",
213     r"    \caption{2025 年 12 月 21 日储能设备的实际充放电电量与 "
214     r"$0{:}00$、$24{:}00$ 储电量}",
215     r"    \label{tab:p3-storage-dec}",
216     r"    \footnotesize",
217     r"    \begin{tabular}{lrr}",
218     r"        \toprule",
219     r"        时段 & 充电量/kWh & 放电量/kWh \\",
220     r"        \midrule",
221 ]
222 for i, lab in enumerate(BLOCK_LAB):
223     lines.append(f"    {texlab(lab)} & {fmt(per[d][i]['充电量_kWh'])}"
224                 f" & {fmt(per[d][i]['放电量_kWh'])} \\\\"
225 lines += [
226     r"        \midrule",
227     f"        $0{:}00$ 储电量/kWh & \multicolumn{{2}}{{c}}"
228     f"{{{fmt(s['期初储电量_kWh'], 1)}}} \\\\",
229     f"        $24{:}00$ 储电量/kWh & \multicolumn{{2}}{{c}}"
230     f"{{{fmt(s['期末储电量_kWh'], 1)}}} \\\\",
231     r"        \bottomrule",
232     r"    \end{tabular}",
233     r"\end{table}",
234     "",
235 ]
236 return "\n".join(lines)
237
238
239 # ----- 表 3: 紧急购电
240 def tab_emg(res: dict) -> str:
241     """论文表 3: 指定日期的紧急购电事件。"""
242     sd = res["指定日期"]
243     lines = [
244         r"\begin{table}[htbp]",
245         r"    \centering",
246         r"    \caption{微网在指定日期的紧急购电事件（问题三，按同日连续时段"
247         r"合并）}",
248         r"    \label{tab:p3-emg}",
249         r"    \footnotesize",
250         r"    \begin{tabular}{llrl}",
251         r"        \toprule",
252         r"        日期 & 紧急购电时间段 & 紧急购电量/kWh & 该日紧急费/元 \\",
253         r"        \midrule",
254 ]

```

```

255 for d in SPECIAL:
256     s = sd[d]
257     evs = s["events"]
258     if not evs:
259         lines.append(f"      {DISP[d]} & \\multicolumn{{2}}{{c}}{{无}}"
260                      f" & 0.00 \\")
261         continue
262     n = len(evs)
263     lines.append(
264         f"      \\multirow{{n}}{{*}}{{DISP[d]}}"
265         f" & {texlab(evs[0]['区间标签'])}"
266         f" & {fmt(evs[0]['购电量_kWh'])}"
267         f" & \\multirow{{n}}{{*}}{{fmt(s['紧急费_元'], 0)}} \\")
268     for e in evs[1:]:
269         lines.append(f"      & {texlab(e['区间标签'])}"
270                    f" & {fmt(e['购电量_kWh'])} & \\")
271     lines.append(r"      \midrule")
272 if lines[-1] == r"      \midrule":
273     lines.pop()
274 lines += [
275     r"      \bottomrule",
276     r"      \end{tabular}",
277     r"      \par\vspace{2pt}",
278     r"      \begin{minipage}{.92\textwidth}\footnotesize",
279     r"      注：紧急购电量逐十分钟按 $5p_{tr,t}$ 计费后再把同日连续时段合并"
280     r"成事件，合并不改变费用；合并后电量与逐段之和严格相等。",
281     r"      \end{minipage}",
282     r"      \end{table}",
283     "",
284 ]
285 return "\n".join(lines)
286
287 # ----- 表 4：组合比较
288 def tab_combo(res: dict) -> str:
289     """表：八种预报组合的费用对照，含增量收益与对照策略。"""
290     rows = res["八种组合"]
291     ctrl = res["对照_沿用0点预报版本"]
292     rf = res["敏感性_退款口径"]
293     base = next(r for r in rows if r["组合"] == " ")["合计费用_元"]
294     lines = [
295         r"\begin{table}[htbp]",
296         r"      \centering",
297         r"      \caption{八种预报组合在 2025 年 2 月 1 日--12 月 31 日的实际费用}"
298         r" (各组合单独标定参数，评价区间、负载模型、执行规则与计费口径完全一致)",
299         r"      \label{tab:p3-combo}",
300         r"      \footnotesize",
301         r"      \begin{tabular}{lrrrrrr}",
302         r"      \toprule",
303         r"      使用的预报时刻 & 初始计划费/元 & 调整费/元 & 紧急费/元 & "
304         r"& 合计费用/元 & 相对 $\varnothing$ & 期末储电量/kWh \\",
305         r"      \midrule",
306     ]
307     for r in rows:
308         tot = r["合计费用_元"]
309         rel = "----" if r["组合"] == " " else signed((tot - base) / base * 100)
310         name = "仅 0:00" if r["组合"] == " " else r["组合"]
311         lines.append(
312             f"      {name} & {fmt(r['计划费_元'], 0)}"
313             f" & {fmt(r['调整费_元'], 0)} & {fmt(r['紧急费_元'], 0)}"
314             f" & \\textbf{{{fmt(tot, 0)}}} & {rel}"
315             f" & {fmt(r['期末储电量_kWh'], 1)} \\")
316     lines += [
317         r"      \midrule",
318     ]

```

```

319     f"    对照：重新优化但仍用 0{:}}00 版本 & --- & --- & ---"
320     f" & \\textbf{{{fmt(ctrl['合计费用_元'], 0)}}} & "
321     f"{{signed((ctrl['合计费用_元'] - base) / base * 100)}}}"
322     f" & {{fmt(ctrl['期末储电量_kWh'], 1)}} \\",
323     r"    \midrule",
324     f"    敏感性：退款口径 ($x_t$ 可双向调整) & --- & --- & ---"
325     f" & \\textbf{{{fmt(rf['合计费用_元'], 0)}}} & --- & --- \\",
326     r"    \bottomrule",
327     r"  \end{tabular}",
328     r"  \par\vspace{2pt}",
329     r"  \begin{minipage}{{.92\textwidth}}\footnotesize",
330     r"    注：所有方案都使用 0:00 发布的预报；组合列出的时刻是\textbf{额外}"
331     r"使用的版本。费用按主计费口径计算，\"合计费用\"已经包含初始计划费，"
332     r"不能与计划费、调整费再相加。期末储电量用于说明各方案的跨日状态差异，"
333     r"不能只比较费用。未行为计费口径的敏感性检查：退款口径的目标函数另含"
334     r"调减收益，其总额与上表的\"相对\"列不可直接比较。",
335     r"  \end{minipage}",
336     r" \end{table}",
337     "",
338 ]
339 return "\n".join(lines)
340
341
342 # ----- 表：校验
343 def tab_check(res: dict) -> str:
344     """表：框架 8.2 节七项强制检查的实测最坏值。"""
345     v = res["校验数值口径"]
346     f = res["逐日校验失败天数"]
347     items = [
348         ("信息可用性", None, f["信息可用性"]),
349         ("时间转换", None, f["时间转换"]),
350         ("计划锁定", "主口径最大下调量_kWh", f["计划锁定"]),
351         ("费用核对", "调整费最大复算偏差_元", f["费用核对"]),
352         ("储能与供电", "供需平衡最大残差_kWh", f["储能与供电"]),
353         ("状态衔接", "状态累计最大绝对偏差_kWh", f["状态衔接"]),
354         ("结果一致性", None, f["结果一致性"]),
355     ]
356     lines = [
357         r"\begin{table}[htbp]",
358         r"  \centering",
359         r"  \caption{框架要求的七项检查在主策略全部 334 个交付日上的实测结果}",
360         r"  \label{tab:p3-check}",
361         r"  \footnotesize",
362         r"  \begin{tabular}{llr}",
363         r"    \toprule",
364         r"    检查项 & 实测最坏值 & 未通过天数 \\",
365         r"    \midrule",
366     ]
367     unit = {"主口径最大下调量_kWh": " kWh", "调整费最大复算偏差_元": " 元",
368            "供需平衡最大残差_kWh": " kWh", "状态累计最大绝对偏差_kWh": " kWh"}
369     for name, key, bad in items:
370         if key is None:
371             lines.append(f"    {name} & 结构上恒成立 & {bad} \\")
372         else:
373             lines.append(f"    {name} & ${v[key]:.2e}$"
374                          f"${unit[key]} & {bad} \\")
375     lines += [
376         r"    \bottomrule",
377         r"  \end{tabular}",
378         r"  \par\vspace{2pt}",
379         r"  \begin{minipage}{{.92\textwidth}}\footnotesize",
380         r"    注：另有储电量越界最大量 "
381         rf"${v['储电量越界最大量_kWh']:.2e}$~kWh、充放电越限最大量 "
382         rf"${v['充放电越限最大量_kWh']:.2e}$~kWh、同时充放电时段数 "

```

```

383         rf"${int(v['同时充放电时段数'])}$, 均并入“储能与供电”一项。",
384         r" \end{minipage}",
385         r"\end{table}",
386         "",
387     ]
388     return "\n".join(lines)
389
390
391 HEADER = [
392     "% 本文件由 03_代码/p3_tables.py 自动生成, 请勿手工修改。",
393     "% 数据源: 06_支撑材料/p3_results.json",
394     "",
395 ]
396
397
398 def main() -> None:
399     with open(SUP / "p3_results.json", encoding="utf-8") as fh:
400         res = json.load(fh)
401         if "--split" in sys.argv:
402             # 正文留题目要求的表 1/表 2/表 3 格式结果与组合比较;
403             # 只把预报精度表与七项校验表移到附录, 数据一条不少。
404             body = HEADER + [
405                 tab_buy(res),
406                 tab_storage(res),
407                 tab_emg(res),
408                 tab_combo(res),
409             ]
410             tail = HEADER + [tab_skill(res), tab_check(res)]
411             OUT_TEX.write_text("\n".join(body), encoding="utf-8")
412             OUT_TEX_TAIL.write_text("\n".join(tail), encoding="utf-8")
413             print(f"写入 {OUT_TEX} (正文: 表 1/2/3 格式结果 + 组合比较)")
414             print(f"写入 {OUT_TEX_TAIL} (附录: 预报精度 + 七项校验)")
415             return
416             parts = HEADER + [
417                 tab_skill(res),
418                 tab_buy(res),
419                 tab_storage(res),
420                 tab_emg(res),
421                 tab_combo(res),
422                 tab_check(res),
423             ]
424             OUT_TEX.write_text("\n".join(parts), encoding="utf-8")
425             print(f"写入 {OUT_TEX}")
426
427
428 if __name__ == "__main__":
429     main()

```

(3) p3_figures.py: 插图生成

```

1  # -*- coding: utf-8 -*-
2  r"""
3  2026 CUMCM C 题 问题三 —— 论文插图生成
4  输出到 04_图/, 300 dpi, 供 LaTeX 直接 \includegraphics 引用。
5
6  四张图对应框架 8.3 节:
7  图 1 同一目标时段的多版本光伏预报 (叠加事后实际值)
8  图 2 初始计划与最终购电曲线 (标出正式调整与紧急补购)
9  图 3 参考与实际储电量曲线 (标出 0:00/6:00/12:00/18:00 更新点)
10 图 4 八种预报组合的费用分解
11
12 数据来源 (先运行 03_代码/p3_microgrid.py 生成):

```

```

13 06_支撑材料/p3_detail.csv
14 06_支撑材料/p3_daily.csv
15 06_支撑材料/p3_results.json
16 运行: python3 03_代码/p3_figures.py
17 """
18
19 from __future__ import annotations
20
21 import json
22 import sys
23 from pathlib import Path
24
25 import numpy as np
26 import pandas as pd
27 import matplotlib.pyplot as plt
28 from matplotlib.patches import Patch
29
30 sys.path.insert(0, str(Path(__file__).resolve().parent))
31 from cumcm_plot import savefig, setup_plot # noqa: E402
32
33 sys.path.insert(0, str(Path(__file__).resolve().parent))
34 from p1_microgrid import N, E_MIN, E_MAX, TAU, fmt_time # noqa: E402
35 from p2_microgrid import load_attach2, Forecaster # noqa: E402
36
37 setup_plot()
38
39 ROOT = Path(__file__).resolve().parents[1]
40 SUP = ROOT / "06_支撑材料"
41 OUT = ROOT / "04_图"
42
43 C_VER = {0: "#7f8c8d", 6: "#2980b9", 12: "#e67e22", 18: "#16a085"}
44 C_ACT = "#c0392b"
45 C_GO = "#2980b9"
46 C_X = "#c0392b"
47 C_EMG = "#e67e22"
48 C_SOC = "#16a085"
49 C_REF = "#8e44ad"
50 C_PLANFEE = "#34495e"
51 C_ADJFEE = "#e67e22"
52 C_EMGFEE = "#c0392b"
53
54 SPECIAL = ["2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21"]
55 SPECIAL_TITLE = {"2025-03-20": "2025-03-20 (春分)",
56                  "2025-06-21": "2025-06-21 (夏至)",
57                  "2025-09-23": "2025-09-23 (秋分)",
58                  "2025-12-21": "2025-12-21 (冬至)"}
59
60
61 def hours() -> np.ndarray:
62     return np.arange(N) * 10 / 60
63
64
65 def load_inputs():
66     with open(SUP / "p3_results.json", encoding="utf-8") as f:
67         res = json.load(f)
68     det = pd.read_csv(SUP / "p3_detail.csv", encoding="utf-8-sig")
69     daily = pd.read_csv(SUP / "p3_daily.csv", encoding="utf-8-sig")
70     return res, det, daily
71
72
73 def panels():
74     fig, axes = plt.subplots(2, 2, figsize=(11.2, 4.8))
75     return fig, axes.ravel()
76

```

```

77 # ----- 图 1
78
79 def fig1_versions() -> None:
80     """图 1: 四个发布版本对同一天的光伏预报, 叠加该日实际光伏。"""
81     from p3_microgrid import load_attach3, forecast_energy_slots
82     from p2_microgrid import load_attach2 as _l2
83
84     fp = load_attach3()
85     LOAD, PV, dates = _l2()
86     idx = {d: i for i, d in enumerate(dates)}
87     t = hours()
88
89     fig, axes = panels()
90     for ax, d in zip(axes, SPECIAL):
91         n = idx[d]
92         act = PV[n] * TAU * 6.0 # 10 分钟电量 -> 等效平均功率 kW
93         ax.plot(t, act, color=C_ACT, lw=2.0, label="实际光伏", zorder=5)
94         for hi, hr in enumerate((0, 6, 12, 18)):
95             v = forecast_energy_slots(fp, hi, n) * 6.0
96             m = ~np.isnan(v)
97             ax.plot(t[m], v[m], color=C_VER[hr], lw=1.3, ls="--",
98                     label=f"{hr}:00 预报")
99         ax.set_xlim(0, 24)
100        ax.set_xticks(range(0, 25, 6))
101        ax.set_xlabel("时刻 / h")
102        ax.set_ylabel("光伏功率 / kW")
103        ax.set_title(SPECIAL_TITLE[d], fontsize=11)
104    axes[0].legend(fontsize=8, ncol=2, loc="upper left")
105    fig.suptitle("四个发布版本对同一天的光伏预报与实际值", fontsize=12, y=1.0)
106    savefig("p3_fig1_versions.png")
107
108 # ----- 图 2
109
110 def fig2_purchase(det: pd.DataFrame) -> None:
111     """图 2: 初始计划 g 与最终生效 x, 标出正式调整区间和紧急补购。"""
112     t = hours()
113     fig, axes = panels()
114     for ax, d in zip(axes, SPECIAL):
115         g = det[det["日期"] == d]
116         g0 = g["初始计划购电_kWh"].to_numpy()
117         x = g["最终购电_kWh"].to_numpy()
118         emg = g["紧急购电_kWh"].to_numpy()
119         ax.plot(t, g0, color=C_G0, lw=1.4, ls="--", label="初始计划 $g^0_t$")
120         ax.plot(t, x, color=C_X, lw=1.5, label="最终生效 $x_t$")
121         up = x > g0 + 1e-9
122         if up.any():
123             ax.fill_between(t, g0, x, where=up, color=C_X, alpha=0.28,
124                             step="mid", label="正式调整 (上调)")
125             for h in (6, 12, 18):
126                 if up[h * 6: h * 6 + 36].any():
127                     ax.axvline(h, color=C_X, lw=0.7, ls=":", alpha=0.6)
128         if (emg > 1e-6).any():
129             ax.fill_between(t, 0, emg, where=emg > 1e-6, color=C_EMG,
130                             alpha=0.85, step="mid", label="紧急购电 $r_t$")
131         ax.set_xlim(0, 24)
132         ax.set_xticks(range(0, 25, 6))
133         ax.set_xlabel("时刻 / h")
134         ax.set_ylabel("十分钟电量 / kWh")
135         ax.set_title(SPECIAL_TITLE[d], fontsize=11)
136     axes[0].legend(fontsize=8, loc="upper right")
137     fig.suptitle("初始计划购电与最终生效购电 (虚线为各发布时刻)", fontsize=12,
138                 y=1.0)
139     savefig("p3_fig2_purchase.png")
140

```

```

141
142 # ----- 图 3
143 def fig3_storage(det: pd.DataFrame) -> None:
144     """图 3: 实际储电量与最新参考轨迹, 标出四次更新时刻。"""
145     t = hours()
146     fig, axes = panels()
147     for ax, d in zip(axes, SPECIAL):
148         g = det[det["日期"] == d]
149         E0 = float(g["期初储电量_kWh"].iloc[0])
150         E = g["期末储电量_kWh"].to_numpy()
151         ref = g["参考储电量_kWh"].to_numpy()
152         ax.plot(t, ref, color=C_REF, lw=1.2, ls="--", label="参考轨迹  $\bar{E}_t$ ")
153         ax.plot(t, E, color=C_SOC, lw=1.6, label="实际储电量  $E_t$ ")
154         ax.axhline(E_MIN, color="gray", lw=0.8, ls=":")
155         ax.axhline(E_MAX, color="gray", lw=0.8, ls=":")
156         for h in (0, 6, 12, 18):
157             ax.axvline(h, color="k", lw=0.7, ls=":", alpha=0.5)
158         ax.set_xlim(0, 24)
159         ax.set_ylim(E_MIN - 250, E_MAX + 250)
160         ax.set_xticks(range(0, 25, 6))
161         ax.set_xlabel("时刻 / h")
162         ax.set_ylabel("储电量 / kWh")
163         ax.set_title(f"{SPECIAL_TITLE[d]} (0:00 储电 {E0:.0f} kWh)", fontsize=10)
164     axes[0].legend(fontsize=8, loc="lower left")
165     fig.suptitle("实际储电量与滚动参考轨迹 (竖虚线为 0/6/12/18 时更新点)",
166                 fontsize=12, y=1.0)
167     savefig("p3_fig3_storage.png")
168
169
170 # ----- 图 4
171 def fig4_combos(res: dict) -> None:
172     """图 4: 八种预报组合的费用分解与总额。"""
173     rows = res["八种组合"]
174     ctrl = res["对照_沿用0点预报版本"]
175     labels = [r["组合"] for r in rows] + ["对照"]
176     plan = np.array([r["计划费_元"] for r in rows] + [ctrl["计划费_元"]]) / 1e4
177     adj = np.array([r["调整费_元"] for r in rows] + [ctrl["调整费_元"]]) / 1e4
178     emg = np.array([r["紧急费_元"] for r in rows] + [ctrl["紧急费_元"]]) / 1e4
179     tot = plan + adj + emg
180
181     fig, ax = plt.subplots(figsize=(10.4, 3.45))
182     xp = np.arange(len(labels))
183     ax.bar(xp, plan, 0.62, color=C_PLANFEE, label="计划购电费 (含调整)")
184     ax.bar(xp, adj, 0.62, bottom=plan, color=C_ADJFEE, label="调整 (违约/追加) 费")
185     ax.bar(xp, emg, 0.62, bottom=plan + adj, color=C_EMGFEE, label="紧急购电费")
186     for i, v in enumerate(tot):
187         ax.annotate(f"{v:,.1f}", (i, v), ha="center", va="bottom", fontsize=8,
188                    xytext=(0, 2), textcoords="offset points")
189     ax.set_xticks(xp)
190     ax.set_xticklabels(labels, rotation=20, ha="right")
191     ax.set_ylabel("费用 / 万元")
192     ax.set_title("八种预报组合的实际总费用分解 (对照=重新优化但仍用 0:00 预报版本)",
193                 fontsize=11)
194     ax.legend(fontsize=9)
195     ax.axvline(7.5, color="gray", lw=0.8, ls="--")
196     savefig("p3_fig4_combos.png")
197
198 # 省下的费用不靠图读, 同时落一份数值表供正文引用
199 out = pd.DataFrame({
200     "组合": labels, "计划费_万元": plan, "调整费_万元": adj,
201     "紧急费_万元": emg, "合计_万元": tot,
202 })
203 out.to_csv(SUP / "p3_combo_fees.csv", index=False, encoding="utf-8-sig")
204 print(f" 另存 {SUP / 'p3_combo_fees.csv'}")

```

```

205
206
207 # ----- 附加图
208 def fig5_daily(daily: pd.DataFrame) -> None:
209     """图 5: 逐日的调整费、紧急费与储电量, 刻画收益的月份分布。"""
210     d = pd.to_datetime(daily["date"])
211     fig, axes = plt.subplots(3, 1, figsize=(11.0, 5.4), sharex=True)
212     axes[0].plot(d, daily["cost_adj"], color=C_ADJFEE, lw=0.9)
213     axes[0].set_ylabel("调整费 / 元")
214     axes[1].plot(d, daily["cost_emg"], color=C_EMGFEE, lw=0.9)
215     axes[1].set_ylabel("紧急费 / 元")
216     axes[2].plot(d, daily["E144"], color=C_SOC, lw=1.0)
217     axes[2].axhline(E_MIN, color="gray", lw=0.8, ls=":")
218     axes[2].axhline(E_MAX, color="gray", lw=0.8, ls=":")
219     axes[2].set_ylabel("24:00 储电量 / kWh")
220     axes[2].set_xlabel("日期")
221     fig.suptitle("主策略逐日的调整费、紧急费与日末储电量", fontsize=12, y=0.995)
222     fig.tight_layout()
223     savefig("p3_fig5_daily.png")
224
225
226 def main() -> None:
227     res, det, daily = load_inputs()
228     fig1_versions()
229     print(" 图 1 多版本光伏预报")
230     fig2_purchase(det)
231     print(" 图 2 初始/最终购电曲线")
232     fig3_storage(det)
233     print(" 图 3 参考/实际储电量")
234     fig4_combos(res)
235     print(" 图 4 八种组合费用分解")
236     fig5_daily(daily)
237     print(" 图 5 逐日费用与储电量")
238     print(f"输出目录: {OUT}")
239
240
241 if __name__ == "__main__":
242     main()

```

(4) p3_alignment_scan.py: 附件 3 时间口径判定

```

1 # -*- coding: utf-8 -*-
2 """附件 3 的时间口径判定: 整点瞬时功率, 还是小时平均功率?
3
4 附件 3 的每条记录给出"预报 1 小时"至"预报 24 小时"共 24 个功率值, 但没有说明
5 它对应目标整点的瞬时功率, 还是目标小时内的平均功率。这个口径决定了整点之间该
6 如何取值 (分段线性插值 vs 整小时内取常值), 必须先判定。
7
8 判定方法: 把两种口径分别铺成十分钟功率序列, 与附件 2 的实际光伏按不同时间平移
9 对齐, 比较正式区间 (2025-02-01 至 2025-12-31) 内的全天平均绝对偏差 (MAE, kW)。
10 偏差最小的口径即为正确口径; 同时报告"前移一小时"作为反例, 用以说明对齐方式确实
11 被区分开了。
12
13 输出: 06_支撑材料/p3_alignment_scan.json
14
15 论文正文引用的三个数字即由本脚本产生。运行:
16     python3 03_代码/p3_alignment_scan.py
17 """
18 from __future__ import annotations
19
20 import json
21 import sys

```

```

22 from pathlib import Path
23
24 import numpy as np
25
26 sys.path.insert(0, str(Path(__file__).resolve().parent))
27 import p3_microgrid as m # noqa: E402
28 from p1_microgrid import N, TAU, interval_label # noqa: E402
29 from p2_microgrid import N_DAY, REPORT_START, REPORT_END # noqa: E402
30
31 OUT = m.OUT_DIR / "p3_alignment_scan.json"
32 RELEASE_HOURS = (0, 6, 12, 18)
33
34
35 def _interp(A: np.ndarray, minutes: float) -> float:
36     """在整点功率序列 A[0..24] 上按分钟作分段线性插值。"""
37     x = minutes / 60.0
38     if x <= 0:
39         return float(A[0])
40     if x >= 24:
41         return float(A[24])
42     k = int(np.floor(x))
43     theta = x - k
44     return float((1 - theta) * A[k] + theta * A[k + 1])
45
46
47 def _step(A: np.ndarray, minutes: float) -> float:
48     """小时平均口径: A[j] 是第 j 个小时 ((j-1)*60 ~ j*60 分钟) 内的平均功率。"""
49     if minutes <= 0:
50         return float(A[0])
51     if minutes >= 24 * 60:
52         return float(A[24])
53     j = int(np.ceil(minutes / 60.0))
54     return float(A[min(max(j, 1), 24)])
55
56
57 def scan() -> dict:
58     LOAD, PV, dates = m.load_attach2()
59     fp = m.load_attach3()
60
61     # load_attach2 返回的 (LOAD, PV) 单位已经是 kW (求解器用 PV * TAU 换算成
62     # 十分钟电量), 故这里直接就是十分钟平均功率, 不要再除 TAU。
63     pv_kw = PV
64
65     variants = {
66         "整点对齐 (分段线性插值)": (_interp, 0),
67         "前移一小时": (_interp, -60),
68         "按小时平均 (整小时内取常值)": (_step, 0),
69     }
70     acc = {k: [] for k in variants}
71     per_release = {k: {h: [] for h in RELEASE_HOURS} for k in variants}
72
73     for n in range(REPORT_START, min(REPORT_END, N_DAY)):
74         for hi, rel_h in enumerate(RELEASE_HOURS):
75             A = fp.F[hi, n, :]
76             t0 = 36 * hi # 发布时刻之前的时段该版本不预测
77             for t in range(t0, N):
78                 ra = t * 10.0 - 60.0 * rel_h # 相对发布时刻的分钟数
79                 actual = pv_kw[n, t]
80                 if not np.isfinite(actual):
81                     continue
82                 for name, (fn, shift) in variants.items():
83                     if ra + shift < 0:
84                         continue # 平移后落到发布前, 无定义
85                     dev = abs(fn(A, ra + shift) - actual)

```

```

86         acc[name].append(dev)
87         per_release[name][rel_h].append(dev)
88
89     rep = {}
90     for name in variants:
91         rep[name] = {
92             "MAE_kW": float(np.mean(acc[name])),
93             "样本数": int(len(acc[name])),
94             "分版本MAE_kW": {
95                 f"{h}:00": round(float(np.mean(per_release[name][h])), 2)
96                 for h in RELEASE_HOURS},
97         }
98
99     best = min(rep, key=lambda k: rep[k]["MAE_kW"])
100     return {
101         "区间": f"{dates[REPORT_START]} 至 {dates[REPORT_END - 1]}",
102         "各口径MAE_kW": {k: round(v["MAE_kW"], 2) for k, v in rep.items()},
103         "分版本MAE_kW": {k: v["分版本MAE_kW"] for k, v in rep.items()},
104         "判定": best,
105         "明细": rep,
106     }
107
108
109 def main() -> None:
110     res = scan()
111     OUT.write_text(json.dumps(res, ensure_ascii=False, indent=2), encoding="utf-8")
112     print(f"区间: {res['区间']}")
113     print(f"{'口径':<28}{'MAE/kW':>10}")
114     for k, v in res["各口径MAE_kW"].items():
115         mark = "    <-- 最小, 判定为该口径" if k == res["判定"] else ""
116         print(f"{k:<28}{v:>10.2f}{mark}")
117     print("\n分版本 MAE/kW: ")
118     for k, d in res["分版本MAE_kW"].items():
119         print(f"    {k:<28}" + " ".join(f"{h}:00={v:.2f}" for h, v in d.items()))
120     print(f"\n判定: 附件 3 为{res['判定']}")
121     print(f"写入 {OUT}")
122
123
124 if __name__ == "__main__":
125     main()

```

G 问题三结果文件说明

结果按题目要求的 result3.xlsx 与支撑材料中的内部文件两套输出。

(1) result3.xlsx (按附件 5 指定格式)

- 工作表 Sheet1: 逐日汇总, 共 334 行 (2025-02-01 至 2025-12-31), 每行含日期、初始计划购电量、最终常规购电量、紧急购电量、实际收到总电量与三项费用;
- 工作表 Sheet2-Sheet5: 四个发布时刻的细化记录, 逐日给出该时刻是否提交调整、提交了哪些时段、调整前后的购电量;
- 每行的储电量记录为当天 24:00 的值, 与次日 0:00 的初值一致, 跨日连续、不要求日循环。

(2) 内部结果文件 (支撑材料 06_ 支撑材料/)

- p3_detail.csv: 逐时段明细 (334 × 144 = 48096 行), 含电价、预测负载、0:00 版光伏预报、净负荷预测、风险余量、风险净负荷、参考储电量、实际负载/光伏、初始计划购电量、最终购电量、上调量、下调量、实际充放电、紧急购电、弃电、期初期末储电量与三项时段费用;

- `p3_daily.csv`: 逐日汇总 (334 行), 含全天电量、三项费用、储电量、调整提交次数与当日生效参数;
- `p3_combinations.csv`: 8 种预报组合的全年汇总, 即表 12 前八行的原始记录;
- `p3_combo_fees.csv`: 在八种组合之外补入”重新优化但仍用 0:00 版本”的对照, 共 9 行的费用分解, 即表 12 与图 11 的费用口径原始记录; 对照与退款口径的完整逐项指标见 `p3_results.json`;
- `p3_results.json`: 正文全部汇总数字、七项检查结果与标定记录的机器可读版本。附表的 16–17 由 `p3_tables.py` 直接读取该文件与 `p3_detail.csv` 自动生成 (`p3_tables.tex`), **不做任何手工转录**;
- `p3_cache/`: 每个策略求解完成后立即落盘的单策略缓存, 使得”重新生成论文表格”不必再跑一次上万次 MILP 的滚动标定, 也保证了正文数字与首次求解结果严格一致。

H 支撑材料文件列表

按规范第五条，随论文提交的全部支撑材料为一个压缩包，其内容如下表所列。除题目要求的两类结果文件外，其余均为程序的中间与最终输出，**全部由源码自动生成**，不含任何人工填写的数据；问题三所用的四个脚本已在“完整源程序”一节全文列出，其余脚本的提交形式与之相同。

(1) 题目要求的结果文件

- `result1.xlsx`: 问题一要求的结果，含 144 行逐时段计划购电量与充放电量；
- `result2.xlsx`: 问题二要求的结果，按附件 5 的划分口径自动切分为表 1（计划购电量）、表 2（充放电量）、表 3（紧急购电量）三张工作表；
- `result3.xlsx`: 问题三要求的结果，含 Sheet1 逐日汇总与四个发布时刻的细化记录。

(2) 源程序（源码/，共 11 个脚本）

- `p1_microgrid.py`、`p2_microgrid.py`、`p3_microgrid.py`: 三个问题的主求解器；
- `p1_figures.py`、`p2_figures.py`、`p3_figures.py`: 正文插图生成；
- `p2_tables.py`、`p3_tables.py`: 论文表格生成；
- `p2_alphasweep.py`: 问题二的 α 灵敏度扫描；
- `p3_alignment_scan.py`: 附件 3 时间口径判定；
- `cumcm_plot.py`: 绘图样式与中文字体配置。

以上脚本在 Python 3.11 下只需 `numpy/scipy/pandas/openpyxl/matplotlib` 即可按依赖顺序直接运行，不依赖任何外部数据文件（输入全部取自题目附件），也不需要手工输入任何中间结果。

(3) 结果数据

- `p1_detail.csv`、`p1_results.json`: 问题一的逐时段明细与汇总指标；
- `p2_detail.csv`、`p2_daily.csv`、`p2_calibration.csv`、`p2_results.json`、`p2_alpha_sensitivity.json`: 问题二的逐时段明细、逐日汇总、滚动标定记录、汇总指标与灵敏度扫描结果；
- `p3_detail.csv`、`p3_daily.csv`、`p3_combinations.csv`、`p3_combo_fees.csv`、`p3_results.json`: 问题三的对文件，其中后两者即正文表 12 与图 11 的原始记录；
- `p3_cache/`: 问题三 10 个求解实例（8 种预报组合、1 个对照策略、1 个退款口径敏感性）各自的逐策略缓存，使得论文表格可在不重跑滚动标定的前提下复现；
- `p3_alignment_scan.json`: 附件 3 三种时间口径的扫描结果，对应正文 7.2 节的判定。

I 补充分析与讨论

本附录汇集正文因篇幅限制未能展开的四组支撑性分析。正文各节只保留结论，数据、逐项推理与全部对照口径均在本节完整给出。

9.1 问题一：基准对照与灵敏度分析

9.1.1 基准对照：储能与光伏的价值

为量化储能设备与光伏发电的经济价值，构造“光伏供电、缺口直接向外网购买、电池保持不动”的基准方案： $g_t^{\text{base}} = \max\{\ell_t - v_t, 0\}$, $c_t = d_t = 0$, $E_t \equiv 6000$ ，多余光伏弃用。该方案满足同样的日循环约束，因而是原问题的一个可行解，同一口径下最优费用必然不高于它，可据此提供独立的一致性检验。

表 18 不同方案的经济性对照

方案	全天购电量/kWh	全天购电费/元	弃光电量/kWh
无光伏无储能	111024.81	88319.36	—
无储能	61789.94	48052.05	6247.96
本模型（含储能）	59482.70	35126.95	0.00

由表 18 可见，配置储能设备后，全天购电费由 48052.05 元降至 35126.95 元，**节省 12925.10 元，降幅 26.90%**；较“无光伏无储能”方案节省 53192.41 元（60.23%）。更值得注意的是，含储能方案的**购电量反而更少**（59482.70 kWh 对比 61789.94 kWh）：因为储能将无储能方案中被迫弃掉的 6247.96 kWh 光伏盈余转移到了无光照时段使用，从而减少了对电网的净购电需求。储能的**价值**不仅在于“低买高卖”的价差套利，更在于**提升光伏本地消纳率**。优化费用 35126.95 元低于基准费用 48052.05 元，与上述可行性论证一致。

9.1.2 灵敏度分析

题目只给出一个“充放电效率为 90%”的参数，其在充电与放电两侧的分配方式存在解释空间：可以是单程效率（ $\eta_c = \eta_d = 0.9$ ，往返 0.81），也可以是往返效率（对称折算为 $\eta_c = \eta_d = \sqrt{0.9} \approx 0.9487$ ，往返 0.90）。此外，储能的日循环水平、充放电决策的时间粒度也均可能影响结论。为此设计六组对照口径：

表 19 不同建模口径下的求解结果对照

口径	全天购电量/kWh	全天购电费/元	弃光/kWh
主模型（往返 0.81, $E_0 = 6000$, 10 min）	59482.70	35126.95	0.00
放电无损（ $\eta_c = 0.90$, $\eta_d = 1.00$, 往返 0.90）	57580.98	33416.53	0.00
对称折算（ $\eta_c = \eta_d = \sqrt{0.90}$, 往返 0.90）	57526.24	33801.50	0.00
E_0 自由（仅要求 $E_0 = E_{144}$, 收敛至 8550）	59482.70	35118.60	0.00
粗化为 1 小时粒度	59924.95	36771.29	768.44
粗化为 4 小时粒度	61302.10	43648.06	2293.67

注：“往返”指 $\eta_c \eta_d$ ；除注明外均为 $\eta_c = \eta_d = 0.90$ 、 $E_0 = 6000$ kWh、10 分钟决策粒度。

对表 19 的分析可得以下结论：

(1) 效率约定影响有限，主要结论稳健。若将 90% 的效率解释为往返效率并对称折算（ $\eta_c = \eta_d = \sqrt{0.9}$ ），购电费为 33801.50 元，较主模型降低 3.77%；若极端地假设放电无损（ $\eta_c = 0.90$, $\eta_d = 1.00$ ），购电费为 33416.53 元，降低 4.87%。两者构成单程/往返两种解释下的费用区间 [33416.53, 35126.95] 元，跨度不足 5%。原因是放电效率越低，储能放电时需额外多消耗的储存电量越多，越直接地削弱峰谷套利收益。**两种口径下“谷充峰放”的策略结构与零弃光结论均不变**，说明主模型对效率参数的分配方式不敏感。

(2) 给定的初始储电量接近最优循环水平。若解除 $E_0 = 6000$ kWh 的固定约束、仅要求日循环 $E_0 = E_{144}$ ，模型自动收敛到 $E_0 = 8550$ kWh，购电费仅下降 8.35 元（0.024%）。这说明附录 1 给定的 6000 kWh 已非常接近成本最优的日循环工作点，无需大幅调整。

该结果也验证了主模型采用 $E_0 = 6000 \text{ kWh}$ 的合理性：即使放开，最优解也不会显著偏离；同时它从反面说明，把初始储电量当作可自由优化的资源只会带来不足万分之三的收益，不值得为此偏离附录给定值。

(3) 决策粒度具有实质经济价值。这是灵敏度分析中最显著的效应。当充放电决策由 10 分钟粗化为 1 小时时，购电费上升 4.68% 并产生 768.44 kWh 弃光；粗化为 4 小时时，购电费上升 24.26%（达 43648.06 元），弃光量进一步增至 2293.67 kWh。原因是粗粒度下储能功率在各时段内被迫恒定，无法跟踪电价与光伏的快速波动，峰谷套利的时机被显著钝化，光伏盈余与储能充电能力也难以精确匹配，只能弃光。因此**采用 10 分钟的精细决策粒度是必要的**，这也从侧面说明题目要求以 10 分钟为间隔给出购电量是合理的。

9.2 问题二：弃用电量的成因

全年弃用电量达 2704833.51 kWh，约占同期光伏发电量 19068890.0 kWh 的 14.2%。这一数字需要正确解读：题目未对弃电设置惩罚或收益，因此**弃电不会带来任何额外扣款**；但它仍是日前计划与实时运行之间偏差的**指示器**。

按逐时段口径分解全年弃电，可以看到它来自**两个性质不同的来源**。70.0% 的弃电（1892666.5 kWh）发生在实际光伏大于实际负载的时段，属于光伏富余超出储能吸纳能力——储能受 5000 kW 功率上限与 10800 kWh 容量上限双重约束，在光伏最集中的时段无法全额吸收；风险修正又使日前计划量系统性偏高，与高于预期的实际光伏叠加后进一步加剧了这一部分。另 30.0% 的弃电（812167.0 kWh）发生在**光伏并无富余**的时段，其中 173360.7 kWh 甚至出现在光伏出力为零的深夜与凌晨——这部分电量不可能来自光伏，只能来自**已按计划电价付费、但因实际负载低于预期而用不掉的计划购电量**。按各时段电价核算，这部分被弃掉的已购电量当初共付出 544354 元，约占全年计划购电费的 4.07%。换言之，弃电并非严格“零成本”：其中约三成是“按 5 倍电价买保险”所预付的保费中未能出险的部分。

从经济学角度看，这一取舍是**经过权衡的最优选择**：减少弃电最直接的办法是降低计划购电量，但那会同时削弱对紧急购电的防护。本文在固定 $\rho = 1$ 、 $\lambda = 0$ 下重跑了 $\alpha \in \{0.50, 0.70, 0.80, 0.90\}$ 四点的敏感性验证（逐点见支撑材料 p2_alpha_sensitivity.json）： α 由 0.80 降至 0.50 时，弃电量从 2662.5 MWh 降至 1500.1 MWh，但紧急购电量由 179.8 MWh 猛增至 642.9 MWh，**总费用反而从 1407.2 万元升至 1513.1 万元**；反向升至 0.90 时，紧急购电量降至 86.0 MWh，弃电量却升至 3423.4 MWh，总费用回升至 1425.7 万元； $\alpha = 0.70$ 为 1414.1 万元。四点中 $\alpha = 0.80$ 取到最小值且两侧点均高于它，“以适量弃电换取紧急购电防护”的取舍是可验证的。（这四点均固定 $\rho = 1$ ；主策略因滚动重标定多数时段取到 $\rho = 0$ ，全年 1402.2 万元还略低于该网格最优点。）

图 6 给出紧急购电量的季节分布与逐日费用构成，其中还暴露了一个对风险管理很有意义的事实：**紧急购电的负担在时间上高度集中**。334 天中有 160 天发生紧急购电，但其中位日仅 150.6 kWh；而最严重的 6 月 1 日单日紧急购电 10568.76 kWh、费用 64812.44 元，**一天就占去全年紧急购电费的 9.8%**，其后紧接的 6 月 2 日（7093.72 kWh、42379.60 元）与 12 月 2 日、7 月 6 日同样显著高出常态。这说明全年紧急购电费并非由 160 个“小缺口”平摊，而是由少数几个**连续多日的高压时段**主导——6 月初的连日高负荷恰逢光伏出力波动，使储能在数日内持续被压至下限，缺口只能在 5 倍电价下补足。

这一分布形态具有直接的运营含义：若把有限的额外储能容量或预测投入用于降低**极端日**的缺口，其边际收益将远高于用于改善 150 kWh 级别的日常误差。

9.3 问题二：模型的边界

需要明确本文模型的适用边界。本文的“风险余量 + 日前 MILP + 因果执行规则”是一条便于实现和解释的**近似策略**，其优劣完全由按时间顺序的实际运行回放判定，并不

等同于已求出所有随机控制策略中的全局最优解。当 $\lambda = 0$ 时，日前模型的目标中不含任何对未来的价值评估，因此它不会主动为次日保留电量，其合理性由跨日连续状态的自然约束保证，而非由模型内的随机优化保证。更严格的改进方向是把执行策略显式建模为非预见性约束下的场景优化（所有场景共用同一份 0:00 购电计划，且观测历史相同的场景必须作出相同动作），或采用因果滚动优化，每步只用当前观测值重排后续储能轨迹。就本题给定的数据规模与 5000 kW 功率上限而言，现行方案已经取得了较“无储能”基准 32.34% 的费用节约，是否值得进一步引入上述模型，应依据其在历史回放中的实际改善与计算代价决定。

9.4 问题三：模型的边界

第一，交付块策略是一种结算口径的选择，不是题目的唯一解。本文限定每个六小时交付块只在块首调整一次，从而让每个时段只与自己的 g_t^0 结算一次。若允许在 6:00、12:00 反复修订同一个傍晚时段，就必须另行声明是“最终净偏差结算一次”还是“逐次变更分别收费”，两者会得到不同的费用。本文的结论首先评价的是这一基础策略。

第二，本文求的是可实现的滚动近似策略，而不是未知未来下的全局最优解。每个决策时点只使用当时已经发布的信息，没有对未来尚未发布的预报作多阶段随机优化，因此 13658321 元（对照）与 13457698 元（主策略）都是**相对于信息集可达到的费用**，而不是全知情情形下的下界。

第三，参数标定是窗口内的近视最优。网格搜索的目标是过去 35 天的实际总费用，它并不保证全年最优；这是“不许用未来数据选参”这一硬约束的必然代价。因此表中各组合的费用差异应当理解为**同一规则下的可比结果**，而不是各组合的最优可能值。

第四，计费口径的歧义会影响结论的方向。第 7.3 节已说明，主口径下调减没有费用优势，而退款口径下调减每度净省 $0.5p$ ，最优解会落在 $[Q_{0.7}, Q_{0.9}]$ 的“不必调整区间”内，且调减行为会真实出现。本文把退款口径作为独立的敏感性检查一并报告（表 12 末行），供按另一种理解读题的读者参考；但不应把退款当作题目已明确给出的条件。

第五，时段粒度与记账时点。所有结果建立在 10 分钟离散记账、且每段实际负载与光伏在该段结束时即可观测的假设上；实际运行中若观测与调度存在滞后，需要在执行层加入相应的安全裕量。